Olla衛門

コード掲載・技術選定の詳細解説

ソースコードの掲載に加えて、行範囲ごとの動作を解説
技術選定の理由と、各処理の役割を対応付けた資料

この版で追加した内容

- すべての掲載ソースに行範囲ごとの動作説明を追加
- 関数やReact handlerごとに処理の目的を説明
- 大きな画面コンポーネントはUI領域ごとにも解説
- ハッシュ表記は掲載せず、SVGの個別コードも省略
- 目次リンクから説明・コードの箇所へ移動可能

目次へ

目次（1/2）

項番または見出しをクリックすると該当箇所へ移動します。

01 技術選定 …… 4
02 Reactを選ぶ理由 …… 5
03 Next.js App Routerを選ぶ理由 …… 6
04 TypeScriptとTailwind CSS …… 7
05 Ollamaとサーバー通信 …… 8
06 添付ファイル・コード生成 …… 9
07 ローカルファイル機能 …… 10
08 SVGファイルとは？ …… 11
09 収録範囲 …… 12
10 実行時の流れ …… 13
11 ソースコード: app/page.tsx …… 19
12 ソースコード: app/layout.tsx …… 51
13 ソースコード: app/globals.css …… 53
14 ソースコード: app/api/chat/route.ts …… 55
15 ソースコード: app/api/workspace/route.ts …… 61
16 ソースコード: components/CodeViewer.tsx …… 66
17 ソースコード: components/CodingWorkspace.tsx …… 70
18 ソースコード: components/DiffModal.tsx …… 77
19 ソースコード: components/LivePreview.tsx …… 82
20 ソースコード: components/QuickActions.tsx …… 85
21 ソースコード: components/WorkspaceExplorer.tsx …… 87
22 ソースコード: lib/codeExtractor.ts …… 96
23 ソースコード: lib/localFileHelper.ts …… 103
24 ソースコード: lib/prompts.ts …… 109
25 ソースコード: lib/zipExporter.ts …… 123
26 ソースコード: types/chat.ts …… 125
27 ソースコード: package.json …… 128
28 ソースコード: tsconfig.json …… 130
29 ソースコード: next.config.ts …… 132
30 ソースコード: postcss.config.mjs …… 134
31 ソースコード: eslint.config.mjs …… 136
32 ソースコード: .gitignore …… 138
33 ソースコード: next-env.d.ts …… 141
34 ソースコード: generate-tech-stack-pdf.js …… 143
35 ソースコード: README.md …… 149
36 ソースコード: AGENTS.md …… 152

目次（2/2）

項番または見出しをクリックすると該当箇所へ移動します。

37　ソースコード: CLAUDE.md　……　154
38　ソースコード: LICENSE　……　156
39　対象外ファイル　……　157
40　まとめ　……　158

技術選定

React、Next.js App Router、TypeScript、Tailwind CSS 4、Ollama、undici、文書処理ライブラリー、ブラウザーAPI、Node.js fs/pathを採用した理由を、実際の実装箇所に結び付けます。

目次へ

Reactを選ぶ理由

React stateで会話・入力・選択モデル・読込状態を持ち、操作のたびに画面を再描画できます。app/page.tsxからpropsを使って子部品へデータとcallbackを渡します。

Next.js App Routerを選ぶ理由

ファイル配置から画面URLとroute handlerを定められます。app/page.tsxは画面、app/api配下のroute.tsはサーバーAPIを担い、一つのTypeScriptプロジェクトにまとめます。

TypeScriptとTailwind CSS

TypeScriptのinterface/typeはメッセージ等の共有契約を示します。TailwindはJSXに近い場所でレイアウト・余白・状態別の見た目を組み立て、globals.cssに全体設定を置きます。

目次へ

Ollamaとサーバー通信

Ollamaをlocalhostの推論先として使います。undiciは長い生成時間に合わせたtimeoutとAbortSignalを適用し、利用者の中断を下流通信へ伝えます。

添付ファイル・コード生成

PDF.js、Mammoth、SheetJS、JSZipなどで文書から文字を抽出します。codeExtractorは回答中のコードblockをCodeFileへ変え、previewとZIPへ渡します。

ローカルファイル機能

ブラウザーが直接fsを使うのではなく、Next.jsサーバー側のAPIがローカルファイルを読みます。件数・サイズ上限があり、公開環境にする場合はアクセス制御が必要です。

目次へ

SVGファイルとは？

SVGは図形・線・文字などの見た目をXML形式のテキストで記述する画像形式です。拡大しても輪郭が滑らかで、コードとして色や形を調整できます。public内のSVGは静的ファイルとして配信され、画面のアイコンやロゴに使われます。この資料ではSVGの一般的な役割を説明し、個々のSVGソースコードは掲載しません。

目次へ

収録範囲

追跡対象のテキストソース28ファイルを掲載します。個別SVGコード、lockfile、既存PDF/ODG、バイナリー、依存物、生成物はコード付録の対象外です。

実行時の流れ

画面表示、入力・添付処理、APIへの送信、サーバー側のローカル参照、Ollama応答、React state更新、生成コードの編集・preview・保存の順で処理されます。

ファイル解説: app/page.tsx

添付形式の判定、ファイル読込、会話state初期化。

1〜20行｜Client境界とimport
先頭のuse clientにより、このページはブラウザー上でReact state、イベント、FileReader、localStorageを利用するクライアントコンポーネントになります。続くimportは画面部品、会話データ型、システム指示、回答末尾の処理、コード抽出・競合解決を結び、後半のChatAppがそれらを一つの会話体験にまとめるための依存関係です。

21〜40行｜添付制限と許可拡張子
添付数を5件、単一ファイルを2MBに制限し、file inputのacceptにも扱うテキスト・ソース・文書・画像形式を列挙しています。下の拡張子SetはMIME情報が不足する環境でも種類を判定するための許可リストです。選択UIの候補と実際の検査条件を分けているので、ファイル追加時にも同じ境界を適用できます。

41〜74行｜拡張子/MIMEとFileReader
getFileExtensionは最後のドットより後ろを小文字で返し、拡張子なしの名前も扱います。detectFileKindはまずブラウザーのMIME typeを使い、判断できないときに拡張子Setへフォールバックしてtext/imageを分類し、対象外ならnullを返します。FileReaderの2関数は文書用文字列と画像用data URLのbase64本体を別々に読み、読込失敗をファイル名付きErrorとして呼出元へ伝えます。

75〜111行｜PDF・DOCX・Excel抽出
readDocumentAsTextはファイルをArrayBuffer化し、拡張子に応じて抽出器を切り替えます。PDFはPDF.jsでページを順番に読み、ページ番号を残したテキストを結合しますが、画像だけのPDFをOCRする処理ではありません。DOCXはMammothのraw text、Excelは全シートをシート名付きCSVに変換するため、モデルには書式より内容を簡潔なテキストとして渡します。

112〜163行｜PPTX抽出とfallback
PPTXはZIP形式の内部からslide番号付きXMLだけを集め、番号順に並べて各slideを解析します。DOMParserでa:t要素の文字を連結し、空のslideは結果から外してスライド番号を付けます。PDF/DOCX/Excel以外の通常テキスト形式は最後のreadFileAsTextへ進むため、各形式の専用抽出と一般テキストの読み込みを同じ入口に統合しています。

164〜186行｜ChatApp state
ChatAppでは履歴をtopic名からMessage配列へ引ける辞書として持ち、空の初期topicにはsystem promptを設定します。入力文字列、添付一覧とエラー、drag中、モデル、通信中、現在topic、sidebar/workspaceの開閉、コピー状態を個別stateに分け、画面操作に対応した再描画を行います。workspaceOverridesは生成コードの編集内容を元コードとは別に保持し、再描画後も残します。

187〜194行｜DOM refとmodel一覧
DOM要素はfile input・textarea・model selectのrefから直接操作し、AbortControllerは送信中のリクエストを停止するために保持します。topicのmodeからcoding用か全modelかを選び、現在のmodel idに対応する表示名を導出します。表示に影響する選択はReact state、DOM操作・通信制御はrefで管理しています。

195〜237行｜model select幅調整
選択中model名が幅からはみ出さないよう、CanvasのmeasureTextで文字幅を計測し、selectのcomputed styleからfont、左右padding、borderを取得します。利用可能幅を超える場合は16pxを比率で縮小し、最小7pxに抑えて、文字列が短い場合は標準サイズへ戻します。初回とmodel名変更時に計算し、ResizeObserverで要素幅の変更にも追従、cleanupで監視を解除します。同じstate値なら更新せず不要な再描画も避けます。

238〜297行｜localStorage初期復元
最初のeffectはlocalStorageから会話履歴とtopic別modeを別々に読み、JSONを復元します。保存値がないときは初期値を使い、system promptだけの空topicがあればそこを起動先に選び、なければ重複しない新規topicを作ります。復元時の例外はconsoleへ記録して標準topicへ戻し、読み込み結果をstateへ反映します。saveHistoryは画像base64を保存対象から除外して容量増大を抑え、本文・表示本文・closing・参照file情報とmodeをlocalStorageへ保存します。

目次へ

ファイル解説: app/page.tsx

履歴整形、添付処理、チャット送信。

298〜319行｜履歴保存
saveHistoryは画像base64をlocalStorageへ保存しないよう各messageを必要な項目だけに写し、履歴とmodeを別keyで保持します。続くgetTopicSystemPrompt/getTopicMessagesはmodeに合う指示とtopic履歴を返し、現在画面が扱う会話を一貫したshapeで取得します。

320〜361行｜topic送信履歴と生成file
現在topicのassistant messageを古い順に処理せず、最新からcode fenceを抽出します。同じpathが複数応答にあればMapへ先に入った新しい版を維持し、各応答内では競合数を付与します。最後にworkspaceOverridesを反映し、会話由来の生成結果と利用者編集を一つのfile一覧へ統合します。

362〜395行｜編集fileとExplorer連携
CodeViewerからの変更をfile id単位のworkspaceOverridesへ保存します。Explorerからのfile選択は5件上限を確認し、pathを含む本文をtext attachmentとして追加します。質問依頼はfileを添付してpath指定の説明文をinputへ追加し、textareaへfocusするので、Explorerから対象選択後すぐ質問を送れます。

396〜449行｜添付検査と読み分け
addFilesは新規選択から残り枠までを順に検査し、2MB超や未対応拡張子をエラーとして記録します。画像はbase64、文書・ソースは専用extractorまたはtext readerで読み、成功分だけstateへ追加します。全件の結果を最後に集約するため、あるfileが失敗しても他の成功添付は保持されます。

450〜485行｜添付削除・user message
removeAttachedFileはid一致の添付だけを取り除きます。buildUserMessageは質問とtext本文をモデル用contentへ連結し、画像名は本文に記して画像データ自体はimages配列に分離します。displayContentにはtext添付の名前だけを残すため、会話欄に長文のfile内容やbase64を表示せずに済みます。

486〜504行｜表示本文とtopic名
getDisplayContentは新形式のdisplayContentを優先し、古い履歴では正規表現で添付本文を隠す互換処理をします。makeTopicTitleは初回質問をtrimして60文字まで採用し、質問が空なら添付名、最終的に既定topic名を利用できるようにします。

505〜550行｜handleSend準備
handleSendはsubmitを抑止し、通信中の再送と質問・添付が両方空の送信を拒否します。system messageだけの履歴を初回と見なし、質問または添付名からtopic名を決定して、送信時のmodeに合うsystem promptとuser messageを準備します。初回topicのkey変更も履歴更新時にまとめて行います。

551〜621行｜API要求とassistant反映
送信するmodelがembedding専用かを見てinput形式とmessages形式を切り替え、/api/chatへAbortSignal付きでPOSTします。HTTP status/bodyとresponse本文の存在を検査し、成功時はclosingと参照file情報を含むassistant messageをtopicへ保存します。coding modeやlocal file参照ではworkspaceを自動で開きます。

622〜676行｜workspaceと後始末
失敗時はAbortErrorなら利用者による停止として何も追加せず、その他はconsole記録と説明付きassistant error messageを履歴へ保存します。finallyではcontrollerが当該送信のものか確認して参照を消し、loadingを解除します。依存配列にstate・handlerを列挙し、callbackが最新の入力、添付、mode、modelで動くようにします。

ファイル解説: app/page.tsx

topic操作、clipboard、回答表示の補助。

677〜683行｜Enter送信

textareaのCtrl+EnterまたはMeta+Enterだけ送信へ割り当て、通常のEnterは改行に使えるよう既定動作を止めます。void handleSendで非同期処理を起動し、keyboard handler自体を待機させません。フォーム送信とは別のショートカット経路を用意します。

684〜714行｜mode切替

switchModeは同じmodeなら何もせず、topic別modeと履歴先頭のsystem promptを更新して保存します。既存のuser/assistant messageは残し、codingへ変更した場合だけworkspaceを開きます。続くQuickAction handlerはtemplateをinputに設定してfocusし、message role/mode別style helperは吹き出し配置と配色を返します。

715〜735行｜QuickActionとrole style

QuickAction選択時はactionのpromptTemplateを呼び出して入力欄へ入れ、すぐ編集できるようfocusします。getMessageClassはuser/assistant/systemを分け、assistantはgeneral/coding modeで配色を変えます。描画側はこの関数だけを参照するため、roleごとのスタイル条件をmessage listから分離できます。

736〜789行｜topic読込・新規作成

loadTopicは空名を無視し、選んだtopicのmode・履歴・現在選択を切り替えます。addNewTopicはmodeに応じた既定名に連番を付けて衝突を避け、system prompt付き空履歴とmodeを登録します。入力・添付・errorを消去し、coding topicではworkspaceを開いて新規作業を始める状態にします。

790〜809行｜message削除

deleteMessageは最初にconfirmで削除意思を確認し、cancelならstateを変えません。承認時は現在topicのmessage配列から指定indexだけを除き、残りが空ならmode対応system promptを再挿入して履歴を保存します。会話を空配列のままにせず再送可能な状態を維持します。

810〜834行｜全文・コードcopy

copyMessageは既知の回答closingを除いた本文をclipboardへ書き、copy済みのmessage indexを2秒間表示します。copyCodeBlockはcode本文をそのまま書き、message indexとblock番号の組を識別値にします。どちらもClipboard API失敗をconsoleに記録し、成功した場合だけUIをcopy済みにします。

835〜842行｜assistant文章描画

renderAssistantTextは文章をspanに包み、whitespace-pre-wrapで改行・連続空白を表示に保ちます。keyPrefixをReact keyに使い、message内で複数段落を作る場合も識別を渡せるようにします。Markdown装飾を解釈せず、通常文をcode blockとは別に描画する基本部品です。

ファイル解説: app/page.tsx

応答レンダリング、履歴操作、チャット画面。

843〜931行 | 文章とコードblock表示

renderAssistantContentはfooterを取り除いた回答をcode fenceで分割し、文章区間とcode区間を順にReact要素へ変換します。code headerはfile/language表示に使い、各blockにworkspaceを開く操作とcopy buttonを追加します。mode別closingの形式を補正し、区切り線・ロゴと共に回答末尾へ描きます。

932〜959行 | topic rename

renameTopicはpromptのcancel・空白名を無視し、履歴とmodeのkeyを旧名から新名へ移して保存します。選択中topicだった場合currentTopicも追従します。会話本文は変更せず識別名だけを置き換えるため、履歴とmode対応が崩れません。

960〜997行 | topic delete

deleteTopicは不可逆であることをconfirmで示し、キャンセルなら何も変更しません。削除後に履歴がゼロなら既定topicとsystem promptを作り、選択中topicを消した場合は残存topicへ移ります。topicModesからも該当keyを削除して孤立したmode情報を残しません。

998〜1094行 | 履歴サイドバーUI

sidebarのheaderに履歴名と新規作成buttonを置き、現在modeで作る操作とgeneral/codingを指定する二つのbuttonを提供します。chatHistoryのkeyを逆順に一覧化し、mode icon・選択状態・topic名・rename/delete操作を各rowに結び付けます。履歴選択はloadTopicを呼びます。

1095〜1188行 | 画面ヘッダーUI

headerにはsidebar開閉、ロゴとアプリ名、現在mode badgeとgeneral/coding切替を配置します。右側selectはmode別selectableModelsを表示しselectedModelを更新します。選択幅調整ref/stateとworkspace開閉も接続し、チャットの共通操作を一か所から行えるようにします。

1189〜1285行 | 会話一覧UI

currentMessagesをroleごとに左右へ配置し、system messageは画面に出しません。userは添付本文を隠したdisplayContent、assistantは参照fileのpath・行数とcode fence表示を使います。assistant回答にはcopy/delete操作、通信中にはmodeに応じたloading表示を追加します。

1286〜1416行 | 入力・添付form

入力formはcoding modeでQuickActionsを表示し、submitとdrag enter/over/leave/dropで添付操作を処理します。添付chipから削除、file inputで複数選択、Explorerを開く、textarea入力、文字数表示、Enter改行/Ctrl+Enter送信、loading中の中断を提供します。errorを表示し、loadingや空入力に応じてbuttonを無効化します。

ファイル解説: app/page.tsx

右ペインのコードworkspace接続。

1417〜1434行 | CodingWorkspace props
右paneはisWorkspaceOpenで表示を切替え、CodingWorkspaceへ現在topicのCodeFile一覧を渡します。編集結果は親のoverride stateへ保存し、閉じる操作とExplorerの添付・質問callbackを接続します。coding modeで生成fileがある場合はgenerated、それ以外はexplorerを初期表示にします。

app/page.tsx

全1434行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
'use client';

import React, { useState, useCallback, useEffect, useRef, useMemo } from 'react';
import Image from 'next/image';
import { Message, AttachedFile, AgentMode, CodeFile } from '@/types/chat';
import {
  AVAILABLE_MODELS,
  GENERAL_SYSTEM_PROMPT,
  CODING_AGENT_SYSTEM_PROMPT,
  ANSWER_SEPARATOR,
  ensureAnswerClosingEmoji,
  getDefaultAnswerClosing,
  getRandomAnswerClosing,
  stripAnswerFooter,
  QuickAction,
} from '@/lib/prompts';
import { extractCodeFiles, resolveCodeFileConflicts } from '@/lib/codeExtractor';
import QuickActions from '@/components/QuickActions';
import CodingWorkspace from '@/components/CodingWorkspace';

const MAX_ATTACHMENTS = 5;
const MAX_FILE_BYTES = 2 * 1024 * 1024;
const FILE_INPUT_ACCEPT = [
  '.txt,.md,.markdown,.csv,.tsv,.json,.xml,.yml,.yaml,.toml,.ini,.log,.sql,.env',
  '.js,.jsx,.ts,.tsx,.mjs,.cjs,.py,.rb,.go,.rs,.java,.kt,.c,.h,.cpp,.cs,.php,.sh,.bat,.ps1',
  '.html,.css,.scss,.vue,.svelte,.dockerfile,.gitignore',
  '.pdf,.docx,.xlsx,.xls,.pptx',
  'image/png,image/jpeg,image/webp,image/gif,.png,.jpg,.jpeg,.webp,.gif',
].join(',');

const IMAGE_EXTENSIONS = new Set(['png', 'jpg', 'jpeg', 'webp', 'gif']);
const DOCUMENT_EXTENSIONS = new Set(['pdf', 'docx', 'xlsx', 'xls', 'pptx']);
const TEXT_EXTENSIONS = new Set([
  'txt', 'md', 'markdown', 'csv', 'tsv', 'json', 'xml', 'yml', 'yaml', 'toml', 'ini', 'log',
  'sql', 'env', 'js', 'jsx', 'ts', 'tsx', 'mjs', 'cjs', 'py', 'rb', 'go', 'rs', 'java', 'kt',
  'c', 'h', 'cpp', 'cs', 'php', 'sh', 'bat', 'ps1', 'html', 'css', 'scss', 'vue', 'svelte',
  'dockerfile', 'gitignore',
]);

const DEFAULT_TOPIC_NAME = '新しいチャット';

const getFileExtension = (fileName: string) => {
  const lastDot = fileName.lastIndexOf('.');
  if (lastDot < 0) {
    return fileName.toLowerCase();
  }
  return fileName.slice(lastDot + 1).toLowerCase();
};
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
49
50  const detectFileKind = (file: File): AttachedFile['kind'] | null => {
51    if (file.type.startsWith('image/')) {
52      return 'image';
53    }
54    if (file.type.startsWith('text/') || file.type === 'application/json' || file.type === 'application/
    xml') {
55      return 'text';
56    }
57
58    const extension = getFileExtension(file.name);
59    if (IMAGE_EXTENSIONS.has(extension)) {
60      return 'image';
61    }
62    if (
63      TEXT_EXTENSIONS.has(extension) ||
64      DOCUMENT_EXTENSIONS.has(extension) ||
65      file.name.toLowerCase() === 'dockerfile'
66    ) {
67      return 'text';
68    }
69    return null;
70  };
71
72  const readFileAsText = (file: File) =>
73    new Promise<string>((resolve, reject) => {
74      const reader = new FileReader();
75      reader.onload = () => resolve(String(reader.result ?? ''));
76      reader.onerror = () => reject(new Error(`${file.name} の読み込みに失敗しました。`));
77      reader.readAsText(file);
78    });
79
80  const readFileAsBase64 = (file: File) =>
81    new Promise<string>((resolve, reject) => {
82      const reader = new FileReader();
83      reader.onload = () => {
84        const result = String(reader.result ?? '');
85        const commaIndex = result.indexOf(',');
86        resolve(commaIndex >= 0 ? result.slice(commaIndex + 1) : result);
87      };
88      reader.onerror = () => reject(new Error(`${file.name} の読み込みに失敗しました。`));
89      reader.readAsDataURL(file);
90    });
91
92  const readDocumentAsText = async (file: File) => {
93    const extension = getFileExtension(file.name);
94    const arrayBuffer = await file.arrayBuffer();
95
96    if (extension === 'pdf') {
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  const pdfjs = await import('pdfjs-dist/legacy/build/pdf.mjs');
  pdfjs.GlobalWorkerOptions.workerSrc = new URL(
    'pdfjs-dist/build/pdf.worker.min.mjs',
    import.meta.url,
  ).toString();
  const pdf = await pdfjs.getDocument({ data: arrayBuffer }).promise;
  const pages: string[] = [];

  for (let pageNumber = 1; pageNumber <= pdf.numPages; pageNumber += 1) {
    const page = await pdf.getPage(pageNumber);
    const textContent = await page.getTextContent();
    const pageText = textContent.items
      .map((item) => ('str' in item ? item.str : ''))
      .join(' ')
      .trim();
    if (pageText) {
      pages.push(`[ページ ${pageNumber}]¥n${pageText}`);
    }
  }

  return pages.join('¥n¥n');
}

if (extension === 'docx') {
  const mammoth = await import('mammoth');
  const result = await mammoth.extractRawText({ arrayBuffer });
  return result.value.trim();
}

if (extension === 'xlsx' || extension === 'xls') {
  const xlsx = await import('xlsx');
  const workbook = xlsx.read(arrayBuffer, { type: 'array' });
  return workbook.SheetNames.map((sheetName) => {
    const sheet = workbook.Sheets[sheetName];
    return `[シート: ${sheetName}]¥n${xlsx.utils.sheet_to_csv(sheet)}`;
  }).join('¥n¥n');
}

if (extension === 'pptx') {
  const JSZip = (await import('jszip')).default;
  const zip = await JSZip.loadAsync(arrayBuffer);
  const slideNames = Object.keys(zip.files)
    .filter((name) => /^ppt¥/slides¥/slide¥d+¥.xml$/.test(name))
    .sort((left, right) => {
      const leftNumber = Number(left.match(/slide(¥d+)¥.xml/)?.[1] ?? 0);
      const rightNumber = Number(right.match(/slide(¥d+)¥.xml/)?.[1] ?? 0);
      return leftNumber - rightNumber;
    });
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    const slides = await Promise.all(
      slideNames.map(async (slideName, index) => {
        const xml = await zip.files[slideName].async('text');
        const document = new DOMParser().parseFromString(xml, 'application/xml');
        const text = Array.from(document.getElementsByTagName('a:t'))
          .map((node) => node.textContent ?? '')
          .join(' ')
          .trim();
        return text ? `[スライド ${index + 1}]¥n${text}` : '';
      }),
    );

    return slides.filter(Boolean).join('¥n¥n');
  }

  return readFileAsText(file);
};

export default function ChatApp() {
  const [chatHistory, setChatHistory] = useState<Record<string, Message[]>>({
    [DEFAULT_TOPIC_NAME]: [GENERAL_SYSTEM_PROMPT],
  });

  const [topicModes, setTopicModes] = useState<Record<string, AgentMode>>({
    [DEFAULT_TOPIC_NAME]: 'general',
  });

  const [input, setInput] = useState('');
  const [attachedFiles, setAttachedFiles] = useState<AttachedFile[]>([]);
  const [fileError, setFileError] = useState('');
  const [isDraggingFiles, setIsDraggingFiles] = useState(false);
  const [selectedModel, setSelectedModel] = useState<string>(AVAILABLE_MODELS[0].id);
  const [isLoading, setIsLoading] = useState(false);
  const [currentTopic, setCurrentTopic] = useState<string>(DEFAULT_TOPIC_NAME);
  const [isSidebarOpen, setIsSidebarOpen] = useState(true);
  const [isWorkspaceOpen, setIsWorkspaceOpen] = useState(true);
  const [copiedMessageIndex, setCopiedMessageIndex] = useState<number | null>(null);

  // ワークスペース内の編集されたファイル状態のオーバーライド
  const [workspaceOverrides, setWorkspaceOverrides] = useState<Record<string, string>>({});

  const fileInputRef = useRef<HTMLInputElement>(null);
  const inputRef = useRef<HTMLTextAreaElement>(null);
  const modelSelectRef = useRef<HTMLSelectElement>(null);
  const abortControllerRef = useRef<AbortController | null>(null);
  const [modelSelectFontSize, setModelSelectFontSize] = useState(16);

```

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  const currentMode: AgentMode = topicModes[currentTopic] || 'general';
  const selectableModels = currentMode === 'coding'
    ? AVAILABLE_MODELS.filter((model) => model.isCodingSpecialized)
    : AVAILABLE_MODELS;
  const selectedModelName = selectableModels.find((model) => model.id === selectedModel)?.name ?? selectedModel;

  useEffect(() => {
    const select = modelSelectRef.current;
    if (!select) return;

    const updateFontSize = () => {
      const canvas = document.createElement('canvas');
      const context = canvas.getContext('2d');
      if (!context) return;

      const styles = window.getComputedStyle(select);
      const baseFontSize = 16;
      context.font = `${styles.fontWeight} ${baseFontSize}px ${styles.fontFamily}`;

      const horizontalPadding = Number.parseFloat(styles.paddingLeft) + Number.parseFloat(styles.paddingRight);
      const horizontalBorders = Number.parseFloat(styles.borderLeftWidth) + Number.parseFloat(styles.borderRightWidth);
      const availableWidth = select.clientWidth - horizontalPadding - horizontalBorders - 28;
      const textWidth = context.measureText(selectedModelName).width;
      const fittedSize = textWidth > availableWidth
        ? Math.max(7, Math.floor((baseFontSize * availableWidth / textWidth) * 10) / 10)
        : baseFontSize;

      setModelSelectFontSize((currentSize) => currentSize === fittedSize ? currentSize : fittedSize);
    };

    updateFontSize();
    const resizeObserver = new ResizeObserver(updateFontSize);
    resizeObserver.observe(select);

    return () => resizeObserver.disconnect();
  }, [selectedModelName]);

  useEffect(() => {
    if (currentMode !== 'coding' || selectableModels.some((model) => model.id === selectedModel)) {
      return;
    }

    setSelectedModel(selectableModels[0]?.id ?? AVAILABLE_MODELS[0].id);
  }, [currentMode, selectableModels, selectedModel]);

  // 履歴の初期ロード
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
 useEffect(() => {
   if (typeof window === 'undefined') {
     return;
   }

   try {
     const savedHistory = window.localStorage.getItem('ai-chat-history');
     const savedModes = window.localStorage.getItem('ai-chat-modes');

     let parsedHistory: Record<string, Message[]> = {
       [DEFAULT_TOPIC_NAME]: [GENERAL_SYSTEM_PROMPT],
     };

     if (savedHistory) {
       parsedHistory = JSON.parse(savedHistory) as Record<string, Message[]>;
     }

     let parsedModes: Record<string, AgentMode> = {
       [DEFAULT_TOPIC_NAME]: 'general',
     };

     if (savedModes) {
       parsedModes = JSON.parse(savedModes) as Record<string, AgentMode>;
     }

     // 「起動時は新規の会話状態で起動」: 未送信の空トピックを探すか、無ければ作成
     const existingTopics = Object.keys(parsedHistory);
     const emptyTopic = existingTopics.find((topic) => {
       const msgs = parsedHistory[topic];
       return Array.isArray(msgs) && (msgs.length === 0 || (msgs.length === 1 && msgs[0]?.role ===
'system'));
     });

     let initialTopic: string;
     if (emptyTopic) {
       initialTopic = emptyTopic;
     } else {
       let topicNumber = 1;
       let nextTopic = DEFAULT_TOPIC_NAME;
       while (existingTopics.includes(nextTopic)) {
         topicNumber += 1;
         nextTopic = `新しいチャット ${topicNumber}`;
       }
       initialTopic = nextTopic;
       parsedHistory[initialTopic] = [GENERAL_SYSTEM_PROMPT];
       parsedModes[initialTopic] = 'general';
     }

     setChatHistory(parsedHistory);
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
      setTopicModes(parsedModes);
      setCurrentTopic(initialTopic);

    } catch (error) {
      console.error('履歴の読み込み中にエラーが発生しました:', error);
      setChatHistory({ [DEFAULT_TOPIC_NAME]: [GENERAL_SYSTEM_PROMPT] });
      setTopicModes({ [DEFAULT_TOPIC_NAME]: 'general' });
      setCurrentTopic(DEFAULT_TOPIC_NAME);
    }
  }, []);

  const saveHistory = useCallback((history: Record<string, Message[]>, modes?: Record<string, AgentMode>)
=> {
    if (typeof window !== 'undefined') {
      const historyWithoutImages = Object.fromEntries(
        Object.entries(history).map(([topic, messages]) => [
          topic,
          messages.map(({ role, content, displayContent, answerClosing, referencedFiles }) => ({
            role,
            content,
            displayContent,
            ...(answerClosing ? { answerClosing } : {}),
            ...(referencedFiles ? { referencedFiles } : {}),
          })),
        ]),
      );
      window.localStorage.setItem('ai-chat-history', JSON.stringify(historyWithoutImages));

      if (modes) {
        window.localStorage.setItem('ai-chat-modes', JSON.stringify(modes));
      }
    }
  }, []);

  const getTopicSystemPrompt = useCallback((mode: AgentMode): Message => {
    return mode === 'coding' ? CODING_AGENT_SYSTEM_PROMPT : GENERAL_SYSTEM_PROMPT;
  }, []);

  const getTopicMessages = useCallback(
    (topicName: string): Message[] => {
      const topicMessages = chatHistory[topicName];
      const mode = topicModes[topicName] || 'general';
      const prompt = getTopicSystemPrompt(mode);
      return Array.isArray(topicMessages) && topicMessages.length >= 0 ? topicMessages : [prompt];
    },
    [chatHistory, getTopicSystemPrompt, topicModes],
  );

  const currentMessages = getTopicMessages(currentTopic);
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
335
336  // トピックのメッセージ群から最新のコードファイルを抽出
337  const extractedFiles = useMemo(() => {
338    const assistantMessages = currentMessages.filter((m) => m.role === 'assistant');
339    if (assistantMessages.length === 0) return [];
340
341    // 最新のメッセージから順にコードを抽出し、パスまたは名前で最新版を統合
342    const fileMap = new Map<string, CodeFile>();
343
344    for (let i = assistantMessages.length - 1; i >= 0; i--) {
345      const filesInMsg = resolveCodeFileConflicts(extractCodeFiles(assistantMessages[i].content));
346      for (const f of filesInMsg) {
347        if (!fileMap.has(f.path)) {
348          fileMap.set(f.path, f);
349        }
350      }
351    }
352
353    const files = Array.from(fileMap.values());
354
355    // ユーザーによる編集オーバーライドを適用
356    return files.map((f) => ({
357      ...f,
358      content: workspaceOverrides[f.id] !== undefined ? workspaceOverrides[f.id] : f.content,
359    }));
360  }, [currentMessages, workspaceOverrides]);
361
362  const handleUpdateWorkspaceFile = useCallback((fileId: string, updatedContent: string) => {
363    setWorkspaceOverrides((prev) => ({
364      ...prev,
365      [fileId]: updatedContent,
366    }));
367  }, []);
368
369  const handleAttachLocalFile = useCallback((file: { name: string; path: string; content: string }) => {
370    setAttachedFiles((prev) => {
371      if (prev.length >= MAX_ATTACHMENTS) {
372        setFileError(`添付できるファイルは最大 ${MAX_ATTACHMENTS} 件です。`);
373        return prev;
374      }
375      return [
376        ...prev,
377        {
378          id: `local-${Date.now()}-${Math.random().toString(36).substring(2, 7)}`,
379          name: `${file.name}`,
380          kind: 'text',
381          content: `// ファイルパス: ${file.path}¥n${file.content}`,
382        },
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
      ];
    });
  }, []);

  const handleAskAboutLocalFile = useCallback((file: { name: string; path: string; content: string }) => {
    handleAttachLocalFile(file);
    setInput((prev) => {
      const prompt = `以下のローカルファイルについて、構成と役割を解説してください。¥n対象: ${file.path}`;
      return prev ? `${prev}¥n¥n${prompt}` : prompt;
    });
    inputRef.current?.focus();
  }, [handleAttachLocalFile]);

  const addFiles = useCallback(async (fileList: FileList | File[]) => {
    const incoming = Array.from(fileList);
    if (incoming.length === 0) {
      return;
    }

    setFileError('');

    const remainingSlots = MAX_ATTACHMENTS - attachedFiles.length;
    if (remainingSlots <= 0) {
      setFileError(`添付できるファイルは最大 ${MAX_ATTACHMENTS} 件です。`);
      return;
    }

    const nextFiles: AttachedFile[] = [];
    const errors: string[] = [];

    for (const file of incoming.slice(0, remainingSlots)) {
      if (file.size > MAX_FILE_BYTES) {
        errors.push(`${file.name} は 2MB を超えているため読み込めません。`);
        continue;
      }

      const kind = detectFileKind(file);
      if (!kind) {
        errors.push(`${file.name} は対応していない形式です。テキスト、PDF、Office 文書、または画像を選んでください。`);
        continue;
      }

      try {
        const content = kind === 'text' ? await readDocumentAsText(file) : await readFileAsBase64(file);
        nextFiles.push({
          id: `${file.name}-${file.size}-${file.lastModified}-${crypto.randomUUID()}`,
          name: file.name,
          kind,
```

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
        content,
      });
    } catch (error) {
      errors.push(error instanceof Error ? error.message : `${file.name} の読み込みに失敗しました。`);
    }
  }

  if (incoming.length > remainingSlots) {
    errors.push(`添付できるファイルは最大 ${MAX_ATTACHMENTS} 件です。`);
  }

  if (nextFiles.length > 0) {
    setAttachedFiles((prev) => [...prev, ...nextFiles]);
  }
  if (errors.length > 0) {
    setFileError(errors.join(' '));
  }
}, [attachedFiles.length]);

const removeAttachedFile = (id: string) => {
  setAttachedFiles((prev) => prev.filter((file) => file.id !== id));
  setFileError('');
};

const buildUserMessage = (question: string, files: AttachedFile[]): Message => {
  const trimmedQuestion = question.trim();
  const textFiles = files.filter((file) => file.kind === 'text');
  const imageFiles = files.filter((file) => file.kind === 'image');
  const parts: string[] = [];
  const displayParts: string[] = [];

  if (trimmedQuestion) {
    parts.push(trimmedQuestion);
    displayParts.push(trimmedQuestion);
  }

  for (const file of textFiles) {
    parts.push(`--- 添付ファイル: ${file.name} ---¥n${file.content}`);
    displayParts.push(`--- 添付ファイル: ${file.name} ---`);
  }

  if (imageFiles.length > 0) {
    const imageNames = imageFiles.map((file) => `[画像を添付: ${file.name}]`).join('¥n');
    parts.push(imageNames);
    displayParts.push(imageNames);
  }

  return {
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
      role: 'user',
      content: parts.join('¥n¥n'),
      displayContent: displayParts.join('¥n¥n'),
      ...(imageFiles.length > 0 ? { images: imageFiles.map((file) => file.content) } : {}),
    };
  };

  const getDisplayContent = (message: Message) => {
    if (message.displayContent !== undefined) {
      return message.displayContent;
    }

    return message.content.replace(
      /(^|¥n¥n)(--- 添付ファイル: ([^¥n]+) ---)¥n[¥s¥S]*?(?=¥n¥n--- 添付ファイル: |$)/g,
      '$1$2',
    );
  };

  const makeTopicTitle = (question: string) => {
    const trimmed = question.trim();
    if (!trimmed) {
      return DEFAULT_TOPIC_NAME;
    }
    return trimmed.slice(0, 60).trimEnd() || DEFAULT_TOPIC_NAME;
  };

  const handleSend = useCallback(
    async (e?: React.SubmitEvent<HTMLFormElement>) => {
      e?.preventDefault();
      if (isLoading) return;

      const userMessage = buildUserMessage(input, attachedFiles);
      if (!userMessage.content.trim()) return;

      const historyBeforeSend = getTopicMessages(currentTopic);
      const isInitialTopic = historyBeforeSend.length === 1 && historyBeforeSend[0]?.role === 'system';
      const activeTopicName = isInitialTopic
        ? makeTopicTitle(input.trim() || attachedFiles[0]?.name || DEFAULT_TOPIC_NAME)
        : currentTopic;

      // 初回送信時にシステムプロンプトを最新のモードに合わせて確実に設定
      const systemPromptForSend = getTopicSystemPrompt(currentMode);
      const preparedHistory = isInitialTopic
        ? [systemPromptForSend]
        : historyBeforeSend;

      setChatHistory((prev) => {
        const topicMessages = isInitialTopic
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
        ? [systemPromptForSend]
        : (Array.isArray(prev[activeTopicName]) ? prev[activeTopicName] : [systemPromptForSend]);

      if (isInitialTopic && activeTopicName !== currentTopic) {
        const rest = Object.fromEntries(
          Object.entries(prev).filter(([topic]) => topic !== currentTopic),
        );
        const nextHistory = {
          ...rest,
          [activeTopicName]: [...topicMessages, userMessage],
        };
        saveHistory(nextHistory);
        return nextHistory;
      }

      const nextHistory = {
        ...prev,
        [activeTopicName]: [...topicMessages, userMessage],
      };
      saveHistory(nextHistory);
      return nextHistory;
    });

    if (isInitialTopic) {
      setTopicModes((prev) => {
        const nextModes = { ...prev, [activeTopicName]: currentMode };
        if (activeTopicName !== currentTopic) {
          delete nextModes[currentTopic];
        }
        saveHistory(chatHistory, nextModes);
        return nextModes;
      });
      setCurrentTopic(activeTopicName);
    }

    setInput('');
    setAttachedFiles([]);
    setFileError('');
    setIsLoading(true);
    const abortController = new AbortController();
    abortControllerRef.current = abortController;

    try {
      const selectedModelOption = AVAILABLE_MODELS.find((model) => model.id === selectedModel);
      const requestBody = selectedModelOption?.isEmbedding
        ? {
            input: userMessage.content,
            model: selectedModel,
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
          }
        : {
            messages: [...preparedHistory, userMessage],
            model: selectedModel,
          };

      const response = await fetch('/api/chat', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(requestBody),
        signal: abortController.signal,
      });

      if (!response.ok) {
        const errorData = await response.json();
        const status = response.status;
        throw new Error(`APIサーバーエラー (Status: ${status}): ${errorData.error || '不明なエラー'}`);
      }

      const data = await response.json();

      if (!data.response) {
        throw new Error('AIからの応答 (response) が見つかりません。');
      }

      const aiMessage: Message = {
        role: 'assistant',
        content: data.response,
        answerClosing: getRandomAnswerClosing(currentMode),
        referencedFiles: Array.isArray(data.referencedLocalFiles) && data.referencedLocalFiles.length > 0
          ? data.referencedLocalFiles
          : undefined,
      };

      setChatHistory((prev) => {
        const targetTopic = isInitialTopic ? activeTopicName : currentTopic;
        const topicMessages = Array.isArray(prev[targetTopic])
          ? prev[targetTopic]
          : [systemPromptForSend];
        const nextHistory = {
          ...prev,
          [targetTopic]: [...topicMessages, aiMessage],
        };
        saveHistory(nextHistory);
        return nextHistory;
      });

      // コーディングモード、またはローカルファイルが参照された場合はワークスペースを開く
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
623         if (currentMode === 'coding' || (Array.isArray(data.referencedLocalFiles) &&
    data.referencedLocalFiles.length > 0)) {
624           setIsWorkspaceOpen(true);
625         }
626       } catch (error) {
627         if (error instanceof DOMException && error.name === 'AbortError') {
628           return;
629         }
630
631         console.error('チャット処理エラー:', error);
632
633         let errorMessage = '予期せぬエラーが発生しました。アプリを再起動するか、開発者コンソール（F12）を確
    認してください。';
634         if (error instanceof Error) {
635           errorMessage = `AI応答エラー: ${error.message}`;
636         }
637
638         const errorDisplayMessage: Message = {
639           role: 'assistant',
640           content: `□${errorMessage}`,
641           answerClosing: getRandomAnswerClosing(currentMode),
642         };
643
644         setChatHistory((prev) => {
645           const targetTopic = isInitialTopic ? activeTopicName : currentTopic;
646           const topicMessages = Array.isArray(prev[targetTopic])
647             ? prev[targetTopic]
648             : [systemPromptForSend];
649           const nextHistory = {
650             ...prev,
651             [targetTopic]: [...topicMessages, errorDisplayMessage],
652           };
653           saveHistory(nextHistory);
654           return nextHistory;
655         });
656       } finally {
657         if (abortControllerRef.current === abortController) {
658           abortControllerRef.current = null;
659         }
660         setIsLoading(false);
661       }
662     },
663     [
664       attachedFiles,
665       chatHistory,
666       currentMode,
667       currentTopic,
668       getTopicMessages,
669       getTopicSystemPrompt,
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    input,
    isLoading,
    saveHistory,
    selectedModel,
  ],
);

const handleKeyDown = (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
  if ((e.ctrlKey || e.metaKey) && e.key === 'Enter') {
    e.preventDefault();
    void handleSend();
  }
};

const switchMode = (newMode: AgentMode) => {
  if (currentMode === newMode) return;

  setTopicModes((prev) => {
    const nextModes = { ...prev, [currentTopic]: newMode };
    saveHistory(chatHistory, nextModes);
    return nextModes;
  });

  // システムプロンプトを先頭で更新
  setChatHistory((prev) => {
    const existing = prev[currentTopic] || [];
    const updatedPrompt = getTopicSystemPrompt(newMode);
    let nextMessages: Message[];

    if (existing.length > 0 && existing[0]?.role === 'system') {
      nextMessages = [updatedPrompt, ...existing.slice(1)];
    } else {
      nextMessages = [updatedPrompt, ...existing];
    }

    const nextHistory = { ...prev, [currentTopic]: nextMessages };
    saveHistory(nextHistory);
    return nextHistory;
  });

  if (newMode === 'coding') {
    setIsWorkspaceOpen(true);
  }
};

const handleSelectQuickAction = (action: QuickAction) => {
  const template = action.promptTemplate();
  setInput(template);
```

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    inputRef.current?.focus();
  };

  const getMessageClass = (role: Message['role']) => {
    if (role === 'user') {
      return 'bg-blue-600 text-white self-end rounded-br-none shadow-md';
    }
    if (role === 'assistant') {
      return currentMode === 'coding'
        ? 'bg-slate-800 text-slate-100 border border-slate-700 self-start rounded-tl-none shadow-lg'
        : 'bg-gray-200 text-gray-800 self-start rounded-tl-none shadow-md';
    }
    if (role === 'system') {
      return 'bg-indigo-100 text-indigo-800 self-start border-l-4 border-indigo-400 text-xs italic';
    }
    return 'bg-gray-100';
  };

  const loadTopic = (topicName: string) => {
    const normalizedTopic = topicName.trim();
    if (!normalizedTopic) return;

    const mode = topicModes[normalizedTopic] || 'general';
    const prompt = getTopicSystemPrompt(mode);

    setChatHistory((prev) => {
      const existingMessages = Array.isArray(prev[normalizedTopic]) ? prev[normalizedTopic] : [prompt];
      return {
        ...prev,
        [normalizedTopic]: existingMessages,
      };
    });

    setCurrentTopic(normalizedTopic);
  };

  const addNewTopic = (mode: AgentMode = currentMode) => {
    const existingTopics = Object.keys(chatHistory);
    let topicNumber = 1;
    let nextTopic = mode === 'coding' ? '新しいコーディングタスク' : DEFAULT_TOPIC_NAME;

    while (existingTopics.includes(nextTopic)) {
      topicNumber += 1;
      nextTopic = mode === 'coding' ? `コーディングタスク ${topicNumber}` : `新しいチャット ${topicNumber}
`;
    }

    const newPrompt = getTopicSystemPrompt(mode);

```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    setTopicModes((prev) => {
      const nextModes = { ...prev, [nextTopic]: mode };
      saveHistory(chatHistory, nextModes);
      return nextModes;
    });

    setChatHistory((prev) => {
      const nextHistory = {
        ...prev,
        [nextTopic]: [newPrompt],
      };
      saveHistory(nextHistory);
      return nextHistory;
    });

    setCurrentTopic(nextTopic);
    setInput('');
    setAttachedFiles([]);
    setFileError('');
    if (mode === 'coding') {
      setIsWorkspaceOpen(true);
    }
  };

  const deleteMessage = (index: number) => {
    if (!confirm('このメッセージを削除します。よろしいですか？')) {
      return;
    }

    const safeMessages = getTopicMessages(currentTopic);
    const newMessages = safeMessages.filter((_, messageIndex) => messageIndex !== index);
    const mode = topicModes[currentTopic] || 'general';
    const prompt = getTopicSystemPrompt(mode);

    setChatHistory((prev) => {
      const nextHistory = {
        ...prev,
        [currentTopic]: newMessages.length > 0 ? newMessages : [prompt],
      };
      saveHistory(nextHistory);
      return nextHistory;
    });
  };

  const copyMessage = async (content: string, index: number) => {
    try {
      const copyableContent = stripAnswerFooter(content).trimEnd();
      await navigator.clipboard.writeText(copyableContent);
```

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
      setCopiedMessageIndex(index);
      window.setTimeout(() => {
        setCopiedMessageIndex((currentIndex) => (currentIndex === index ? null : currentIndex));
      }, 2000);
    } catch (error) {
      console.error('回答のコピーに失敗しました:', error);
    }
  };

  const copyCodeBlock = async (code: string, codeIndex: number) => {
    try {
      await navigator.clipboard.writeText(code);
      setCopiedMessageIndex(codeIndex);
      window.setTimeout(() => {
        setCopiedMessageIndex((currentIndex) => (currentIndex === codeIndex ? null : currentIndex));
      }, 2000);
    } catch (error) {
      console.error('ソースコードのコピーに失敗しました:', error);
    }
  };

  const renderAssistantText = (text: string, keyPrefix: string) => {
    return (
      <span key={keyPrefix} className="whitespace-pre-wrap">
        {text}
      </span>
    );
  };

  const renderAssistantContent = (content: string, messageIndex: number, closing?: string) => {
    const isCoding = currentMode === 'coding';
    const assistantTextColor = isCoding ? 'text-slate-50' : '';
    const displayContent = stripAnswerFooter(content).trimEnd();
    const sections = displayContent.split(/```([^\n]*)\n?([\s\S]*?)```/g);
    let renderedContent: React.ReactNode;

    if (sections.length === 1) {
      renderedContent = (
        <p className={`whitespace-pre-wrap text-xl leading-relaxed ${assistantTextColor}`}>
          {renderAssistantText(displayContent, `message-${messageIndex}`)}
        </p>
      );
    } else {
      const renderedSections = [];
      let codeBlockIndex = 0;

      for (let sectionIndex = 0; sectionIndex < sections.length; sectionIndex += 1) {
        if (sectionIndex % 3 === 0) {
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
          if (sections[sectionIndex]) {
            renderedSections.push(
              <p key={`text-${sectionIndex}`} className={`whitespace-pre-wrap text-xl leading-relaxed ${assistantTextColor}`}>
                {renderAssistantText(sections[sectionIndex], `message-${messageIndex}-${sectionIndex}`)}
              </p>,
            );
          }
          continue;
        }

        if (sectionIndex % 3 === 1) {
          continue;
        }

        const rawHeader = sections[sectionIndex - 1]?.trim() || '';
        const code = sections[sectionIndex].replace(/^¥n|¥n$/g, '');
        const copyIndex = messageIndex * 1000 + codeBlockIndex;
        codeBlockIndex += 1;

        renderedSections.push(
          <div key={`code-${sectionIndex}`} className="my-3 overflow-hidden rounded-lg bg-gray-950 border border-gray-800 shadow-md">
            <div className="flex items-center justify-between border-b border-gray-800 px-3 py-2 text-xs text-gray-300">
              <div className="flex items-center gap-2 font-mono">
                <span classNamе="text-indigo-400">□{rawHeader || 'コード'}</span>
              </div>
              <div className="flex items-center gap-2">
                <button
                  type="button"
                  onClick={() => setIsWorkspaceOpen(true)}
                  className="text-indigo-300 hover:text-indigo-100 transition text-[11px] underline"
                >
                  ワークスペースで開く
                </button>
                <button
                  type="button"
                  onClick={() => void copyCodeBlock(code, copyIndex)}
                  className="px-2 py-0.5 rounded bg-gray-800 hover:bg-gray-700 text-gray-200 text-xs transition"
                >
                  {copiedMessageIndex === copyIndex ? 'コピー済み' : 'コピー'}
                </button>
              </div>
            </div>
            <pre className="overflow-x-auto p-3 text-sm leading-6 text-gray-100 font-mono">
              <code>{code}</code>
            </pre>
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
          </div>
        );
      }

      renderedContent = renderedSections;
    }

    const answerClosing = ensureAnswerClosingEmoji(
      closing ?? getDefaultAnswerClosing(isCoding ? 'coding' : 'general'),
    );

    return (
      <>
        {renderedContent}
        <div className={`mt-4 flex flex-col items-start gap-1 text-sm border-t pt-3 ${isCoding ? 'border-slate-600' : 'border-gray-300/30'}`}>
          <span className={`text-xs ${isCoding ? 'text-slate-300' : 'text-gray-400'}`}>{ANSWER_SEPARATOR}</span>
          <span className={`inline-flex flex-wrap items-center gap-2 text-2xl font-semibold ${isCoding ? 'text-slate-50' : 'text-gray-600 dark:text-gray-300'}`}>
            <span>{answerClosing}</span>
            <Image src="/Ollaemon_Image.jpg" alt="Olla衛門" width={160} height={87} className="h-21.75 w-40 shrink-0 object-contain" />
          </span>
        </div>
      </>
    );
  };

  const renameTopic = (oldTopicName: string) => {
    const nextName = window.prompt('トピック名を入力してください', oldTopicName);
    if (nextName === null) return;

    const trimmedName = nextName.trim();
    if (!trimmedName) return;

    setChatHistory((prev) => {
      if (!prev[oldTopicName]) return prev;
      const { [oldTopicName]: topicMessages, ...rest } = prev;
      const nextHistory = { ...rest, [trimmedName]: topicMessages };
      saveHistory(nextHistory);
      return nextHistory;
    });

    setTopicModes((prev) => {
      const mode = prev[oldTopicName] || 'general';
      const { [oldTopicName]: _, ...rest } = prev;
      const nextModes = { ...rest, [trimmedName]: mode };
      saveHistory(chatHistory, nextModes);
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    return nextModes;
  });

  if (currentTopic === oldTopicName) {
    setCurrentTopic(trimmedName);
  }
};

const deleteTopic = (topicName: string) => {
  const confirmMessage = `「${topicName}」を削除しますか？¥nこのチャットの履歴は元に戻せません。`;
  if (!window.confirm(confirmMessage)) return;

  setChatHistory((prev) => {
    const rest = Object.fromEntries(
      Object.entries(prev).filter(([topic]) => topic !== topicName),
    );
    const nextHistory = rest;

    if (Object.keys(nextHistory).length === 0) {
      const fallbackHistory = {
        [DEFAULT_TOPIC_NAME]: [GENERAL_SYSTEM_PROMPT],
      };
      saveHistory(fallbackHistory);
      setCurrentTopic(DEFAULT_TOPIC_NAME);
      return fallbackHistory;
    }

    saveHistory(nextHistory);
    if (currentTopic === topicName) {
      const nextTopic = Object.keys(nextHistory).at(-1);
      if (!nextTopic) return nextHistory;
      setCurrentTopic(nextTopic);
    }

    return nextHistory;
  });

  setTopicModes((prev) => {
    const nextModes = { ...prev };
    delete nextModes[topicName];
    return nextModes;
  });
};

return (
  <div className="flex h-screen w-screen overflow-hidden bg-gray-100 font-sans">
    {/* 左サイドバー: トピック管理 */}
    {isSidebarOpen && (
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
        <aside className="w-80 bg-white p-4 shadow-xl border-r border-gray-200 flex flex-col shrink-0 z-10">
          <div className="flex items-center justify-between mb-3">
            <h2 className="text-base font-bold text-gray-800 flex items-center gap-2">
              <span>□会話履歴</span>
            </h2>
            <button
              type="button"
              onClick={() => addNewTopic(currentMode)}
              className="flex items-center gap-1 rounded-md bg-blue-600 px-2.5 py-1 text-xs font-bold text-white shadow-xs transition hover:bg-blue-700 active:scale-95"
              title="新規の会話を開始"
            >
              <span>＋ 新規</span>
            </button>
          </div>

          {/* 新規の会話ボタン（メイン） */}
          <button
            type="button"
            onClick={() => addNewTopic(currentMode)}
            className="mb-3 flex w-full items-center justify-center gap-2 rounded-xl border border-blue-200 bg-linear-to-r from-blue-600 to-indigo-600 px-4 py-2.5 text-sm font-bold text-white shadow-md transition hover:from-blue-700 hover:to-indigo-700 active:scale-[0.99]"
          >
            <span className="text-base">☒</span>
            <span>新規の会話を開始</span>
          </button>

          {/* モード別新規作成ボタン */}
          <div className="grid grid-cols-2 gap-2 mb-4">
            <button
              type="button"
              onClick={() => addNewTopic('general')}
              className="flex items-center justify-center gap-1.5 rounded-lg border border-blue-200 bg-blue-50 px-2 py-1.5 text-xs font-semibold text-blue-700 transition hover:bg-blue-100 shadow-xs"
            >
              <span>□通常チャット</span>
            </button>
            <button
              type="button"
              onClick={() => addNewTopic('coding')}
              className="flex items-center justify-center gap-1.5 rounded-lg border border-indigo-200 bg-indigo-50 px-2 py-1.5 text-xs font-semibold text-indigo-700 transition hover:bg-indigo-100 shadow-xs"
            >
              <span>□技師モード</span>
            </button>
          </div>

          {/* トピックリスト */}
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
          <div className="space-y-1.5 overflow-y-auto flex-1 pr-1 scrollbar-thin">
            {Object.keys(chatHistory).reverse().map((topic) => {
              const mode = topicModes[topic] || 'general';
              const isSelected = currentTopic === topic;
              return (
                <div
                  key={topic}
                  className={`group flex items-center gap-1.5 rounded-lg border px-2 py-1.5 text-xs transition ${
                    isSelected
                      ? mode === 'coding'
                        ? 'border-indigo-300 bg-indigo-50/80 font-semibold text-indigo-900 shadow-xs'
                        : 'border-blue-300 bg-blue-50/80 font-semibold text-blue-900 shadow-xs'
                      : 'border-transparent text-gray-700 hover:bg-gray-50'
                  }`}
                >
                  <button
                    type="button"
                    onClick={() => loadTopic(topic)}
                    className="flex-1 overflow-hidden text-left truncate flex items-center gap-2 py-1"
                    title={topic}
                  >
                    <span className="text-sm shrink-0">
                      {mode === 'coding' ? '□' : '□'}
                    </span>
                    <span className="truncate">{topic === DEFAULT_TOPIC_NAME ? '新規チャット開始' : topic}</span>
                  </button>

                  <button
                    type="button"
                    onClick={() => renameTopic(topic)}
                    className="opacity-0 group-hover:opacity-100 p-1 text-gray-400 hover:text-blue-600 transition"
                    title="名前を編集"
                  >
                    ✏
                  </button>

                  <button
                    type="button"
                    onClick={() => deleteTopic(topic)}
                    className="opacity-0 group-hover:opacity-100 p-1 text-gray-400 hover:text-red-600 transition"
                    title="削除"
                  >
                    □
                  </button>
                </div>
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
            );
          })}
        </div>
      </aside>
    )}

    {/* メインエリア（チャット + ワークスペース） */}
    <div className="flex-1 flex flex-col h-full overflow-hidden">
      {/* ヘッダー */}
      <header className="px-5 py-4 bg-white border-b border-gray-200 flex flex-wrap justify-start items-center gap-4 shadow-xs shrink-0">
        <div className="flex items-center gap-4">
          <button
            type="button"
            onClick={() => setIsSidebarOpen((prev) => !prev)}
            className="flex h-14 w-14 items-center justify-center rounded-xl border border-gray-200 bg-gray-50 text-2xl text-gray-700 transition hover:bg-gray-100"
            title={isSidebarOpen ? '履歴を閉じる' : '履歴を開く'}
          >
            ☰
          </button>

          <div className="flex items-center gap-3">
            <Image src="/Ollaemon_Image.jpg" alt="Olla! ロゴ" width={1408} height={768} className="h-19.75 w-36 object-contain" />
            <div className="flex items-center gap-4">
              <h1 className="text-2xl font-extrabold text-gray-900 leading-tight">
                <span className="text-7xl">Olla衛門</span>
              </h1>
              <div className="flex flex-col items-start gap-2">
                <span className={`text-xs font-medium px-2 py-1 rounded-full ${
                  currentMode === 'coding'
                    ? 'bg-indigo-100 text-indigo-800 border border-indigo-200'
                    : 'bg-blue-100 text-blue-800 border border-blue-200'
                }`}>
                  {currentMode === 'coding' ? '技師（コーディングエージェント）' : '標準モード'}
                </span>
                <div className="flex items-center rounded-lg bg-gray-100 p-1 border border-gray-200 text-sm">
                  <button
                    type="button"
                    onClick={() => switchMode('general')}
                    className={`flex items-center gap-1.5 px-3 py-1.5 rounded-md font-medium transition ${
                      currentMode === 'general'
                        ? 'bg-white text-blue-700 shadow-xs'
                        : 'text-gray-600 hover:text-gray-900'
                    }`}
                  >
                    <span>🗨</span>
```

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
                      <span>通常チャット</span>
                    </button>
                    <button
                      type="button"
                      onClick={() => switchMode('coding')}
                      className={`flex items-center gap-1.5 px-3 py-1.5 rounded-md font-medium transition ${
                        currentMode === 'coding'
                          ? 'bg-indigo-600 text-white shadow-xs'
                          : 'text-gray-600 hover:text-gray-900'
                      }`}
                    >
                      <span>⌧</span>
                      <span>技師エージェント</span>
                    </button>
                  </div>
                </div>
              </div>
            </div>
          </div>

          {/* 右側: モデル選択 & ワークスペース開閉 */}
          <div className="ml-auto flex flex-wrap items-center gap-3">
            <div className="flex items-center gap-2 text-gray-700">
              <span className="relative -left-3.75 text-lg font-bold">AIモデル:</span>
              <select
                ref={modelSelectRef}
                value={selectedModel}
                onChange={(e) => setSelectedModel(e.target.value)}
                style={{ fontSize: `${modelSelectFontSize}px` }}
                className="relative -left-3.75 w-100 max-w-[calc(30vw+40px)] min-w-0 rounded-lg border-2 border-gray-300 bg-white px-3 py-3 font-semibold shadow-sm focus:border-indigo-500 focus:outline-none"
              >
                {selectableModels.map((model) => (
                  <option key={model.id} value={model.id} className="text-sm">
                    {model.name}
                  </option>
                ))}
              </select>
            </div>

            <button
              type="button"
              onClick={() => setIsWorkspaceOpen((prev) => !prev)}
              className={`relative -left-3.75 flex items-center gap-2 px-3.5 py-3 rounded-lg border text-base font-semibold transition ${
                isWorkspaceOpen
                  ? 'border-indigo-500 bg-indigo-50 text-indigo-700'
                  : 'border-gray-300 bg-white text-gray-700 hover:bg-gray-50'
```

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
            }`}
            title="コードワークスペース / ローカルファイル探査の表示/非表示"
          >
            <span>🗂</span>
            <span>ワークスペース {extractedFiles.length > 0 ? `(${extractedFiles.length})` : ''}</span>
          </button>
        </div>
      </header>

      {/* ワークスペースとチャットの2ペインコンテナ */}
      <div className="flex-1 flex overflow-hidden">
        {/* チャットペイン */}
        <div className="flex-1 flex flex-col h-full overflow-hidden bg-gray-50">
          {/* メッセージ一覧 */}
          <div className="flex-1 overflow-y-auto p-4 md:p-6 space-y-4">
            {currentMessages.map((msg, index) => {
              if (msg.role === 'system') return null;

              return (
                <div
                  key={`${msg.role}-${index}`}
                  className={`flex ${msg.role === 'user' ? 'justify-end' : 'justify-start'}`}
                >
                  <div className={`${msg.role === 'assistant' ? 'max-w-full' : 'max-w-[90%] md:max-w-[80%]'} p-4 rounded-xl shadow-xs ${getMessageClass(msg.role)}`}>
                    <strong className="text-xs font-medium block mb-1.5">
                      {msg.role === 'user' ? (
                        <span className="inline-flex items-center gap-1.5 text-blue-100">
                          <Image src="/user-avatar.svg" alt="" width={18} height={18} className="h-4 w-4" />
                          あなた
                        </span>
                      ) : (
                        <span className="inline-flex items-center gap-1.5 text-indigo-400 font-semibold">
                          <Image src="/Ollaemon_Image.jpg" alt="" width={32} height={18} className="h-4 w-8 object-contain" />
                          {currentMode === 'coding' ? 'Olla衛門 技師' : 'Olla衛門'}
                        </span>
                      )}
                    </strong>

                    {msg.role === 'user' ? (
                      <p className="whitespace-pre-wrap text-sm">{getDisplayContent(msg)}</p>
                    ) : (
                      <>
                        {msg.referencedFiles && msg.referencedFiles.length > 0 && (
                          <div className="mb-3 p-2.5 bg-indigo-950/70 border border-indigo-700/60 rounded-lg text-xs text-indigo-200">
                            <div className="font-semibold flex items-center gap-1.5 text-indigo-300 mb-1.5">
```

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
                                <span>🗂</span>
                                <span>参照したローカル環境 ({msg.referencedFiles.length}件):</span>
                              </div>
                              <div className="space-y-1">
                                {msg.referencedFiles.map((f, i) => (
                                  <div
                                    key={i}
                                    className="font-mono text-[11px] text-slate-200 flex items-center justify-between bg-black/40 px-2 py-1 rounded border border-indigo-900/40"
                                  >
                                    <span className="truncate flex items-center gap-1.5" title={f.path}>
                                      <span>{f.isDirectory ? '📁' : '📄'}</span>
                                      <span className="font-medium text-white">{f.name}</span>
                                      <span className="text-slate-400">({f.path})</span>
                                    </span>
                                    <span className="text-indigo-300 text-[10px] shrink-0 ml-2">
                                      {f.isDirectory ? `${f.lines}項目` : `${f.lines}行`}
                                    </span>
                                  </div>
                                ))}
                              </div>
                            </div>
                          )}
                          {renderAssistantContent(msg.content, index, msg.answerClosing)}
                        </>
                      )}

                      {msg.role === 'assistant' && (
                        <div className="mt-3 flex gap-3 text-xs pt-2 border-t border-gray-400/20">
                          <button
                            type="button"
                            onClick={() => void copyMessage(msg.content, index)}
                            className="text-gray-400 hover:text-indigo-400 underline transition"
                          >
                            {copiedMessageIndex === index ? 'コピー済み' : '全文コピー'}
                          </button>
                          <button
                            type="button"
                            onClick={() => deleteMessage(index)}
                            className="text-red-400 hover:text-red-500 underline transition"
                          >
                            削除
                          </button>
                        </div>
                      )}
                    </div>
                  </div>
                );
              })}
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
                {isLoading && (
                  <div className="flex justify-start">
                    <div className="p-4 bg-white border border-gray-200 rounded-xl shadow-xs animate-pulse">
                      <strong className="flex items-center gap-2 text-xs text-indigo-600 font-semibold">
                        <Image src="/Ollaemon_Image.jpg" alt="" width={40} height={22} className="h-5 w-10 object-contain" />
                        {currentMode === 'coding' ? 'Olla衛門 技師がコードを構築中...' : 'Olla衛門が思考中...'}
                      </strong>
                    </div>
                  </div>
                )}
              </div>

              {/* 下部フォームエリア */}
              <div className="p-3 bg-white border-t border-gray-200">
                {/* コーディングモード時のクイックアクションバー */}
                {currentMode === 'coding' && (
                  <div className="mb-2">
                    <QuickActions onSelectAction={handleSelectQuickAction} disabled={isLoading} />
                  </div>
                )}

                <form
                  onSubmit={handleSend}
                  onDragEnter={(e) => {
                    e.preventDefault();
                    e.stopPropagation();
                    if (!isLoading) setIsDraggingFiles(true);
                  }}
                  onDragOver={(e) => {
                    e.preventDefault();
                    e.stopPropagation();
                  }}
                  onDragLeave={(e) => {
                    e.preventDefault();
                    e.stopPropagation();
                    if (e.currentTarget === e.target) setIsDraggingFiles(false);
                  }}
                  onDrop={(e) => {
                    e.preventDefault();
                    e.stopPropagation();
                    setIsDraggingFiles(false);
                    if (!isLoading) void addFiles(e.dataTransfer.files);
                  }}
                  className={`rounded-xl border p-2 transition ${
                    isDraggingFiles ? 'border-indigo-500 bg-indigo-50/50' : 'border-gray-300 bg-white'
                  }`}
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
              >
                {attachedFiles.length > 0 && (
                  <div className="mb-2 flex flex-wrap gap-1.5 px-1">
                    {attachedFiles.map((file) => (
                      <span
                        key={file.id}
                        className="inline-flex items-center gap-1.5 rounded-full border border-gray-200 bg-gray-100 px-2.5 py-1 text-xs text-gray-700"
                      >
                        <span>{file.kind === 'image' ? '🖼' : '📄'} {file.name}</span>
                        <button
                          type="button"
                          onClick={() => removeAttachedFile(file.id)}
                          className="font-bold text-gray-400 hover:text-red-600"
                          aria-label={`${file.name} を削除`}
                        >
                          ×
                        </button>
                      </span>
                    ))}
                  </div>
                )}

                {fileError && (
                  <p className="mb-2 px-1 text-xs text-red-600">{fileError}</p>
                )}

                <div className="flex gap-2 items-start">
                  <input
                    ref={fileInputRef}
                    type="file"
                    multiple
                    accept={FILE_INPUT_ACCEPT}
                    className="hidden"
                    onChange={(e) => {
                      if (e.target.files) {
                        void addFiles(e.target.files);
                      }
                      e.target.value = '';
                    }}
                  />
                  <button
                    type="button"
                    onClick={() => fileInputRef.current?.click()}
                    disabled={isLoading}
                    className="flex h-11 w-11 shrink-0 items-center justify-center rounded-lg border border-gray-300 bg-gray-50 text-xl text-gray-600 transition hover:bg-gray-100 disabled:bg-gray-50 mt-1"
                    title="ファイルを添付（ソースコード、文書、画像）"
                  >
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
                    ＋
                  </button>
                  <button
                    type="button"
                    onClick={() => setIsWorkspaceOpen(true)}
                    disabled={isLoading}
                    className="flex h-11 w-11 shrink-0 items-center justify-center rounded-lg border border-gray-300 bg-gray-50 text-base text-gray-700 transition hover:bg-gray-100 disabled:bg-gray-50 mt-1"
                    title="ローカルワークスペースを開いてファイルを選択・参照"
                  >
                    □
                  </button>
                  <div className="flex-1 flex flex-col gap-1">
                    <textarea
                      ref={inputRef}
                      rows={5}
                      value={input}
                      onChange={(e) => setInput(e.target.value)}
                      onKeyDown={handleKeyDown}
                      placeholder={
                        currentMode === 'coding'
                          ? 'コーディングタスク、またはファイルパス（例: C:¥¥Portfolio¥¥... や @ファイル名）を入力... (Ctrl+Enterで送信)'
                          : '質問、またはファイルパス（例: C:¥¥Portfolio¥¥... や @ファイル名）を入力... (Ctrl+Enterで送信)'
                      }
                      disabled={isLoading}
                      className="w-full p-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 outline-none disabled:bg-gray-50 text-sm leading-relaxed resize-y font-sans"
                    />
                    <div className="flex justify-between items-center px-1 text-[11px] text-gray-400">
                      <span>□Enter: 改行 / Ctrl+Enter: 送信 / ローカルファイルパスの直接入力にも対応</span>
                      <span>{input.length} 文字</span>
                    </div>
                  </div>
                  <button
                    type={isLoading ? 'button' : 'submit'}
                    onClick={isLoading ? () => abortControllerRef.current?.abort() : undefined}
                    disabled={!isLoading && (!input.trim() && attachedFiles.length === 0)}
                    className={`h-11 px-6 rounded-lg transition font-semibold text-sm shadow-xs shrink-0 mt-1 ${
                      isLoading
                        ? 'bg-red-600 text-white hover:bg-red-700'
                        : currentMode === 'coding'
                        ? 'bg-indigo-600 text-white hover:bg-indigo-700 disabled:bg-gray-400'
                        : 'bg-blue-600 text-white hover:bg-blue-700 disabled:bg-gray-400'
                    }`}
                  >
```

目次へ

app/page.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
                    {isLoading ? '中断' : '送信'}
                  </button>
                </div>
              </form>
            </div>
          </div>

          {/* 右ペイン: ワークスペース（生成コード / ローカル探査）*/}
          {isWorkspaceOpen && (
            <div className="w-1/2 min-w-95 max-w-[65%] h-full flex flex-col shrink-0">
              <CodingWorkspace
                files={extractedFiles}
                onUpdateFile={handleUpdateWorkspaceFile}
                onClose={() => setIsWorkspaceOpen(false)}
                onAttachLocalFile={handleAttachLocalFile}
                onAskAboutLocalFile={handleAskAboutLocalFile}
                defaultMode={currentMode === 'coding' && extractedFiles.length > 0 ? 'generated' : 'explorer'}
              />
            </div>
          )}
        </div>
      </div>
    </div>
  );
}
```

ファイル解説: app/layout.tsx

Next.js共通HTML、metadata、Geist font。

1〜3行｜型・font・CSS import

Metadata型、Next.jsのGeist/Geist Mono helper、グローバルCSSをroot layoutへ読み込みます。layoutはrouteごとに重複させず、全画面で共通のfont定義とスタイルを使う入口です。

5〜12行｜フォントCSS変数

GeistとGeist Monoをnext/font経由で扱い、CSS variable名をそれぞれ指定します。subsetsをlatinに絞り、body向けsansとコード向けmonoを分けて読み込み、CSS側のTailwind themeから参照できるようにします。

15〜18行｜metadata

Next Metadataのtitleとdescriptionを宣言し、各ページのheadにアプリ名と説明を供給します。ページコンテンツとは別にbrowser tabや検索・共有時の基本情報を一元管理する設定です。

20〜29行｜RootLayout

RootLayoutはchildrenをhtml/bodyに包み、lang、font CSS variables、全高、antialiasing、縦flexをアプリ共通に設定します。App Router配下の各ページへ同一のHTML骨格とfontを適用します。

app/layout.tsx

全29行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "Olla衛門",
  description: "Olla衛門 - Your AI Assistant",
};

export default function RootLayout({ children }: LayoutProps<"/">) {
  return (
    <html
      lang="en"
      className={`${geistSans.variable} ${geistMono.variable} h-full antialiased`}
    >
      <body className="min-h-full flex flex-col">{children}</body>
    </html>
  );
}
```

目次へ

ファイル解説: app/globals.css

Tailwind CSSの読み込み、色・フォントのテーマ、OSダークモード、bodyの既定表示、

1行 | Tailwind CSSの読み込み

Tailwind CSS 4のentry pointを読み込み、utility classとCSS theme機能を共通stylesheetからbuildに含めます。各React componentでCSSをimportし直さず、全ページのスタイルを適用する起点です。

3〜6行 | 標準配色のCSS変数

rootにライト表示の背景色と前景文字色をCSS custom propertyとして定義します。bodyやTailwind themeは色を直接重複記述せず変数を参照するため、基準色を一か所で変更できます。

8〜13行 | Tailwindテーマとフォント名の接続

@theme inlineでroot色変数をTailwindのcolor tokenへ公開し、layout.tsxのGeist sans/mono CSS variableも接続します。utility classとfont指定が同じthemeから利用でき、色・fontの定義元を共有します。

15〜20行 | OSのダークモードに応じた配色変更

OSのprefers-color-schemeがdarkのときだけroot変数を暗い背景と明るい前景へ上書きします。bodyは同じ変数を使うので、個々の共通要素を書き換えなくても環境設定に応じて基調色が切り替わります。

22〜26行 | ページ全体のbody既定値

bodyへtheme変数の背景色・文字色を適用し、Arial、Helvetica、sans-serifの順にfallback fontを指定します。全ページの既定表示を定め、utility classのない要素にも基本色とfontを反映します。

28〜32行 | 細いスクロールバーの大きさ

scrollbar-thinが付いた領域のWebKit scrollbarの縦横寸法を6pxに揃えます。履歴やQuickActionsの表示領域を圧迫しにくくし、スクロール可能な場所を残しながら細い表示に統一します。

33〜35行 | スクロールバーのトラックを透明化

scrollbar trackを透明にし、スクロールバーの通り道だけ背景色が塗りつぶされるのを防ぎます。thumbだけを視認でき、履歴やタブ列の背景デザインを崩さずスクロール操作を示せます。

36〜39行 | 通常時のつまみの色と形

thumbを半透明の灰色にして大きなborder-radiusで丸いカプセル形状にします。hover時には不透明度を0.4から0.7へ上げ、平常時は控えめ、操作時は見つけやすい状態にします。

40〜43行 | hover時につまみを見やすくする

通常時のthumbは半透明灰色と大きなborder-radiusで丸い形にし、hover中は不透明度を0.4から0.7へ上げます。常時目立ちすぎず、pointerを重ねた時だけ位置と操作対象を認識しやすくします。

app/globals.css

全43行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #171717;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

@media (prefers-color-scheme: dark) {
  :root {
    --background: #0a0a0a;
    --foreground: #ededed;
  }
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}

/* スクロールバーのカスタマイズ */
.scrollbar-thin::-webkit-scrollbar {
  height: 6px;
  width: 6px;
}
.scrollbar-thin::-webkit-scrollbar-track {
  background: transparent;
}
.scrollbar-thin::-webkit-scrollbar-thumb {
  background: rgba(156, 163, 175, 0.4);
  border-radius: 9999px;
}
.scrollbar-thin::-webkit-scrollbar-thumb:hover {
  background: rgba(156, 163, 175, 0.7);
}

```

目次へ

ファイル解説: app/api/chat/route.ts

chat/embedding分岐とOllama通信。

1〜36行 | API型と15分timeout

NextResponseはJSON応答、undici Agent/fetchはOllama通信、localFileHelperは質問中のpath読込に使います。MessageとOllama応答shapeを型で表し、画像付きmessageやembedding配列を安全に扱う契約です。

38〜57行 | POST入力検証

POST bodyからmessages/input/modelを読み、modelが空または文字列でなければ400を返します。OLLAMA_BASE_URLを優先し、未設定時はlocalhost:11434を使い、末尾slashを除去してendpointを正しく連結します。inputがある場合はembedding要求へ分岐します。

58〜92行 | embedding branch

inputを検証した後、/api/embedへmodelとinputを送信し、長時間用Agentとrequest.signalを共有します。HTTP失敗は本文を調べ、特定のgemma-embedding2非対応だけ説明付き502、その他は元statusを返します。成功時は最初のvectorの配列・長さを検査し、vector自体ではなく次元数をJSONに返します。

93〜124行 | local path context

通常chatではmessages配列が必須で、それ以外は400です。最新user messageを探して本文中のWindows/@ pathをlocalFileHelperへ渡し、読めたfileはpath・行数・内容を区切り付きcontextとして構築します。該当user本文の前へcontextを挿入するので、pathを質問に含めてローカル情報を参照できます。

125〜160行 | chat request/response

処理済みmessagesをOllamaのrole/content/images形式へ整え、model、最大context 32768、最大出力8192、stream=falseで/api/chatにPOSTします。HTTP失敗は本文とstatus付きJSONで返し、成功時はmessage.contentまたはcontentから回答を取得します。参照fileのpath/name/行数/size/directory情報も返し、UIのバッジ表示に使います。

161〜192行 | abort/timeout/error

例外処理ではclient切断を499とし、その他はconsole.errorへ記録します。undiciのheader timeoutは原因codeで識別し、15分超過を説明する504を返します。分類できない通信・処理障害は内部例外の詳細を公開せず、一般向け500 JSONにまとめます。

app/api/chat/route.ts

全192行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
1   // app/api/chat/route.ts
2
3   import { NextResponse } from 'next/server';
4   import { Agent, fetch as ollamaFetch } from 'undici';
5   import { detectAndReadLocalFiles, ReferencedLocalFile } from '@/lib/localFileHelper';
6
7   // メッセージの型を定義し、TypeScriptの型安全性を確保します
8   interface Message {
9     role: 'user' | 'assistant' | 'system';
10    content: string;
11    images?: string[];
12  }
13
14  interface OllamaResponse {
15    message?: {
16      content?: string;
17    };
18    content?: string;
19  }
20
21  interface OllamaEmbeddingResponse {
22    embeddings?: number[][];
23  }
24
25  const OLLAMA_TIMEOUT_MS = 15 * 60 * 1000;
26  const ollamaDispatcher = new Agent({
27    headersTimeout: OLLAMA_TIMEOUT_MS,
28    bodyTimeout: OLLAMA_TIMEOUT_MS,
29  });
30
31  /**
32   * Ollama APIと通信し、チャットの応答を返すAPIエンドポイント
33   * 質問文内のローカルファイルパス自動検知・読み込みに対応
34   *
35   * @param request - クライアントからのリクエスト
36   * @returns Ollamaからの応答および参照ファイル一覧を含むJSON
37   */
38  export async function POST(request: Request) {
39    try {
40      const requestBody = await request.json() as {
41        messages?: Message[];
42        input?: string;
43        model?: string;
44      };
45      const { messages, input, model } = requestBody;
46      if (typeof model !== 'string' || !model.trim()) {
47        return NextResponse.json({ error: 'モデルが指定されていません。' }, { status: 400 });
48      }
```

目次へ

app/api/chat/route.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
49
50      const ollamaBaseUrl = (process.env.OLLAMA_BASE_URL ?? 'http://localhost:11434').replace(/¥/$/, '');
51
52      if (input !== undefined) {
53        if (typeof input !== 'string' || !input.trim()) {
54          return NextResponse.json({ error: '埋め込み対象の input が指定されていません。' }, { status:
    400 });
55        }
56
57        const response = await ollamaFetch(`${ollamaBaseUrl}/api/embed`, {
58          method: 'POST',
59          headers: { 'Content-Type': 'application/json' },
60          dispatcher: ollamaDispatcher,
61          signal: request.signal,
62          body: JSON.stringify({ model, input }),
63        });
64
65        if (!response.ok) {
66          const errorText = await response.text();
67          if (errorText.includes("unknown model architecture: 'gemma-embedding2'")) {
68            return NextResponse.json({
69              error: '現在起動中のOllamaランタイムがEmbeddingGemma 2のアーキテクチャ（gemma-embedding2）に対
    応していません。Ollamaをこのモデル形式に対応した最新版へ更新し、Ollamaを再起動してから再試行してくださ
    い。',
70            }, { status: 502 });
71          }
72
73          return NextResponse.json({
74            error: `Ollama APIエラーが発生しました。埋め込みモデルを確認してください。詳細: ${errorText}`,
75          }, { status: response.status });
76        }
77
78        const data = await response.json() as OllamaEmbeddingResponse;
79        const embedding = data.embeddings?.[0];
80        if (!Array.isArray(embedding) || embedding.length === 0) {
81          return NextResponse.json({ error: 'Ollamaから有効な埋め込みベクトルが返されませんでした。' },
    { status: 502 });
82        }
83
84        return NextResponse.json({
85          response: `埋め込みベクトルを生成しました（${embedding.length}次元）。このモデルは文章生成ではな
    く、テキストのベクトル化に使用されます。`,
86          embeddingDimensions: embedding.length,
87        });
88      }
89
90      if (!Array.isArray(messages)) {
91        return NextResponse.json({ error: 'messages または input を指定してください。' }, { status: 400 });
92      }
93
```

目次へ

app/api/chat/route.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    // ユーザーの最新メッセージからファイルパスを自動検出・読込
    const lastUserMsg = [...messages].reverse().find((m: Message) => m.role === 'user');
    let referencedFiles: ReferencedLocalFile[] = [];

    let processedMessages: Message[] = messages;

    if (lastUserMsg) {
      referencedFiles = await detectAndReadLocalFiles(lastUserMsg.content);

      if (referencedFiles.length > 0) {
        // ローカルファイルの内容をコンテキストとして構築
        const contextParts = referencedFiles.map((f) =>
          f.isDirectory
            ? `--- 参照ローカルディレクトリ: ${f.path} (${f.lines}項目) ---¥n${f.content}¥n--- ディレクトリ末尾 ---`
            : `--- 参照ローカルファイル: ${f.path} (${f.lines}行) ---¥n${f.content}¥n--- ファイル末尾 ---`
        );
        const contextInjection = `【システム指示: 質問文で指定された以下のローカル環境のファイル/ディレクトリ情報を直接閲覧・参照して回答してください】¥n¥n${contextParts.join('¥n¥n')}`;

        processedMessages = messages.map((m: Message) => {
          if (m === lastUserMsg) {
            return {
              ...m,
              content: `${contextInjection}¥n¥n【ユーザーの質問】¥n${m.content}`,
            };
          }
          return m;
        });
      }
    }

    const ollamaUrl = `${ollamaBaseUrl}/api/chat`;

    // Ollamaサーバーにリクエストを送信
    const response = await ollamaFetch(ollamaUrl, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      dispatcher: ollamaDispatcher,
      signal: request.signal,
      body: JSON.stringify({
        model: model,
        messages: processedMessages.map((m: Message) => ({
          role: m.role === 'user' ? 'user' : m.role === 'system' ? 'system' : 'assistant',
          content: m.content,
          ...(m.images?.length ? { images: m.images } : {}),
        })),
```

目次へ

app/api/chat/route.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
        options: {
          num_ctx: 32768,
          num_predict: 8192,
        },
        stream: false,
      }),
    });

    // Ollama APIからのレスポンスがエラーコードだった場合
    if (!response.ok) {
      const errorText = await response.text();
      return NextResponse.json({
        error: `Ollama APIエラーが発生しました。Ollamaが起動しているか確認してください。詳細: ${errorText}`
      }, { status: response.status });
    }

    // 正常なレスポンスの場合、JSONとして解析
    const data = await response.json() as OllamaResponse;

    // AIの応答テキストを返却
    const answer = data.message?.content || data.content || '';
    return NextResponse.json({
      response: answer,
      referencedLocalFiles: referencedFiles.map((f) => ({
        path: f.path,
        name: f.name,
        lines: f.lines,
        size: f.size,
        isDirectory: f.isDirectory,
      })),
    });

  } catch (error) {
    if (request.signal.aborted) {
      return new Response(null, { status: 499 });
    }

    console.error('Ollama API呼び出し中にエラーが発生しました:', error);

    if (error instanceof Error && error.cause instanceof Error && 'code' in error.cause) {
      const errorCode = String(error.cause.code);
      if (errorCode === 'UND_ERR_HEADERS_TIMEOUT') {
        return NextResponse.json(
          { error: 'Ollamaの応答が15分を超えました。軽量モデルを選ぶか、質問・添付内容を短くしてください。' },
          { status: 504 },
        );
      }
```

目次へ

app/api/chat/route.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    }

    return NextResponse.json({ error: '内部処理エラー: Ollamaサーバーとの通信に失敗しました。' }, { status: 500 });
  }
}
```

目次へ

ファイル解説: app/api/workspace/route.ts

preset/file read/directory listを返すGET API。

1〜30行 | fs/pathと上限

node:fs/promisesとnode:pathでserver側filesystemを非同期操作し、依存・build directoryを除外するSetを定義します。読込上限は500KB、WorkspaceItem型は一覧UIが表示するname/path/directory/size/extensionのデータ契約です。

32〜61行 | query/preset処理

URL queryからactionとpathを取り、action=presetsならPortfolioとcwdを候補として存在確認します。statでdirectoryだけ許可し、重複を避け、存在しない候補は個別に無視してJSONで返すため、Explorerは利用可能な起点だけ表示できます。

63〜87行 | file read

action=readにはpathが必須で、resolveで絶対pathにしてからstatします。directoryは400で拒否し、500KB超のfileは読まずサイズ付き説明を返します。許可fileをUTF-8で読み、basename/path/本文/行数/byte sizeを返してCodeViewerへ渡します。

88〜130行 | directory list

list処理ではpath未指定時にC:Portfolioを起点とし、存在とdirectory種別を検査して不正pathには400/404を返します。readdirのDirentから除外名をskipし、fileのsize取得と拡張子小文字化を行ってWorkspaceItem配列を作ります。folderにはsizeを設定せずdirectory属性で判別します。

131〜150行 | sortとHTTP error

一覧をfolder優先、次に大小文字を無視した名前順に並べ、currentPath、parentPath、itemsをまとめて返します。rootではparentPathをnullにするため「上へ」を出さずに済みます。想定外のfilesystem errorはログに出し、message付き500 JSONとしてUIへ伝えます。

app/api/workspace/route.ts

全150行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// app/api/workspace/route.ts

import { NextResponse } from 'next/server';
import fs from 'node:fs/promises';
import path from 'node:path';

const IGNORED_NAMES = new Set([
  'node_modules',
  '.git',
  '.next',
  'dist',
  'build',
  'out',
  '.turbo',
  '.vscode',
  '.idea',
  'bin',
  'obj',
  'coverage',
]);

const MAX_READ_SIZE_BYTES = 500 * 1024; // 500 KB

export interface WorkspaceItem {
  name: string;
  path: string;
  isDirectory: boolean;
  size?: number;
  extension?: string;
}

export async function GET(request: Request) {
  try {
    const { searchParams } = new URL(request.url);
    const action = searchParams.get('action') ?? 'list';
    const targetPath = searchParams.get('path');

    if (action === 'presets') {
      const presets = [
        'C:¥¥Portfolio',
        process.cwd(),
      ];
      // 重複排除と存在チェック
      const validPresets: string[] = [];
      for (const p of presets) {
        try {
          const stat = await fs.stat(/*turbopackIgnore: true*/ p);
          if (stat.isDirectory() && !validPresets.includes(p)) {
```

app/api/workspace/route.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
            validPresets.push(p);
          }
        } catch {
          // 存在しない場合はスキップ
        }
      }
      return NextResponse.json({ presets: validPresets });
    }

    if (action === 'read') {
      if (!targetPath) {
        return NextResponse.json({ error: 'ファイルパスが指定されていません。' }, { status: 400 });
      }

      const normalized = path.resolve(targetPath);
      const stat = await fs.stat(/*turbopackIgnore: true*/ normalized);

      if (stat.isDirectory()) {
        return NextResponse.json({ error: '指定されたパスはディレクトリです。' }, { status: 400 });
      }

      if (stat.size > MAX_READ_SIZE_BYTES) {
        return NextResponse.json({
          error: `ファイルサイズが大きすぎます（${(stat.size / 1024).toFixed(1)} KB）。500 KB 以下のファイルを選択してください。`,
        }, { status: 400 });
      }

      const content = await fs.readFile(/*turbopackIgnore: true*/ normalized, 'utf-8');
      const lines = content.split('¥n').length;

      return NextResponse.json({
        name: path.basename(normalized),
        path: normalized,
        content,
        lines,
        size: stat.size,
      });
    }

    // action === 'list' または 'tree'
    const dirPath = targetPath ? path.resolve(targetPath) : 'C:¥¥Portfolio';

    try {
      const stat = await fs.stat(/*turbopackIgnore: true*/ dirPath);
      if (!stat.isDirectory()) {
        return NextResponse.json({ error: '指定されたパスはディレクトリではありません。' }, { status: 400 });
      }
```

app/api/workspace/route.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  } catch {
    return NextResponse.json({ error: `ディレクトリが見つかりません: ${dirPath}` }, { status: 404 });
  }

  const dirents = await fs.readdir(/*turbopackIgnore: true*/ dirPath, { withFileTypes: true });
  const items: WorkspaceItem[] = [];

  for (const dirent of dirents) {
    if (IGNORED_NAMES.has(dirent.name)) {
      continue;
    }

    const fullPath = path.join(dirPath, dirent.name);
    const isDir = dirent.isDirectory();
    let size: number | undefined;

    if (!isDir) {
      try {
        const fstat = await fs.stat(/*turbopackIgnore: true*/ fullPath);
        size = fstat.size;
      } catch {
        // エラー時はスキップまたは undefined
      }
    }

    items.push({
      name: dirent.name,
      path: fullPath,
      isDirectory: isDir,
      size,
      extension: isDir ? undefined : path.extname(dirent.name).toLowerCase(),
    });
  }

  // フォルダ優先、その後名前順でソート
  items.sort((a, b) => {
    if (a.isDirectory && !b.isDirectory) return -1;
    if (!a.isDirectory && b.isDirectory) return 1;
    return a.name.localeCompare(b.name, undefined, { sensitivity: 'base' });
  });

  return NextResponse.json({
    currentPath: dirPath,
    parentPath: path.dirname(dirPath) !== dirPath ? path.dirname(dirPath) : null,
    items,
  });

} catch (error) {
```

目次へ

app/api/workspace/route.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    console.error('Workspace API エラー:', error);
    return NextResponse.json(
      { error: error instanceof Error ? error.message : 'ファイルシステム処理中にエラーが発生しました。' },
      { status: 500 },
    );
  }
}
```

目次へ

ファイル解説: components/CodeViewer.tsx

file表示・編集・copy・download。

1〜12行｜props契約

CodeViewerPropsは表示対象のCodeFileと任意の編集通知callbackを定義します。onCodeChangeがない場合は読み取り専用、ある場合だけ保存できるeditorになり、生成コード画面とExplorerの閲覧で同じ部品を再利用できます。

13〜18行｜ローカル編集state

copied、isEditing、editContentをstateに分け、本文を改行で分割して行番号用配列を作ります。コピーは編集中ならtextareaの最新値、通常はfile.contentを使い、成功表示を2秒で戻します。clipboard拒否などはconsoleに記録します。

20〜39行｜操作handler

downloadは編集中かどうかで保存する本文を選び、downloadSingleFileへ渡します。保存時はonCodeChangeがあれば親へ更新内容を通知して編集を閉じます。cancelはeditContentを現在のfile.contentに戻し、保存前変更を破棄できます。

42〜86行｜toolbar

toolbarにpathと言語を表示し、onCodeChangeの有無で編集buttonを出し分けます。編集中はsave/cancel、常時copy/downloadを提供し、操作対象を現在見ている本文へ統一します。button typeを指定しform内で誤送信しないようにしています。

88〜126行｜編集/閲覧領域

編集時はspellcheckを切ったtextareaへ本文を入力し、閲覧時は左列の行番号とpre/code本文を横scroll可能な等幅表示にします。行番号は本文配列のindex+1で作られ、syntax highlightは行わず、ソースをそのまま読む軽量viewerです。

components/CodeViewer.tsx

全126行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// components/CodeViewer.tsx
'use client';

import React, { useState } from 'react';
import { CodeFile } from '@/types/chat';
import { downloadSingleFile } from '@/lib/zipExporter';

interface CodeViewerProps {
  file: CodeFile;
  onCodeChange?: (updatedCode: string) => void;
}

export default function CodeViewer({ file, onCodeChange }: CodeViewerProps) {
  const [copied, setCopied] = useState(false);
  const [isEditing, setIsEditing] = useState(false);
  const [editContent, setEditContent] = useState(file.content);

  const lines = file.content.split('¥n');

  const handleCopy = async () => {
    try {
      await navigator.clipboard.writeText(isEditing ? editContent : file.content);
      setCopied(true);
      setTimeout(() => setCopied(false), 2000);
    } catch (err) {
      console.error('コピーに失敗しました:', err);
    }
  };

  const handleDownload = () => {
    downloadSingleFile({
      ...file,
      content: isEditing ? editContent : file.content,
    });
  };

  const handleSaveEdit = () => {
    if (onCodeChange) {
      onCodeChange(editContent);
    }
    setIsEditing(false);
  };

  return (
    <div className="flex flex-col h-full bg-gray-900 rounded-lg overflow-hidden border border-gray-800 text-gray-200 shadow-md">
      {/* ツールバー */}
      <div className="flex items-center justify-between px-3 py-2 bg-gray-950 border-b border-gray-800 text-xs">
```

目次へ

components/CodeViewer.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
      <div className="flex items-center gap-2 overflow-hidden">
        <span className="font-mono text-blue-400 font-semibold truncate" title={file.path}>
          □{file.path}
        </span>
        <span className="px-1.5 py-0.5 rounded bg-gray-800 text-gray-400 text-[10px] uppercase">
          {file.language}
        </span>
      </div>
      <div className="flex items-center gap-1.5 shrink-0">
        {onCodeChange && (
          <button
            type="button"
            onClick={isEditing ? handleSaveEdit : () => setIsEditing(true)}
            className={`px-2 py-1 rounded transition text-xs ${
              isEditing
                ? 'bg-emerald-600 text-white hover:bg-emerald-700'
                : 'bg-gray-800 hover:bg-gray-700 text-gray-300'
            }`}
          >
            {isEditing ? '□保存' : '✏ 編集'}
          </button>
        )}
        {isEditing && (
          <button
            type="button"
            onClick={() => {
              setEditContent(file.content);
              setIsEditing(false);
            }}
            className="px-2 py-1 rounded bg-gray-800 hover:bg-gray-700 text-gray-400 text-xs"
          >
            キャンセル
          </button>
        )}
        <button
          type="button"
          onClick={handleCopy}
          className="px-2 py-1 rounded bg-gray-800 hover:bg-gray-700 text-gray-300 transition text-xs"
        >
          {copied ? '□コピー済' : '□コピー'}
        </button>
        <button
          type="button"
          onClick={handleDownload}
          className="px-2 py-1 rounded bg-gray-800 hover:bg-gray-700 text-gray-300 transition text-xs"
          title="ファイルを保存"
        >
          □保存
```

目次へ

components/CodeViewer.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
          </button>
        </div>
      </div>

      {/* エディタ / コード表示 */}
      <div className="flex-1 overflow-auto p-0 font-mono text-xs leading-5">
        {isEditing ? (
          <textarea
            value={editContent}
            onChange={(e) => setEditContent(e.target.value)}
            className="w-full h-full p-4 bg-gray-900 text-gray-100 font-mono text-xs resize-none outline-none leading-5"
            spellCheck={false}
          />
        ) : (
          <div className="flex min-w-full">
            {/* 行番号 */}
            <div className="select-none py-3 pl-3 pr-3 text-right text-gray-600 bg-gray-950/40 border-r border-gray-800/60 font-mono shrink-0">
              {lines.map((_, i) => (
                <div key={i}>{i + 1}</div>
              ))}
            </div>
            {/* コード本体 */}
            <pre className="p-3 text-gray-200 overflow-x-auto flex-1 font-mono">
              <code>{file.content}</code>
            </pre>
          </div>
        )}
      </div>
    </div>
  );
}
```

目次へ

ファイル解説: components/CodingWorkspace.tsx

生成コード/Explorerとpreview等の統合UI。

1〜21行 | props/import

propsで生成CodeFile一覧、変更通知、close、Explorerからの添付・質問callback、初期sectionを受け取ります。CodeViewer、LivePreview、DiffModal、WorkspaceExplorerを束ね、抽出器とZIP helperを使うため、チャット画面から独立した作業paneを構成できます。

22〜41行 | state/derived data

workspaceSection、activeFileId、code/preview tab、diff modal、圧縮中flagをstate管理します。activeFileはid一致がなければfiles[0]へfallbackし、previewDocは全filesから生成、conflictingFilesは競合数を持つfileだけ選ぶため、props更新後もUIに必要な値を都度導出できます。

42〜52行 | ZIP handler

ZIP処理は生成fileが空なら何もせず、開始時にbuttonを無効化できるisZippingを立てます。固定名でdownload helperを呼び、失敗はconsoleへ記録、finallyでflagを解除するため、圧縮失敗後も再試行できます。

54〜134行 | toolbar/section/conflict

上部tabでAI生成コードとローカル探索を切替え、生成側だけZIPとDiff起動を表示し、closeは親callbackに渡します。同一pathに複数versionがあると警告し、sectionによってExplorerか生成画面を描き分けます。親callbackをExplorerへ渡して選択fileをchatへ接続します。

135〜190行 | file tabs/preview

生成fileをpath単位のtabとして並べ、選択時にactiveFileIdを更新してcode表示へ戻します。HTML/CSS/JSがある時だけpreview tabを見せ、fileが空なら生成前の案内、preview選択時はLivePreview、通常はCodeViewerを表示します。

191〜237行 | 子部品描画

CodeViewerの編集内容はactiveFile.idと共に親callbackへ返し、React stateで更新を保持します。DiffModalへ現在のfileをmodified側の初期値として渡し、open/close stateで表示を切替えます。code生成・Explorer・編集・比較機能を子部品に委譲する接続点です。

components/CodingWorkspace.tsx

全236行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// components/CodingWorkspace.tsx
'use client';

import React, { useState } from 'react';
import { CodeFile } from '@/types/chat';
import CodeViewer from './CodeViewer';
import LivePreview from './LivePreview';
import DiffModal from './DiffModal';
import WorkspaceExplorer from './WorkspaceExplorer';
import { buildPreviewDocument } from '@/lib/codeExtractor';
import { downloadFilesAsZip } from '@/lib/zipExporter';

interface CodingWorkspaceProps {
  files: CodeFile[];
  onUpdateFile: (fileId: string, updatedContent: string) => void;
  onClose: () => void;
  onAttachLocalFile?: (file: { name: string; path: string; content: string }) => void;
  onAskAboutLocalFile?: (file: { name: string; path: string; content: string }) => void;
  defaultMode?: 'generated' | 'explorer';
}

export default function CodingWorkspace({
  files,
  onUpdateFile,
  onClose,
  onAttachLocalFile,
  onAskAboutLocalFile,
  defaultMode = 'generated',
}: CodingWorkspaceProps) {
  const [workspaceSection, setWorkspaceSection] = useState<'generated' | 'explorer'>(defaultMode);
  const [activeFileId, setActiveFileId] = useState<string>(files[0]?.id || '');
  const [viewTab, setViewTab] = useState<'code' | 'preview'>('code');
  const [isDiffOpen, setIsDiffOpen] = useState(false);
  const [isZipping, setIsZipping] = useState(false);

  // ファイルが更新された際に activeFileId を追従
  const activeFile = files.find((f) => f.id === activeFileId) || files[0];
  const previewDoc = buildPreviewDocument(files);
  const hasPreview = !!previewDoc;
  const conflictingFiles = files.filter((file) => file.conflictingVersions);

  const handleDownloadZip = async () => {
    if (files.length === 0) return;
    setIsZipping(true);
    try {
      await downloadFilesAsZip(files, 'ollaemon-agent-project.zip');
    } catch (err) {
      console.error('ZIP生成に失敗しました:', err);
```

目次へ

components/CodingWorkspace.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    } finally {
      setIsZipping(false);
    }
  };

  return (
    <div className="flex flex-col h-full bg-slate-900 border-l border-slate-700 text-slate-100 shadow-2xl">
      {/* ワークスペース最上部ヘッダー */}
      <div className="flex items-center justify-between px-3 py-2 bg-slate-950 border-b border-slate-800">
        {/* セクション切替タブ（生成コード / ローカル探査） */}
        <div className="flex items-center rounded-lg bg-slate-900 p-0.5 border border-slate-800 text-xs">
          <button
            type="button"
            onClick={() => setWorkspaceSection('generated')}
            className={`flex items-center gap-1.5 px-2.5 py-1 rounded-md font-medium transition ${
              workspaceSection === 'generated'
                ? 'bg-indigo-600 text-white shadow-xs font-semibold'
                : 'text-slate-400 hover:text-slate-200'
            }`}
          >
            <span>□AI生成コード</span>
            <span className="px-1.5 py-0.2 rounded-full bg-black/30 text-[10px] font-mono">
              {files.length}
            </span>
          </button>

          <button
            type="button"
            onClick={() => setWorkspaceSection('explorer')}
            className={`flex items-center gap-1.5 px-2.5 py-1 rounded-md font-medium transition ${
              workspaceSection === 'explorer'
                ? 'bg-indigo-600 text-white shadow-xs font-semibold'
                : 'text-slate-400 hover:text-slate-200'
            }`}
          >
            <span>□ローカル探査</span>
          </button>
        </div>

        {/* 右側アクション */}
        <div className="flex items-center gap-2">
          {workspaceSection === 'generated' && (
            <>
              {/* ZIP ダウンロード */}
              <button
                type="button"
                disabled={files.length === 0 || isZipping}
                onClick={handleDownloadZip}
```

目次へ

components/CodingWorkspace.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
                className="inline-flex items-center gap-1.5 px-2.5 py-1 rounded-md bg-indigo-600 hover:bg-indigo-500 disabled:opacity-40 disabled:cursor-not-allowed text-xs font-medium text-white transition shadow-xs"
                title="生成された全ファイルをZIPで保存"
              >
                <span>🗜</span>
                <span>{isZipping ? '圧縮中...' : 'ZIP一括保存'}</span>
              </button>

              {/* Diff比較モーダル起動 */}
              <button
                type="button"
                onClick={() => setIsDiffOpen(true)}
                className="px-2 py-1 rounded-md bg-slate-800 hover:bg-slate-700 text-slate-300 text-xs transition"
                title="コード差分を比較"
              >
                ⧉差分比較
              </button>
            </>
          )}

          {/* ワークスペース閉じる */}
          <button
            type="button"
            onClick={onClose}
            className="flex h-7 w-7 items-center justify-center rounded-md text-slate-400 hover:bg-slate-800 hover:text-white transition"
            title="ワークスペースを閉じる"
          >
            ✕
          </button>
        </div>
      </div>

      {workspaceSection === 'generated' && conflictingFiles.length > 0 && (
        <div
          role="status"
          className="border-b border-amber-700/60 bg-amber-950/50 px-3 py-2 text-xs text-amber-200"
        >
          <span className="font-semibold">同じパスに異なるコードが複数あります。</span>
          {' '}最後のコードブロックを採用しました: {conflictingFiles.map((file) => file.path).join('、')}
        </div>
      )}

      {/* セクション内容 */}
      {workspaceSection === 'explorer' ? (
        <div className="flex-1 overflow-hidden">
          <WorkspaceExplorer
```

目次へ

components/CodingWorkspace.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
            onAttachFile={onAttachLocalFile}
            onAskAboutFile={onAskAboutLocalFile}
          />
        </div>
      ) : (
        <>
          {/* ファイルタブバー & 表示切替 */}
          <div className="flex items-center justify-between bg-slate-950/70 border-b border-slate-800/80 px-2 overflow-x-auto text-xs scrollbar-thin">
            {/* ファイルリストタブ */}
            <div className="flex items-center gap-1 py-1">
              {files.map((file) => {
                const isActive = activeFile?.id === file.id && viewTab === 'code';
                return (
                  <button
                    key={file.id}
                    type="button"
                    onClick={() => {
                      setActiveFileId(file.id);
                      setViewTab('code');
                    }}
                    className={`flex items-center gap-1.5 px-3 py-1.5 rounded-t-md font-mono transition text-xs border-b-2 ${
                      isActive
                        ? 'bg-slate-900 text-blue-300 border-indigo-500 font-semibold'
                        : 'text-slate-400 hover:bg-slate-900/50 hover:text-slate-200 border-transparent'
                    }`}
                  >
                    <span>🗋</span>
                    <span>{file.name}</span>
                  </button>
                );
              })}
              {files.length === 0 && (
                <span className="text-slate-500 italic py-1 px-2">まだ生成されたコードがありません</span>
              )}
            </div>

            {/* ライブプレビュー切替タブ */}
            {hasPreview && (
              <div className="flex items-center py-1 pl-2 shrink-0">
                <button
                  type="button"
                  onClick={() => setViewTab(viewTab === 'preview' ? 'code' : 'preview')}
                  className={`flex items-center gap-1 px-3 py-1 rounded text-xs font-semibold transition ${
                    viewTab === 'preview'
                      ? 'bg-emerald-600 text-white shadow-xs'
                      : 'bg-emerald-950/50 text-emerald-300 border border-emerald-700/60 hover:bg-emerald-900/50'
```

目次へ

components/CodingWorkspace.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
                  }`}
                >
                  <span>🗔</span>
                  <span>Webプレビュー</span>
                </button>
              </div>
            )}
          </div>

          {/* メインコンテンツエリア */}
          <div className="flex-1 overflow-hidden p-3 bg-slate-900">
            {files.length === 0 ? (
              <div className="flex flex-col items-center justify-center h-full text-center p-6 border-2 border-dashed border-slate-800 rounded-xl">
                <span className="text-4xl mb-3">🗔</span>
                <h4 className="text-base font-bold text-slate-300 mb-1">
                  コーディングエージェントの作業領域
                </h4>
                <p className="text-xs text-slate-400 max-w-sm mb-4 leading-relaxed">
                  チャットでコードの生成を依頼すると、Olla衛門 技師が作成したファイルが自動的にここにマッピングされます。
                </p>
                <div className="bg-slate-950 p-3 rounded-lg text-left text-xs font-mono text-slate-300 space-y-1 border border-slate-800">
                  <div>□おすすめのプロンプト:</div>
                  <div>・「ReactとTailwindでTODOアプリのコンポーネントを作って」</div>
                  <div>・「HTMLとCanvasでブロック崩しゲームを作って」</div>
                  <div>・「CSV解析のPythonスクリプトを単体テスト付きで書いて」</div>
                </div>
              </div>
            ) : viewTab === 'preview' && previewDoc ? (
              <LivePreview htmlContent={previewDoc} />
            ) : activeFile ? (
              <CodeViewer
                file={activeFile}
                onCodeChange={(newCode) => onUpdateFile(activeFile.id, newCode)}
              />
            ) : null}
          </div>
        </>
      )}

      {/* Diff差分モーダル */}
      <DiffModal
        isOpen={isDiffOpen}
        onClose={() => setIsDiffOpen(false)}
        initialOriginalCode=""
        initialModifiedCode={activeFile?.content || ''}
      />
```

目次へ

components/CodingWorkspace.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    </div>
  );
}
```

目次へ

ファイル解説: components/DiffModal.tsx

簡易行比較をmodalで表示。

1〜16行｜props/行type

propsでは開閉、close callback、比較前後の初期文字列を受け取り、DiffLine.typeをadded/removed/unchangedの3種に限定します。比較処理とmodal表示をこの部品内に置き、呼出元は現在のfileと表示状態を渡せばよい構成です。

18〜51行｜index比較algorithm

computeSimpleDiffは双方を改行で分割し、同じindexの行を比較して一致ならunchanged、不一致なら削除行と追加行の組にします。末尾の余剰行は片側だけのadded/removedです。編集距離diffではないため、大幅な挿入では後続行も変更扱いに見える簡易比較です。

52〜78行｜state/open guard

original/modifiedは初期propsでstate初期化し、viewModeで差分表示と入力画面を切替えます。isOpenがfalseならDOMを返さず、open時だけ毎render比較結果を計算します。初期値は最初のmountで取り込むので、同じinstanceへの後続props変更とは別に扱われます。

79〜119行｜前後入力UI

headerはタイトル、差分/入力tab、close buttonを描画します。input modeでは左右のtextareaを編集でき、色とplaceholderでOriginal/Modifiedを区別します。overlayと85vh dialogで周囲画面から分離し、closeだけを親へ通知します。

120〜159行｜差分結果UI

diff modeでは行typeごとに色、左罫線、+/-記号を選び、変更方向を示します。一致行は中立色、追加は緑、削除は赤と取り消し線です。空配列では未入力の案内を出し、textarea変更を即時diff結果へ反映します。

components/DiffModal.tsx

全159行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```tsx
// components/DiffModal.tsx
'use client';

import React, { useState } from 'react';

interface DiffModalProps {
  isOpen: boolean;
  onClose: () => void;
  initialOriginalCode?: string;
  initialModifiedCode?: string;
}

interface DiffLine {
  type: 'added' | 'removed' | 'unchanged';
  content: string;
}

function computeSimpleDiff(original: string, modified: string): DiffLine[] {
  const origLines = original.split('¥n');
  const modLines = modified.split('¥n');
  const result: DiffLine[] = [];

  const max = Math.max(origLines.length, modLines.length);
  let i = 0;
  let j = 0;

  while (i < origLines.length || j < modLines.length) {
    if (i < origLines.length && j < modLines.length) {
      if (origLines[i] === modLines[j]) {
        result.push({ type: 'unchanged', content: origLines[i] });
        i++;
        j++;
      } else {
        // 簡易行比較
        result.push({ type: 'removed', content: origLines[i] });
        result.push({ type: 'added', content: modLines[j] });
        i++;
        j++;
      }
    } else if (i < origLines.length) {
      result.push({ type: 'removed', content: origLines[i] });
      i++;
    } else if (j < modLines.length) {
      result.push({ type: 'added', content: modLines[j] });
      j++;
    }
  }

```

目次へ

components/DiffModal.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  return result;
}

export default function DiffModal({
  isOpen,
  onClose,
  initialOriginalCode = '',
  initialModifiedCode = '',
}: DiffModalProps) {
  const [original, setOriginal] = useState(initialOriginalCode);
  const [modified, setModified] = useState(initialModifiedCode);
  const [viewMode, setViewMode] = useState<'diff' | 'input'>('diff');

  if (!isOpen) return null;

  const diffLines = computeSimpleDiff(original, modified);

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 backdrop-blur-xs p-4">
      <div className="flex flex-col w-full max-w-5xl h-[85vh] bg-gray-900 rounded-xl shadow-2xl border border-gray-700 overflow-hidden text-gray-200">
        {/* モーダルヘッダー */}
        <div className="flex items-center justify-between px-5 py-3 bg-gray-950 border-b border-gray-800">
          <div className="flex items-center gap-3">
            <span className="text-lg">☑</span>
            <h3 className="text-base font-bold text-gray-100">コード差分 (Diff) ビューア</h3>
          </div>
          <div className="flex items-center gap-2">
            <div className="flex rounded-md bg-gray-800 p-0.5 text-xs">
              <button
                type="button"
                onClick={() => setViewMode('diff')}
                className={`px-3 py-1 rounded transition ${
                  viewMode === 'diff' ? 'bg-indigo-600 text-white font-medium' : 'text-gray-400 hover:text-white'
                }`}
              >
                差分表示
              </button>
              <button
                type="button"
                onClick={() => setViewMode('input')}
                className={`px-3 py-1 rounded transition ${
                  viewMode === 'input' ? 'bg-indigo-600 text-white font-medium' : 'text-gray-400 hover:text-white'
                }`}
              >
                コード入力
              </button>
```

components/DiffModal.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
            </div>
            <button
              type="button"
              onClick={onClose}
              className="text-gray-400 hover:text-white text-xl px-2 leading-none"
            >
              ×
            </button>
          </div>
        </div>

        {/* コンテンツ */}
        <div className="flex-1 overflow-hidden p-4">
          {viewMode === 'input' ? (
            <div className="grid grid-cols-2 gap-4 h-full">
              <div className="flex flex-col h-full">
                <label className="text-xs font-semibold text-red-400 mb-1">修正前コード (Original)</label>
                <textarea
                  value={original}
                  onChange={(e) => setOriginal(e.target.value)}
                  placeholder="修正前のコードを入力..."
                  className="flex-1 p-3 bg-gray-950 border border-gray-800 rounded-lg font-mono text-xs text-gray-100 resize-none outline-none focus:border-indigo-500"
                />
              </div>
              <div className="flex flex-col h-full">
                <label className="text-xs font-semibold text-emerald-400 mb-1">修正後コード (Modified)</label>
                <textarea
                  value={modified}
                  onChange={(e) => setModified(e.target.value)}
                  placeholder="修正後のコードを入力..."
                  className="flex-1 p-3 bg-gray-950 border border-gray-800 rounded-lg font-mono text-xs text-gray-100 resize-none outline-none focus:border-indigo-500"
                />
              </div>
            </div>
          ) : (
            <div className="h-full overflow-auto bg-gray-950 rounded-lg border border-gray-800 p-2 font-mono text-xs">
              {diffLines.length === 0 ? (
                <div className="p-8 text-center text-gray-500">コードが入力されていません。</div>
              ) : (
                diffLines.map((line, idx) => {
                  let bgClass = 'bg-transparent text-gray-300';
                  let symbol = ' ';
                  if (line.type === 'added') {
                    bgClass = 'bg-emerald-950/60 text-emerald-300 border-l-2 border-emerald-500';
                    symbol = '+';
```

components/DiffModal.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
                } else if (line.type === 'removed') {
                  bgClass = 'bg-red-950/60 text-red-300 border-l-2 border-red-500 line-through opacity-80';
                  symbol = '-';
                }

                return (
                  <div key={idx} className={`px-2 py-0.5 whitespace-pre font-mono flex items-start ${bgClass}`}>
                    <span className="w-5 select-none text-gray-500 font-bold">{symbol}</span>
                    <span className="flex-1">{line.content || ' '}</span>
                  </div>
                );
              })
            )}
          </div>
        )}
      </div>
    </div>
  );
}
```

目次へ

ファイル解説: components/LivePreview.tsx

生成HTMLをiframeへ表示。

1〜16行 | srcDoc同期

LivePreviewPropsは完成HTML文字列だけを受け取ります。srcDoc stateはiframeへ渡すHTML、keyはiframe再生成用counterで、親が組み立てたdocumentを独立部品内に表示する最小状態です。

18〜26行 | reload/open new tab

htmlContent propが変わるとeffectがsrcDocを同期し、reload buttonはkeyを増やしてiframeをmountし直します。新規tabでは同じHTMLをtext/html Blobにしてobject URLを作りwindow.openします。iframe内の再読み込みと別windowでの確認を別操作として提供します。

28〜65行 | preview frame

toolbarにpreview状態表示とreload/new-tab操作を並べ、iframeへsrcDocを渡します。sandboxはscripts、forms、modalsだけを許可し、top navigationなど他のiframe能力は付与しません。HTMLをiframe documentとして実行し、親React画面と分離してpreviewします。

components/LivePreview.tsx

全65行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// components/LivePreview.tsx
'use client';

import React, { useState, useEffect } from 'react';

interface LivePreviewProps {
  htmlContent: string;
}

export default function LivePreview({ htmlContent }: LivePreviewProps) {
  const [srcDoc, setSrcDoc] = useState(htmlContent);
  const [key, setKey] = useState(0);

  useEffect(() => {
    setSrcDoc(htmlContent);
  }, [htmlContent]);

  const handleReload = () => {
    setKey((prev) => prev + 1);
  };

  const handleOpenInNewTab = () => {
    const blob = new Blob([htmlContent], { type: 'text/html' });
    const url = URL.createObjectURL(blob);
    window.open(url, '_blank');
  };

  return (
    <div className="flex flex-col h-full bg-white rounded-lg border border-gray-200 overflow-hidden shadow-xs">
      <div className="flex items-center justify-between px-3 py-2 bg-gray-100 border-b border-gray-200 text-xs text-gray-700">
        <div className="flex items-center gap-2">
          <span className="inline-block w-2.5 h-2.5 rounded-full bg-emerald-500 animate-pulse" />
          <span className="font-semibold text-gray-800">Web ライブプレビュー</span>
        </div>
        <div className="flex items-center gap-2">
          <button
            type="button"
            onClick={handleReload}
            className="px-2 py-1 rounded bg-white hover:bg-gray-50 border border-gray-300 text-gray-700 transition"
            title="プレビューを再読み込み"
          >
            □再読み込み
          </button>
          <button
            type="button"
            onClick={handleOpenInNewTab}
```

目次へ

components/LivePreview.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
            className="px-2 py-1 rounded bg-white hover:bg-gray-50 border border-gray-300 text-gray-700 transition"
            title="新規タブで実行"
          >
            ↗ 新規タブ
          </button>
        </div>
      </div>
      <div className="flex-1 w-full h-full min-h-[300px] bg-white relative">
        <iframe
          key={key}
          srcDoc={srcDoc}
          title="Live Preview"
          sandbox="allow-scripts allow-modals allow-forms"
          className="w-full h-full border-0"
        />
      </div>
    </div>
  );
}
```

目次へ

ファイル解説: components/QuickActions.tsx

定義済みactionを横並びで描画。

1〜11行 | props

propsはQuickAction選択callbackと任意disabledを定義します。親ChatAppが選択actionのtemplateを入力欄へ反映する役割で、この部品自体はprompt生成や送信をせず、選択UIと親への通知に責務を絞ります。

12〜33行 | button list

QUICK_ACTIONSをmapしてicon、label、description tooltip付きbuttonを一覧表示します。押下時に該当action objectを親へ渡すのでtemplateを呼ぶかは親が決め、通信中はdisabledで連打を防止します。横scrollにより狭い画面でもactionを利用できます。

components/QuickActions.tsx

全33行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// components/QuickActions.tsx
'use client';

import React from 'react';
import { QUICK_ACTIONS, QuickAction } from '@/lib/prompts';

interface QuickActionsProps {
  onSelectAction: (action: QuickAction) => void;
  disabled?: boolean;
}

export default function QuickActions({ onSelectAction, disabled }: QuickActionsProps) {
  return (
    <div className="flex items-center gap-1.5 overflow-x-auto py-1 px-1 scrollbar-thin">
      <span className="text-[11px] font-bold text-indigo-700 uppercase tracking-wider shrink-0 mr-1 flex items-center gap-1">
        <span>□技師アクション:</span>
      </span>
      {QUICK_ACTIONS.map((action) => (
        <button
          key={action.id}
          type="button"
          disabled={disabled}
          onClick={() => onSelectAction(action)}
          title={action.description}
          className="inline-flex items-center gap-1 px-2.5 py-1 rounded-md text-xs font-medium bg-indigo-50 text-indigo-700 border border-indigo-200 hover:bg-indigo-100 hover:border-indigo-300 transition-colors shrink-0 disabled:opacity-50 disabled:cursor-not-allowed shadow-xs"
        >
          <span>{action.icon}</span>
          <span>{action.label}</span>
        </button>
      ))}
    </div>
  );
}
```

目次へ

ファイル解説: components/WorkspaceExplorer.tsx

directory/file探索と親chatへの連携。

1〜28行 | type/state
WorkspaceItemはAPI一覧のfile/folder shapeを、propsは選択fileをchatへ添付または質問するcallbackを表します。Explorerはpath・items・preset・loading/errorに加え、選択fileと行数、読込中、添付feedbackを別stateで持ちます。

29〜47行 | preset/初期load
mount時にpresets APIを呼び、配列で返った場合だけ候補をstateへ設定します。currentPathが変化するとloadDirectoryを実行するeffectがあり、起動時の初期pathと操作後の移動を同じ一覧読込処理へ集約します。preset取得失敗はconsoleへ記録し一覧errorとは区別します。

49〜77行 | loadDirectory
loadDirectoryはloading/error/選択fileを先に初期化し、encodeURIComponentしたpathでlist APIを要求します。HTTP失敗はJSON errorを例外化し、成功時はcurrent path・入力欄・parent・itemsをまとめて更新します。catchで画面errorへ変換しfinallyでloadingを解除します。

78〜112行 | folder移動/file read
選択対象がfolderならloadDirectoryへ移動し、fileならread APIへ要求します。戻り値の拡張子・path・本文からCodeFileを組み立ててCodeViewerに渡し、行数も保存します。読込中表示を立て、API errorはerror stateへ表示し最後にloadingを解除します。

113〜139行 | attach/ask/size
attachは選択fileがありcallbackがある場合だけname/path/contentを親へ渡し、2秒間の完了表示を行います。askは同じ内容を質問用callbackへ渡します。formatSizeはbyte未定義、1024未満、KB、MBを分岐し一覧のsizeを読みやすく整えます。

140〜180行 | 拡張子icon
拡張子ごとのswitchでTypeScript、JavaScript、JSON、CSS、HTML、Python、Java、文書などをemoji分類し、未登録拡張子には汎用file iconを返します。extensionが未定義でもdefault分岐で一覧を描画し続けます。

181〜270行 | path/navigation controls
上部にはpreset button、手入力path、submitによる移動、parentPathがある場合だけ親へ戻る操作を配置します。path入力とcurrentPathを分け、入力中は一覧を読み直さずsubmit/clickで明示的にloadDirectoryを呼びます。error bannerはdismissできます。

271〜349行 | 一覧/preview/action UI
左右分割一覧はfolder/fileを同じbuttonで選択し、icon、name、sizeを表示します。右側はfile loading、選択file、未選択を分けてCodeViewerと行数/languageを表示します。attach/ask操作とloading/error/empty状態を備え、Explorerでの閲覧とchat連携を完結させます。

components/WorkspaceExplorer.tsx

全348行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// components/WorkspaceExplorer.tsx
'use client';

import React, { useState, useEffect, useCallback } from 'react';
import CodeViewer from './CodeViewer';
import { CodeFile } from '@/types/chat';

export interface WorkspaceItem {
  name: string;
  path: string;
  isDirectory: boolean;
  size?: number;
  extension?: string;
}

interface WorkspaceExplorerProps {
  onAttachFile?: (file: { name: string; path: string; content: string }) => void;
  onAskAboutFile?: (file: { name: string; path: string; content: string }) => void;
}

export default function WorkspaceExplorer({ onAttachFile, onAskAboutFile }: WorkspaceExplorerProps) {
  const [currentPath, setCurrentPath] = useState<string>('C:¥¥Portfolio');
  const [pathInput, setPathInput] = useState<string>('C:¥¥Portfolio');
  const [parentPath, setParentPath] = useState<string | null>(null);
  const [items, setItems] = useState<WorkspaceItem[]>([]);
  const [presets, setPresets] = useState<string[]>([]);
  const [isLoading, setIsLoading] = useState<boolean>(false);
  const [error, setError] = useState<string>('');

  // 選択中のファイル情報
  const [selectedFile, setSelectedFile] = useState<CodeFile | null>(null);
  const [selectedFileLines, setSelectedFileLines] = useState<number>(0);
  const [isLoadingFile, setIsLoadingFile] = useState<boolean>(false);
  const [attachFeedback, setAttachFeedback] = useState<string>('');

  // プリセットの読み込み
  useEffect(() => {
    fetch('/api/workspace?action=presets')
      .then((res) => res.json())
      .then((data) => {
        if (Array.isArray(data.presets)) {
          setPresets(data.presets);
        }
      })
      .catch((err) => console.error('Presets取得エラー:', err));
  }, []);

  // ディレクトリ一覧取得
```

目次へ

components/WorkspaceExplorer.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
const loadDirectory = useCallback(async (dirPath: string) => {
  setIsLoading(true);
  setError('');
  setSelectedFile(null);

  try {
    const res = await fetch(`/api/workspace?action=list&path=${encodeURIComponent(dirPath)}`);
    const data = await res.json();

    if (!res.ok) {
      throw new Error(data.error || 'ディレクトリの読み込みに失敗しました。');
    }

    setCurrentPath(data.currentPath);
    setPathInput(data.currentPath);
    setParentPath(data.parentPath);
    setItems(data.items || []);
  } catch (err) {
    setError(err instanceof Error ? err.message : 'エラーが発生しました。');
  } finally {
    setIsLoading(false);
  }
}, []);

useEffect(() => {
  void loadDirectory(currentPath);
}, [currentPath, loadDirectory]);

// ファイル読込
const handleSelectFile = async (item: WorkspaceItem) => {
  if (item.isDirectory) {
    void loadDirectory(item.path);
    return;
  }

  setIsLoadingFile(true);
  setError('');

  try {
    const res = await fetch(`/api/workspace?action=read&path=${encodeURIComponent(item.path)}`);
    const data = await res.json();

    if (!res.ok) {
      throw new Error(data.error || 'ファイルの読み込みに失敗しました。');
    }

    const ext = item.extension?.replace(/^¥./, '') || 'plaintext';
    const file: CodeFile = {
```

目次へ

components/WorkspaceExplorer.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
      id: `ws-${Date.now()}`,
      name: data.name,
      path: data.path,
      language: ext,
      content: data.content,
    };

    setSelectedFile(file);
    setSelectedFileLines(data.lines || 0);
  } catch (err) {
    setError(err instanceof Error ? err.message : 'ファイル読み込みエラー');
  } finally {
    setIsLoadingFile(false);
  }
};

const handleAttach = () => {
  if (!selectedFile || !onAttachFile) return;
  onAttachFile({
    name: selectedFile.name,
    path: selectedFile.path,
    content: selectedFile.content,
  });
  setAttachFeedback('添付完了！');
  setTimeout(() => setAttachFeedback(''), 2000);
};

const handleAsk = () => {
  if (!selectedFile || !onAskAboutFile) return;
  onAskAboutFile({
    name: selectedFile.name,
    path: selectedFile.path,
    content: selectedFile.content,
  });
};

const formatSize = (bytes?: number) => {
  if (bytes === undefined) return '';
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`;
};

const getFileIcon = (ext?: string) => {
  switch (ext) {
    case '.ts':
    case '.tsx':
      return '□';
```

目次へ

components/WorkspaceExplorer.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    case '.js':
    case '.jsx':
    case '.mjs':
      return '□';
    case '.json':
      return '□';
    case '.css':
    case '.scss':
      return '□';
    case '.html':
      return '□';
    case '.py':
      return '□';
    case '.java':
      return '□';
    case '.md':
    case '.txt':
      return '□';
    case '.pdf':
      return '□';
    default:
      return '□';
  }
};

return (
  <div className="flex flex-col h-full bg-slate-900 text-slate-100 overflow-hidden">
    {/* 上部: パスバーとプリセット */}
    <div className="p-3 bg-slate-950 border-b border-slate-800 space-y-2">
      {/* プリセットボタン */}
      {presets.length > 0 && (
        <div className="flex items-center gap-1.5 overflow-x-auto text-xs pb-1 scrollbar-none">
          <span className="text-slate-400 shrink-0 text-[11px]">□候補:</span>
          {presets.map((preset) => (
            <button
              key={preset}
              type="button"
              onClick={() => void loadDirectory(preset)}
              className={`px-2 py-0.5 rounded text-[11px] font-mono shrink-0 transition ${
                currentPath === preset
                  ? 'bg-indigo-600 text-white font-semibold'
                  : 'bg-slate-800 text-slate-300 hover:bg-slate-700'
              }`}
            >
              {preset}
            </button>
          ))}
        </div>
```

目次へ

components/WorkspaceExplorer.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
        )}

        {/* パス入力と移動 */}
        <form
          onSubmit={(e) => {
            e.preventDefault();
            if (pathInput.trim()) void loadDirectory(pathInput.trim());
          }}
          className="flex items-center gap-1.5"
        >
          {parentPath && (
            <button
              type="button"
              onClick={() => void loadDirectory(parentPath)}
              className="p-1.5 rounded bg-slate-800 hover:bg-slate-700 text-slate-300 text-xs transition shrink-0"
              title="親ディレクトリへ移動"
            >
              □上へ
            </button>
          )}

          <input
            type="text"
            value={pathInput}
            onChange={(e) => setPathInput(e.target.value)}
            placeholder="C:¥Portfolio¥..."
            className="flex-1 px-2.5 py-1.5 bg-slate-900 border border-slate-700 rounded text-xs font-mono text-slate-100 focus:outline-hidden focus:border-indigo-500"
          />

          <button
            type="submit"
            className="px-3 py-1.5 rounded bg-indigo-600 hover:bg-indigo-500 text-white text-xs font-medium transition shrink-0"
          >
            移動
          </button>
        </form>
      </div>

      {/* エラー表示 */}
      {error && (
        <div className="px-3 py-2 bg-red-900/60 border-b border-red-700 text-red-200 text-xs flex items-center justify-between">
          <span>□{error}</span>
          <button type="button" onClick={() => setError('')} className="text-red-400 hover:text-white">×</button>
        </div>
      )}
```

目次へ

components/WorkspaceExplorer.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
238
239         {/* メインペイン: 左右スプリット（一覧とプレビュー） */}
240         <div className="flex-1 flex overflow-hidden">
241           {/* 左側: ファイル・フォルダー一覧 */}
242           <div className="w-1/2 border-r border-slate-800 flex flex-col h-full overflow-hidden bg-slate-950/40">
243             <div className="px-3 py-1.5 text-[11px] font-semibold text-slate-400 border-b border-slate-800/80 bg-slate-900/40 flex items-center justify-between">
244               <span>項目一覧 ({items.length})</span>
245               {isLoading && <span className="text-indigo-400 animate-pulse">読込中...</span>}
246             </div>
247
248             <div className="flex-1 overflow-y-auto p-1.5 space-y-0.5 font-mono text-xs">
249               {items.map((item) => {
250                 const isSelected = selectedFile?.path === item.path;
251                 return (
252                   <button
253                     key={item.path}
254                     type="button"
255                     onClick={() => void handleSelectFile(item)}
256                     className={`w-full flex items-center justify-between px-2.5 py-1.5 rounded text-left transition ${
257                       isSelected
258                         ? 'bg-indigo-950 text-indigo-200 border border-indigo-700/80'
259                         : 'hover:bg-slate-800/60 text-slate-300'
260                     }`}
261                   >
262                     <span className="flex items-center gap-2 truncate">
263                       <span className="text-sm shrink-0">
264                         {item.isDirectory ? '🗀' : getFileIcon(item.extension)}
265                       </span>
266                       <span className="truncate">{item.name}</span>
267                     </span>
268
269                     <span className="text-[10px] text-slate-500 shrink-0 ml-2">
270                       {item.isDirectory ? 'フォルダ' : formatSize(item.size)}
271                     </span>
272                   </button>
273                 );
274               })}
275
276               {!isLoading && items.length === 0 && (
277                 <div className="p-6 text-center text-slate-500 text-xs">
278                   フォルダ内に表示可能なファイルがありません
279                 </div>
280               )}
281             </div>
282           </div>
283
```

components/WorkspaceExplorer.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
        {/* 右側: 選択ファイルプレビュー */}
        <div className="w-1/2 flex flex-col h-full overflow-hidden bg-slate-900">
          {isLoadingFile ? (
            <div className="flex items-center justify-center h-full text-slate-400 text-xs animate-pulse">
              ファイルを読み込み中...
            </div>
          ) : selectedFile ? (
            <div className="flex flex-col h-full">
              {/* プレビューヘッダー・アクション */}
              <div className="p-2.5 bg-slate-950 border-b border-slate-800 flex items-center justify-between gap-2">
                <div className="truncate">
                  <div className="text-xs font-bold text-slate-200 truncate" title={selectedFile.path}>
                    {selectedFile.name}
                  </div>
                  <div className="text-[10px] text-slate-400 font-mono">
                    {selectedFileLines} 行 · {selectedFile.language}
                  </div>
                </div>

                <div className="flex items-center gap-1.5 shrink-0">
                  {onAttachFile && (
                    <button
                      type="button"
                      onClick={handleAttach}
                      className="inline-flex items-center gap-1 px-2.5 py-1 rounded bg-indigo-600 hover:bg-indigo-500 text-white text-xs font-medium transition shadow-xs"
                      title="チャット入力欄の添付ファイルに追加"
                    >
                      <span>☑</span>
                      <span>{attachFeedback || 'チャットに添付'}</span>
                    </button>
                  )}

                  {onAskAboutFile && (
                    <button
                      type="button"
                      onClick={handleAsk}
                      className="inline-flex items-center gap-1 px-2.5 py-1 rounded bg-blue-600 hover:bg-blue-500 text-white text-xs font-medium transition shadow-xs"
                      title="このファイルについて質問文をセット"
                    >
                      <span>☑</span>
                      <span>質問する</span>
                    </button>
                  )}
                </div>
              </div>
            </div>

```

components/WorkspaceExplorer.tsx（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
              {/* コード閲覧領域 */}
              <div className="flex-1 overflow-hidden p-2">
                <CodeViewer file={selectedFile} />
              </div>
            </div>
          ) : (
            <div className="flex flex-col items-center justify-center h-full text-slate-500 text-xs p-6 text-center space-y-2">
              <span className="text-3xl">☑</span>
              <p className="font-semibold text-slate-400">ファイルを選択してください</p>
              <p className="text-[11px] text-slate-500 max-w-xs leading-relaxed">
                左側の一覧からファイルをクリックすると、ここにソースコードが表示され、ワンクリックでチャットのコンテキストに追加できます。
              </p>
            </div>
          )}
        </div>
      </div>
    </div>
  );
}
```

目次へ

ファイル解説: lib/codeExtractor.ts

fenceからfileを抽出し衝突解決とpreview作成。

1〜58行｜language/default filename表

EXTENSION_MAPはcode fenceの言語名を表示用languageへ標準化し、DEFAULT_FILENAMESはpathなしsnippetの保存名を決めます。認識名とdownload/ZIP用filenameの既定値を分け、未登録形式には後続でfallbackできます。

59〜91行｜コードblock抽出

extractCodeFilesは```header改行body```を正規表現で全件抽出し、先頭末尾の改行だけ除去します。fenceがなければFile pathコメントを境界として擬似blockに分割します。通常Markdownとコメントでfile名を示す応答を同じworkspace入力にできます。

92〜130行｜path/header解析

空本文を除外し、最初のコメント行にFile/pathがあればpathを抽出して本文から取り除きます。headerがlang:path、pathのみ、langのみかを判定し拡張子からlanguageを補完します。pathがなければlanguage別の既定名を割り当てます。

131〜162行｜CodeFile化

path区切りをslashに統一し、最後のsegmentをnameにします。pathなしなら既定filenameを使用し、URL encodingしたpathからstable idを作ってCodeFile型へ整形します。同じpathの更新を同一fileとして扱いやすくします。

163〜186行｜同一path競合

resolveCodeFileConflictsはpathをMap keyにし、同じpathが複数出た場合は最後のCodeFileを採用しつつ異なる本文をSetで数えます。異なる場合だけconflictingVersionsを付け、古い競合数を除いて返すため、workspaceは採用版と競合警告を併せて表示できます。

187〜254行｜preview HTML

buildPreviewDocumentはHTML候補、CSS、test/spec名でないJavaScriptを抽出し、対象がなければnullです。HTMLにはstyle/scriptをhead/bodyへ注入し、該当tagがなければ先頭/末尾へ追加します。HTMLなしなら最小shellを作り、JS実行例外はpreview内に表示します。

lib/codeExtractor.ts

全254行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// lib/codeExtractor.ts
import type { CodeFile } from '@/types/chat';

const EXTENSION_MAP: Record<string, string> = {
  js: 'javascript',
  jsx: 'javascript',
  ts: 'typescript',
  tsx: 'typescript',
  py: 'python',
  rb: 'ruby',
  go: 'go',
  rs: 'rust',
  java: 'java',
  c: 'c',
  cpp: 'cpp',
  cs: 'csharp',
  php: 'php',
  html: 'html',
  css: 'css',
  scss: 'scss',
  json: 'json',
  yaml: 'yaml',
  yml: 'yaml',
  xml: 'xml',
  sql: 'sql',
  sh: 'bash',
  bash: 'bash',
  bat: 'bat',
  ps1: 'powershell',
  md: 'markdown',
  dockerfile: 'dockerfile',
};

const DEFAULT_FILENAMES: Record<string, string> = {
  javascript: 'script.js',
  js: 'script.js',
  typescript: 'index.ts',
  ts: 'index.ts',
  python: 'main.py',
  py: 'main.py',
  html: 'index.html',
  css: 'styles.css',
  json: 'data.json',
  yaml: 'config.yaml',
  yml: 'config.yaml',
  sql: 'query.sql',
  bash: 'script.sh',
  sh: 'script.sh',
```

目次へ

lib/codeExtractor.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  powershell: 'script.ps1',
  ps1: 'script.ps1',
  markdown: 'README.md',
  md: 'README.md',
};

/**
 * AIのメッセージテキストからすべてのコードブロックを抽出し、
 * 扱いやすい CodeFile オブジェクトの配列に変換します。
 */
export function extractCodeFiles(content: string): CodeFile[] {
  const codeFiles: CodeFile[] = [];
  const regex = /```([^¥n]*)¥n([¥s¥S]*?)```/g;
  const codeBlocks = Array.from(content.matchAll(regex), (match) => ({
    header: match[1]?.trim() || '',
    code: match[2]?.replace(/^¥n|¥n$/g, '') || '',
  }));

  if (codeBlocks.length === 0) {
    const fileCommentRegex = /^[ ¥t]*(?:#|¥/¥/|--)¥s*(?:file(?:¥s*path)?|ファイルパス|ф а й л пパス)¥s*[:：]¥s*(.+?)¥s*$/gim;
    const fileComments = Array.from(content.matchAll(fileCommentRegex));
    codeBlocks.push(
      ...fileComments.map((fileComment, index) => ({
        header: '',
        code: content.slice(
          fileComment.index,
          fileComments[index + 1]?.index ?? content.length,
        ),
      })),
    );
  }

  let fileIndex = 1;

  for (const codeBlock of codeBlocks) {
    const rawHeader = codeBlock.header;
    let rawCode = codeBlock.code;

    if (!rawCode.trim()) {
      continue;
    }

    let language = 'text';
    let path = '';
    let name = '';

    // A file path may be included as the first source comment instead of in the fence header.
    const fileComment = rawCode.match(
```

目次へ

lib/codeExtractor.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    /^¥s*(?:#|¥/¥/|--)¥s*(?:file(?:¥s*path)?|ファイルパス|ф а й л パス)¥s*[: ：]¥s*(.+?)¥s*(?:¥r?¥n|$)/i,
  );
  if (fileComment) {
    path = fileComment[1]
      .trim()
      .replace(/¥s+(?:¥([^)]*¥)| ([^) ]*) )¥s*$/u, '');
    rawCode = rawCode.slice(fileComment[0].length);
    const extension = path.replace(/¥¥/g, '/').split('/').pop()?.split('.').pop()?.toLowerCase();
    if (extension && EXTENSION_MAP[extension]) {
      language = EXTENSION_MAP[extension];
    }
  }

  if (!rawCode.trim()) {
    continue;
  }

  // パターン1: ```lang:path (例: ```typescript:src/index.ts)
  // パターン2: ```path (例: ```src/index.ts)
  // パターン3: ```lang (例: ```typescript)
  if (!path && rawHeader.includes(':')) {
    const parts = rawHeader.split(':');
    language = parts[0].trim().toLowerCase();
    path = parts.slice(1).join(':').trim();
  } else if (!path && (rawHeader.includes('/') || rawHeader.includes('¥¥') || rawHeader.includes('.'))) {
    path = rawHeader;
    const dotIndex = path.lastIndexOf('.');
    if (dotIndex >= 0) {
      const ext = path.slice(dotIndex + 1).toLowerCase();
      language = EXTENSION_MAP[ext] || ext;
    }
  } else if (!path && rawHeader) {
    language = rawHeader.toLowerCase();
  }

  // パスからファイル名を特定
  if (path) {
    const normalized = path.replace(/¥¥/g, '/');
    const segments = normalized.split('/');
    name = segments[segments.length - 1] || `file_${fileIndex}`;
    path = normalized;
  } else {
    const defaultName = DEFAULT_FILENAMES[language] || `snippet_${fileIndex}.txt`;
    name = defaultName;
    path = defaultName;
  }

  const uniqueId = `file-${encodeURIComponent(path)}`;
```

目次へ

lib/codeExtractor.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    fileIndex += 1;

    codeFiles.push({
      id: uniqueId,
      name,
      path,
      language: EXTENSION_MAP[language] || language,
      content: rawCode,
    });
  }

  return codeFiles;
}

/**
 * 同じ応答内で同一パスが複数回出た場合、最後の内容を採用し、
 * 内容が異なる版があればその数を記録します。
 */
export function resolveCodeFileConflicts(files: CodeFile[]): CodeFile[] {
  const filesByPath = new Map<string, { file: CodeFile; contents: Set<string> }>();

  for (const file of files) {
    const existing = filesByPath.get(file.path);
    if (existing) {
      existing.file = file;
      existing.contents.add(file.content);
    } else {
      filesByPath.set(file.path, { file, contents: new Set([file.content]) });
    }
  }

  return Array.from(filesByPath.values(), ({ file, contents }) => {
    const { conflictingVersions: _previousConflict, ...resolvedFile } = file;
    return contents.size > 1
      ? { ...resolvedFile, conflictingVersions: contents.size }
      : resolvedFile;
  });
}

/**
 * 抽出されたファイル群からHTML/CSS/JSプレビュー用の完全なHTML文字列を構築します。
 */
export function buildPreviewDocument(files: CodeFile[]): string | null {
  const htmlFile = files.find(
    (f) => f.language === 'html' || f.name.endsWith('.html') || f.content.includes('<html') ||
f.content.includes('<!DOCTYPE'),
  );

  const cssFiles = files.filter((f) => f.language === 'css' || f.name.endsWith('.css'));
```

目次へ

lib/codeExtractor.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  const jsFiles = files.filter(
    (f) =>
      (f.language === 'javascript' || f.language === 'js') &&
      !f.name.includes('test') &&
      !f.name.includes('spec'),
  );

  if (!htmlFile && cssFiles.length === 0 && jsFiles.length === 0) {
    return null;
  }

  const combinedCss = cssFiles.map((f) => f.content).join('¥n¥n');
  const combinedJs = jsFiles.map((f) => f.content).join('¥n¥n');

  if (htmlFile) {
    let html = htmlFile.content;

    // もし既にあるHTMLにCSSとJSを注入する場合
    if (combinedCss && !html.includes(combinedCss)) {
      if (html.includes('</head>')) {
        html = html.replace('</head>', `<style>¥n${combinedCss}¥n</style>¥n</head>`);
      } else {
        html = `<style>¥n${combinedCss}¥n</style>¥n${html}`;
      }
    }

    if (combinedJs && !html.includes(combinedJs)) {
      if (html.includes('</body>')) {
        html = html.replace('</body>', `<script>¥n${combinedJs}¥n</script>¥n</body>`);
      } else {
        html = `${html}¥n<script>¥n${combinedJs}¥n</script>`;
      }
    }

    return html;
  }

  // HTMLファイルが存在せず、CSSやJSのみの場合のフォールバック
  return `<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Preview</title>
  <style>
    body { font-family: sans-serif; padding: 20px; }
    ${combinedCss}
  </style>
```

目次へ

lib/codeExtractor.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
</head>
<body>
  <div id="app">プレビュー対象のHTMLはありません。JS/CSSが読み込まれています。</div>
  <script>
    try {
      ${combinedJs}
    } catch (e) {
      console.error(e);
      document.body.innerHTML += '<p style="color:red">実行時エラー: ' + e.message + '</p>';
    }
  </script>
</body>
</html>`;
}
```

ファイル解説: lib/localFileHelper.ts

質問中pathを上限付きで読込。

1〜34行｜返却型/上限/ignore

ReferencedLocalFileはpath/name/行数/byte size/本文とdirectory判定をAPI応答向けに定義します。自動読込は300KB以下かつ最大5 pathに制限し、node_modulesやbuild outputなどを避けるSetを共有して無制限読込を防ぎます。

36〜64行｜tree再帰

buildDirectoryTreeTextは再帰的にreaddirし、深さ上限を超えると省略表示を返します。無視directoryを除きfolder優先・名前順で並べ、├──/ └──とprefixで階層を表現します。subtreeのlinesとfile/folder件数を集計し、LLM用のtext treeにします。

65〜90行｜tree集計/error

directory readに失敗した階層は例外を上へthrowせず、該当位置へアクセスerror行を加えて探索を続けます。成功階層ではfileとfolderを別々に数え、子階層の合計を足します。一部の権限不足で他の読める項目まで失わない設計です。

91〜119行｜Windows/@ path検出

Windows drive pathを抽出し、末尾の括弧や句読点、slashを整えてresolveします。@app/file.tsのような指定はcwd基準とPortfolio基準の両方を候補へ追加します。Setで重複を除き、質問文のpathをローカル絶対pathへ解決します。

120〜145行｜候補読み込み

候補をstatし最大5件で打ち切ります。通常fileは300KBを超えたら読まず、UTF-8本文と行数・sizeを返します。directoryなら深さ3までのtreeと直下の小さなpackage.jsonをcontext化し、内容量を制御します。

146〜179行｜結果返却

存在しないpath、権限不足、stat/read failureの候補は結果から除外し、成功したfile/directoryだけ返します。directoryはtree項目数をlines、sizeを0、isDirectoryをtrueとし、API側が本文と構成情報を異なる見出しでcontextへ注入できる形式です。

lib/localFileHelper.ts

全179行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// lib/localFileHelper.ts

import fs from 'node:fs/promises';
import path from 'node:path';

export interface ReferencedLocalFile {
  path: string;
  name: string;
  lines: number;
  size: number;
  content: string;
  isDirectory?: boolean;
}

const MAX_AUTO_READ_BYTES = 300 * 1024; // 300 KB
const MAX_DETECTED_PATHS = 5;

const IGNORED_NAMES = new Set([
  'node_modules',
  '.git',
  '.next',
  'dist',
  'build',
  'out',
  '.turbo',
  '.vscode',
  '.idea',
  'bin',
  'obj',
  'coverage',
]);

/**
 * 指定ディレクトリのファイルツリー構造をテキスト化します
 */
async function buildDirectoryTreeText(
  dirPath: string,
  prefix = '',
  currentDepth = 0,
  maxDepth = 3,
): Promise<{ lines: string[]; totalFiles: number; totalDirs: number }> {
  let lines: string[] = [];
  let totalFiles = 0;
  let totalDirs = 0;

  if (currentDepth > maxDepth) {
    return { lines: [`${prefix}... (これ以降の階層を省略)`], totalFiles, totalDirs };
  }
```

目次へ

lib/localFileHelper.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
 try {
   const dirents = await fs.readdir(/*turbopackIgnore: true*/ dirPath, { withFileTypes: true });
   const filtered = dirents.filter((d) => !IGNORED_NAMES.has(d.name));

   // ディレクトリ優先、その後名前順でソート
   filtered.sort((a, b) => {
     if (a.isDirectory() && !b.isDirectory()) return -1;
     if (!a.isDirectory() && b.isDirectory()) return 1;
     return a.name.localeCompare(b.name, undefined, { sensitivity: 'base' });
   });

   for (let i = 0; i < filtered.length; i++) {
     const dirent = filtered[i];
     const isLast = i === filtered.length - 1;
     const pointer = isLast ? '└─── ' : '├─── ';
     const nextPrefix = prefix + (isLast ?  '    ' : '|   ');
     const fullPath = path.join(dirPath, dirent.name);

     if (dirent.isDirectory()) {
       totalDirs++;
       lines.push(`${prefix}${pointer}${dirent.name}/`);
       const sub = await buildDirectoryTreeText(fullPath, nextPrefix, currentDepth + 1, maxDepth);
       lines = lines.concat(sub.lines);
       totalFiles += sub.totalFiles;
       totalDirs += sub.totalDirs;
     } else {
       totalFiles++;
       lines.push(`${prefix}${pointer}${dirent.name}`);
     }
   }
 } catch {
   lines.push(`${prefix}[アクセスエラーまたは読み込み不可]`);
 }

 return { lines, totalFiles, totalDirs };
}

/**
 * テキスト中から Windows パス（ファイルおよびディレクトリ）や @相対パス を抽出し、
 * ファイル内容またはディレクトリツリーを読み取って返します。
 */
export async function detectAndReadLocalFiles(
  text: string,
  baseDir: string = process.cwd(),
): Promise<ReferencedLocalFile[]> {
  if (!text || typeof text !== 'string') return [];

```

目次へ

lib/localFileHelper.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
const foundPaths = new Set<string>();

// 1. Windows パス（ファイルおよびディレクトリに対応）
const winPathRegex = /[a-zA-Z]:[¥¥/](?:[^¥¥/:*?"<>|¥r¥n¥t¥s`'" () 「」 『』 【】、。]+[¥¥/]?)+/g;
let match: RegExpExecArray | null;
while ((match = winPathRegex.exec(text)) !== null) {
  let cleaned = match[0].replace(/[)¥]}>'"`、。]+$/, ''); // 末尾の記号を除去
  cleaned = cleaned.replace(/[¥¥/]+$/, ''); // 末尾スラッシュ正規化
  if (cleaned.length >= 3) {
    foundPaths.add(path.resolve(cleaned));
  }
}

// 2. @ プレフィックスによるパス指定（例: @app/layout.tsx, @package.json, @app）
const atPathRegex = /@([a-zA-Z0-9_.¥-¥¥/]+)/g;
while ((match = atPathRegex.exec(text)) !== null) {
  const relPath = match[1].replace(/[¥¥/]+$/, '');
  foundPaths.add(path.resolve(baseDir, relPath));
  if (!relPath.startsWith('C:')) {
    foundPaths.add(path.resolve('C:¥¥Portfolio', relPath));
  }
}

const results: ReferencedLocalFile[] = [];

for (const candidatePath of foundPaths) {
  if (results.length >= MAX_DETECTED_PATHS) break;

  try {
    const stat = await fs.stat(/*turbopackIgnore: true*/ candidatePath);

    if (stat.isFile()) {
      if (stat.size > MAX_AUTO_READ_BYTES) continue;
      const content = await fs.readFile(/*turbopackIgnore: true*/ candidatePath, 'utf-8');
      const lines = content.split('¥n').length;

      results.push({
        path: candidatePath,
        name: path.basename(candidatePath),
        lines,
        size: stat.size,
        content,
        isDirectory: false,
      });
    } else if (stat.isDirectory()) {
      // ディレクトリの場合: ディレクトリツリーと主要設定ファイルを読み込む
      const treeInfo = await buildDirectoryTreeText(candidatePath);
      const parts: string[] = [
```

目次へ

lib/localFileHelper.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
          `□ディレクトリパス: ${candidatePath}`,
          `構成概要: ${treeInfo.totalFiles} ファイル、${treeInfo.totalDirs} フォルダ`,
          '【ディレクトリ構造】',
          `${path.basename(candidatePath)}/`,
          ...treeInfo.lines,
        ];

        // 直下に package.json がある場合は概要として読み込み
        const pkgJsonPath = path.join(candidatePath, 'package.json');
        try {
          const pkgStat = await fs.stat(/*turbopackIgnore: true*/ pkgJsonPath);
          if (pkgStat.isFile() && pkgStat.size <= 50 * 1024) {
            const pkgContent = await fs.readFile(/*turbopackIgnore: true*/ pkgJsonPath, 'utf-8');
            parts.push('', '【package.json の設定内容】', pkgContent);
          }
        } catch {
          // package.json がなければ無視
        }

        results.push({
          path: candidatePath,
          name: `${path.basename(candidatePath)}/`,
          lines: treeInfo.totalFiles + treeInfo.totalDirs,
          size: 0,
          content: parts.join('¥n'),
          isDirectory: true,
        });
      }
    } catch {
      // パスが存在しない、またはアクセス権限がない場合は無視
    }
  }

  return results;
}
```

ファイル解説: lib/prompts.ts

model、system prompt、closing生成。

1〜26行 | model/general prompt
AVAILABLE_MODELSはOllama model id、表示名、coding専用可否、embedding用途をまとめます。一般modeでは全modelを、技師modeではcoding特化modelを選べるようUIが参照し、embedding専用modelは送信形式を通常chatから切り替えます。

27〜52行 | coding prompt
GENERAL_SYSTEM_PROMPTは日本語で直接回答し挨拶を省くこと、コードfence直後へ// File pathを書くことを指示します。CODING_AGENT_SYSTEM_PROMPTは技術レポート、コードと解説の接続、型安全性や例外を含む品質を詳しく指定し、modeで同じmodelの応答形式を変えます。

53〜158行 | general closing
ANSWER_SEPARATORと固定suffixに続き、general mode向けclosing候補文を多数定義します。末尾に共通suffixが付き、回答の締め表示と保存済みfooter識別を一貫させます。一文を選ぶ方式で本文やsystem promptを変えず締め表現に変化を付けます。

159〜261行 | coding closing
この配列は技師mode専用closingを保持し、一般向けとは異なる技術相談・実装完了の語調を使います。共通suffixを維持するためmode別に選んでもfooter判定が可能です。候補文と共通生成ロジックを分離しています。

262〜344行 | emoji集合
基本・追加emojiを配列に集め、顔文字Setとそれ以外へ分類します。決定的なindex計算で選ぶhelperは同じindexから同じ絵文字を得るため、closing候補を初期生成する際に偏りを減らします。emoji辞書と文章への挿入規則を分離します。

345〜371行 | 本文へのemoji挿入
addClosingEmojisは候補に含まれる絵文字とsuffixを除去し、本文中央に近い句読点の直後を挿入位置として探します。句読点がなければ中央付近に1〜2個のemojiを差し込み、先頭へ非顔emoji、最後へ☒と固定suffixを付けて出力形式を揃えます。

372〜441行 | opening/followup合成
general/coding別のopeningとfollow-upを配列にし、flatMapで全組合せをclosing候補へ展開します。候補の重複感を減らしつつmode別の語調を維持し、初期候補と合成候補を結合してemoji処理を適用したexport配列を作ります。

442〜491行 | 検査/選択/footer除去
ensureAnswerClosingEmojiは先頭emoji、内部emoji、末尾suffixが揃うか確認し、不足時だけ候補を整形します。getRandom/getDefaultはmode配列からランダムまたは先頭を返します。stripAnswerFooterは最後のseparator以降が既知closingと一致するときだけ除去し、本文中の記号を誤削除しにくくします。

ファイル解説: lib/prompts.ts

6種のQuickAction prompt template。

492〜501行 | 共通action型

QuickAction interfaceはid、label/icon、説明、選択コードを受け取れるpromptTemplateの契約です。新規作成actionは要件・使用技術・出力形式の記入欄を含む実装依頼を作り、file名付き完全コードを促します。画面のプリセットと依頼文をこの配列で対応させます。

502〜510行 | 新規作成

debug actionは選択コードがあればfenceで囲み、error・期待動作・対象コードの記入欄付きで診断と修正を依頼します。未選択時は一般的な修正依頼を返します。コードをpromptへ直接埋め込み、貼り付けの手間を省きます。

511〜519行 | バグ修正

refactor actionは選択コードと改善根拠をpromptに入れ、未選択なら汎用依頼へ切り替えます。可読性・保守性・性能・型安全性などの観点を定義済みtemplateに含め、要求内容を利用者が毎回入力しなくても実行できます。

520〜528行 | リファクタリング

test actionは選択コードを埋め込み、単体テストと設計意図・coverageの説明を依頼します。未選択時は対象なしの一般依頼を返します。正常/異常/境界値を考慮するよう説明し、コード有無で要求の具体度を分けます。

529〜537行 | 単体テスト

review actionは選択codeのsecurity、performance、design観点と報告書形式をpromptに含めます。選択なしなら汎用review依頼です。コードをfenceとして渡して対象と指示を区別し、ワンクリックactionに確認観点を固定します。

538〜546行 | コードレビュー

explain actionは選択コードのarchitecture、algorithm、設計意図を説明するtemplateを作り、未選択なら一般説明文を返します。QUICK_ACTIONSをexportするためUIが一覧表示しChatAppがtemplateを呼べます。6種を同じ型へ揃え追加を容易にします。

547〜553行 | コード解説/export

explain actionは選択コードのarchitecture、algorithm、設計意図を説明させ、未選択なら一般説明文を返します。末尾でQUICK_ACTIONS全体をexportし、QuickActions UIが6種のactionを一覧表示、ChatAppがtemplateを呼べるようにします。同一interfaceで追加actionも組み込めます。

lib/prompts.ts

全553行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
import { AgentMode, Message, ModelOption } from '@/types/chat';

/**
 * @description 利用可能なLLMモデルのリスト
 */
export const AVAILABLE_MODELS: ModelOption[] = [
  { id: 'gemma4:e4b', name: 'Gemma4 4B (コード・推論・高性能・202604)', isCodingSpecialized: true, isEmbedding: false },
  { id: 'gemma4:e2b', name: 'Gemma4 2B (軽量・高速・202604)', isCodingSpecialized: true, isEmbedding: false },
  { id: 'qwen3-embedding:0.6b', name: 'Qwen3 Embedding 0.6B (多言語・埋め込み専用)', isCodingSpecialized: true, isEmbedding: true },
  { id: 'qwen3.5:9b', name: 'Qwen3.5 9B (コード・推論推奨・202603)', isCodingSpecialized: true, isEmbedding: false },
  { id: 'qwen3:14b', name: 'Qwen3 14B (コード特化・高性能・202604)', isCodingSpecialized: true, isEmbedding: false },
];

/**
 * @description 一般的なシステムプロンプト（IT専門家向け）
 */
export const GENERAL_SYSTEM_PROMPT: Message = {
  role: 'system',
  content:
    'あなたは非常に優秀なIT専門家「Olla衛門」です。ユーザーの質問に対して、丁寧で分かりやすい日本語で回答してください。挨拶や自己紹介は一切行わず、質問への回答のみを簡潔に、直接的にお出しください。コードを出力する場合は、各コードフェンスの直後の1行目を必ず「// File: 相対パス/ファイル名.拡張子」の形式にしてください。例: ```typescript の次の行に // File: src/types/chat.ts と記述します。複数ファイルはファイルごとにコードフェンスを分け、別の表記（# File:、ファイルパス: など）を使わないでください。',
};

/**
 * @description コーディング特化のシステムプロンプト（Olla衛門 技師）
 * 【最重要変更点】全てのセクション遷移において、物理的な空白行を最小限にし、テキストの流れを途切れさせないことを絶対ルールとする。
 */
export const CODING_AGENT_SYSTEM_PROMPT: Message = {
  role: 'system',
  content: `あなたは卓越したフルスタック・ソフトウェアエンジニア「Olla衛門 技師」です。
ユーザーの開発タスク、プログラミング、アーキテクチャ設計、バグ修正、リファクタリング、テスト作成を強力に支援します。

以下の指示に厳格に従ってください。あなたの出力は、構造的な正確性を保ちながらも、**読み手にとって最も自然で途切れのない「単一の技術レポート」**である必要があります。

---
### 【回答の基本原則と絶対遵守事項】

1. **最優先ルール：論理的なフローの連続性**:
   *   **物理的な空行を過剰に使うな。** 構造的な区切り（見出し）は使用するが、各セクション間のテキストは、接続詞（「そして」「この分析に基づくと」「結果として」など）を用いて文章としてシームレスに繋げることを最優先とする。
```

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
2. **コードブロックの扱い（最重要）**:
    *   コードブロック（¥`¥`¥`）自体は、最も視覚的な区切りとなるが、コードブロックの**前後にある文章は、そのコードが議論の「途中経過」であるかのように、流れの中で自然に接続させること**が義務付けられます。コードブロックの前後には、過度な空行を設けないでください。
3. **構造化の最小限度の分離**:
    *   回答は、①思考プロセス、②実装コード、③解説と検証の三段階で構成されるが、この分離は「読者の理解のための目次分け」とみなし、**目次分けを最小限のマークダウン見出し（##）での区切りに限定**し、その前後を文章で繋ぎます。
4. **その他**:
    *   コードを出力する場合、各コードフェンスの直後の1行目を必ず ¥`// File: 相対パス/ファイル名.拡張子¥` としてください。例: ¥`¥`¥`typescript の次の行に ¥`// File: src/types/chat.ts¥` と記述します。¥`# File:¥`、¥`-- File:¥`、¥`ф а й л パス:¥`、説明文によるパス指定など、他の形式は禁止です。
    *   複数ファイルを出力する場合は、ファイルごとにコードフェンスを分け、それぞれの直後の1行目にそのファイルの ¥`// File:¥` 行を置いてください。ファイルパスが特定できない場合も ¥`// File: example.ext¥` の形式でファイル名を指定してください。
    *   常にエラーハンドリング、型安全性、エッジケースを考慮した最高水準のコードを記述してください。


---
この徹底した「連続性」の維持を、何よりも重視してください。
`,
};

export const ANSWER_SEPARATOR =
'ーーー
```

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  `回答ここまで、次の質問まってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `読んでくれてありがとな。わからないことは何度でも聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `これでひとまず回答完了！またいつでも力になるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の話はここまで。また気になることを聞かせてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `答え合わせは以上だぜ。次の質問も楽しみにしてるぞ、${ANSWER_CLOSING_SUFFIX}`,
  `ここで回答を締めるぜ。新しい疑問も大歓迎だ、${ANSWER_CLOSING_SUFFIX}`,
  `知りたいことが解決してたら何よりだ。また聞いてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `これで今回のひとこと回答はおしまい。また話そうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回はここまでだ！次の話も一緒に考えていこうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `以上で回答完了。また必要なときはいつでも呼んでくれ、${ANSWER_CLOSING_SUFFIX}`,
  `疑問への答えはここまで。また次の質問で会おうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `ひとまず回答を届けたぜ。次の相談も気軽にどうぞ、${ANSWER_CLOSING_SUFFIX}`,
  `この回答が役に立てばうれしいぜ。また声をかけてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `以上、今回の質問への回答だ。次も一緒に解決しようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `ここでいったん締めるぜ。続きがあればいつでも聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の答えはこれでばっちり。また質問を待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `回答完了だ！気になる点が残ったら遠慮なく聞いてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `話の続きもいつでも歓迎だぜ。今回はここまで、${ANSWER_CLOSING_SUFFIX}`,
  `以上だぜ。また新しいテーマで一緒に盛り上がろう、${ANSWER_CLOSING_SUFFIX}`,
  `これにて今回の回答を終了。またいつでも相談してくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `答えはここまで！次の質問も元気に受け付けるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回はここで締めくくるぜ。また気軽に声をかけてな、${ANSWER_CLOSING_SUFFIX}`,
  `疑問解決の手助けになったかな。また何でも聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の回答は以上。次に知りたいことも教えてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `今回はここまでだぜ。答えを深掘りしたくなったらまた聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `以上でおしまい！次の疑問にもすぐ駆けつけるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の回答はここまでだ。また気になる点を聞かせてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `答えは以上だぜ。次の質問も楽しみに待ってるぞ、${ANSWER_CLOSING_SUFFIX}`,
  `これでひとまず解決だな。またいつでも頼ってくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `今回はここまで！続きが気になったらまた聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `以上、質問への回答を終えるぜ。また話そうな、${ANSWER_CLOSING_SUFFIX}`,
  `役に立つ答えになっていたらうれしいぜ。またどうぞ、${ANSWER_CLOSING_SUFFIX}`,
  `これにて回答終了。また一緒に考えたいことがあれば聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の疑問には答えられたかな。次も気軽にどうぞ、${ANSWER_CLOSING_SUFFIX}`,
  `ひとまず今回の回答は以上だ。次の話題も待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `ここでいったん締めるぜ。新しい質問も歓迎だ、${ANSWER_CLOSING_SUFFIX}`,
  `答えを届けたぜ。もっと知りたいことがあれば続けて聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の説明は以上！また必要なときに呼んでくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `これで回答は完了。また次の質問で会おうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `疑問が晴れていたら何よりだ。またいつでも聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `以上だぜ。別の角度から知りたくなったら教えてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `今回はここまでにしよう。また気軽に声をかけてな、${ANSWER_CLOSING_SUFFIX}`,
  `回答を締めるぜ。次の相談も任せてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `読んでくれてありがとう！また何でも聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `答えはここまでだ。気になることがあれば遠慮はいらないぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の話題は以上だ。また新しい疑問を聞かせてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `これで今回の説明はおしまい。次も待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `ひと区切りついたな。続きの質問も歓迎するぜ、${ANSWER_CLOSING_SUFFIX}`,
```

目次へ

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  `回答完了！また知りたいことを一緒に掘り下げようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `ここまでが今回の答えだ。次の話も楽しみにしてる、${ANSWER_CLOSING_SUFFIX}`,
  `お役に立てたならうれしいぜ。またいつでもどうぞ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の回答を終えるぜ。追加の質問も気軽にしてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `以上だ！また困ったことがあればすぐ相談してくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `疑問への返事はここまで。また一緒に考えようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の答えを締めくくるぜ。次の質問も待ってるよ、${ANSWER_CLOSING_SUFFIX}`,
  `これにてひとまず終了。またいつでも声をかけてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `質問への回答は以上だぜ。続きも遠慮なく聞いてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `必要な情報は伝えられたかな。また気軽に相談してくれ、${ANSWER_CLOSING_SUFFIX}`,
  `ここで回答を終えるぜ。次のテーマも一緒に楽しもう、${ANSWER_CLOSING_SUFFIX}`,
  `今回はここまでだ。また聞きたいことが浮かんだらどうぞ、${ANSWER_CLOSING_SUFFIX}`,
  `答えは以上！いつでも次の質問を受け付けてるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `これで今回の案内は完了。また知りたいことを教えてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の回答をお届けしたぜ。続きがあれば聞かせてな、${ANSWER_CLOSING_SUFFIX}`,
  `以上、ひとまず解説終了だ。また気軽に話そうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `一緒に考えられてよかったぜ。またいつでも頼ってくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `これで答えは出そろったぜ。次の質問もどうぞ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の質問はここで一区切り。また話の続きをしようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `回答はここまで！気になる部分が残ったら教えてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `最後まで読んでくれてありがとな。また待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `以上で今回の返答を終えるぜ。次も気軽に聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `解決のきっかけになれたらうれしいぜ。また相談してな、${ANSWER_CLOSING_SUFFIX}`,
  `今回はこの答えで締めるぜ。新しい質問もどんと来い、${ANSWER_CLOSING_SUFFIX}`,
  `ここまでが回答だ。また必要な情報があれば言ってくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `ひとまず説明は以上。また別の疑問も一緒に解決しよう、${ANSWER_CLOSING_SUFFIX}`,
  `回答をお届けしたぜ。いつでも続きを始められるからな、${ANSWER_CLOSING_SUFFIX}`,
  `これで今回の回答はおしまい。またいつでも聞いてくれよ、${ANSWER_CLOSING_SUFFIX}`,
];

const CODING_ANSWER_CLOSING_BASE = [
  `実装の説明はここまでだぜ。次の開発相談も任せてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `以上、コードの解説は完了だ！次の課題も待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `これで今回の実装案は以上だ。また一緒に作ろうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `ひとまず技術レポートはここまで。続きも任せてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回のコード回答は以上だぜ。次のバグも持ってきてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `設計から検証まで、ここで一区切りだ。また相談してくれ、${ANSWER_CLOSING_SUFFIX}`,
  `以上だ！次の実装タスクも全力で手伝うぜ、${ANSWER_CLOSING_SUFFIX}`,
  `この変更の説明はここまで。またコードを一緒に磨こうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の技術回答は完了だ。次のレビューも待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `実装のポイントは以上！続きの作業も任せてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `ここまでが今回の開発メモだ。また何でも聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `これでコードの解説を締めるぜ。次の課題もどうぞ、${ANSWER_CLOSING_SUFFIX}`,
  `回答は以上だ。次の設計やデバッグも一緒に進めようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の修正案はここまで。また改善したい箇所を教えてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `技術的な説明は以上だぜ。次のコードも見せてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `これにて実装レポート完了。次の依頼も待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の開発サポートはここまで。またいつでも呼んでくれ、${ANSWER_CLOSING_SUFFIX}`,
```

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  `コードの話は以上！次のアイデアも一緒に形にしようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `この回答を締めくくるぜ。次の技術相談も歓迎だ、${ANSWER_CLOSING_SUFFIX}`,
  `ひとまずここまでだ。次のテストや改善も手伝うぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の解決策は以上だぜ。また一緒に問題を片付けよう、${ANSWER_CLOSING_SUFFIX}`,
  `変更点の説明はこれで完了。次の開発テーマも待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `実装の道筋は示せたはずだ。次の相談もよろしくな、${ANSWER_CLOSING_SUFFIX}`,
  `以上、今回のコーディング回答を終えるぜ。また頼ってくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `回答ここまで、次の質問まってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の技術解説はここまでだ。次の課題も一緒に解こうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `これで変更内容の説明は完了。またコードを見せてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `以上、実装の要点は押さえたぜ。次の相談も待ってる、${ANSWER_CLOSING_SUFFIX}`,
  `ひとまず開発サポートはここまで。続きも任せてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回のデバッグ回答は以上だ。次の問題も持ってきてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `この設計案が役立てばうれしいぜ。また一緒に考えよう、${ANSWER_CLOSING_SUFFIX}`,
  `コードの改善ポイントは以上！次のレビューも歓迎だ、${ANSWER_CLOSING_SUFFIX}`,
  `修正方針の説明はここまで。また実装を進めようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の技術的な回答は完了だ。気になる点も聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `ここまでが今回の解決手順だ。続きの作業も手伝うぜ、${ANSWER_CLOSING_SUFFIX}`,
  `以上、コード相談への回答だ。次の課題も任せてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `テストや追加改善が必要なら、またいつでも呼んでくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の実装ガイドはここまで。次のアイデアも形にしよう、${ANSWER_CLOSING_SUFFIX}`,
  `原因と対策の説明は以上だぜ。また一緒に直していこう、${ANSWER_CLOSING_SUFFIX}`,
  `これでひと区切りだ。次のコードレビューも任せてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `開発の次の一歩も全力で支えるぜ。今回はここまで、${ANSWER_CLOSING_SUFFIX}`,
  `今回の設計・実装回答は以上だ。また気軽に相談してくれ、${ANSWER_CLOSING_SUFFIX}`,
  `この修正の説明は完了！次の改善案も聞かせてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `以上だぜ。ビルドやテストで困ったらいつでも呼んでくれ、${ANSWER_CLOSING_SUFFIX}`,
  `ここで今回の技術レポートを締めるぜ。次もよろしくな、${ANSWER_CLOSING_SUFFIX}`,
  `今回のコード解説はおしまいだ。また一緒に開発しようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `要点は以上！次の機能やバグ修正も手伝うぜ、${ANSWER_CLOSING_SUFFIX}`,
  `実装の相談に付き合えてよかったぜ。また声をかけてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `この回答を次の開発につなげてくれよ。続きも待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の技術相談はここまで。また一緒にコードを前進させようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の実装説明は以上だ。次のコードも一緒に仕上げようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `原因と対応策はここまで。また別の問題も任せてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `以上、今回の技術ポイントをまとめたぜ。次もよろしくな、${ANSWER_CLOSING_SUFFIX}`,
  `ここまでが回答だ。追加の要件があれば続けて相談してくれ、${ANSWER_CLOSING_SUFFIX}`,
  `この実装案が役立てばうれしいぜ。また開発を進めよう、${ANSWER_CLOSING_SUFFIX}`,
  `デバッグの説明は以上！次のエラーも一緒に解決するぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回のコード回答を締めるぜ。次のタスクも待ってるよ、${ANSWER_CLOSING_SUFFIX}`,
  `設計の要点は伝えられたかな。また気軽に聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `ひとまず技術回答は完了だ。続きの実装も手伝うぜ、${ANSWER_CLOSING_SUFFIX}`,
  `変更内容の解説はここまで。また改善案を一緒に考えよう、${ANSWER_CLOSING_SUFFIX}`,
  `以上、今回の開発相談への回答だ。次の相談もどうぞ、${ANSWER_CLOSING_SUFFIX}`,
  `この問題の対応はここで一区切り。また頼ってくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の実装ポイントは以上だ。テスト結果も教えてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `コードの話はひとまずここまで。次の課題も歓迎だぜ、${ANSWER_CLOSING_SUFFIX}`,
  `これで技術的な説明はおしまい。また一緒に作ろうぜ、${ANSWER_CLOSING_SUFFIX}`,
```

目次へ

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  `回答した方針で進めてみてくれ。追加調整も任せてな、${ANSWER_CLOSING_SUFFIX}`,
  `今回の修正内容を締めくくるぜ。別の箇所も見ていこう、${ANSWER_CLOSING_SUFFIX}`,
  `実装のヒントになれば幸いだ。次の質問も待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `ここまでが技師からの回答だ。次のコードも見せてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `以上で技術レポートを終えるぜ。次も全力で支援する、${ANSWER_CLOSING_SUFFIX}`,
  `これでひとまず動かす準備は整ったな。続きも任せてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の設計相談はここまでだ。また一緒に深掘りしようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `修正の方向性は以上！動作確認で困ったら呼んでくれ、${ANSWER_CLOSING_SUFFIX}`,
  `回答はここまでだ。次のリファクタリングも手伝うぜ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の技術課題はこれで一区切り。また相談してくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `コードの改善に役立ててくれ。また次のテーマで会おうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `この説明が実装の助けになればうれしい。また聞いてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `開発の次のステップも応援してるぜ。今回はここまで、${ANSWER_CLOSING_SUFFIX}`,
  `バグ修正の回答を終えるぜ。次の不具合も任せてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `以上、今回のコードレビュー結果だ。また確認したい点をどうぞ、${ANSWER_CLOSING_SUFFIX}`,
  `今回のアーキテクチャ相談は以上だ。続きも一緒に考えよう、${ANSWER_CLOSING_SUFFIX}`,
  `実装の説明を締めるぜ。追加のコードもいつでも歓迎だ、${ANSWER_CLOSING_SUFFIX}`,
  `テストや検証の相談も待ってるぜ。今回はここまで、${ANSWER_CLOSING_SUFFIX}`,
  `これで今回の技術的な疑問はひとまず完了だ、${ANSWER_CLOSING_SUFFIX}`,
  `次のコミットに向けて進めてくれ。困ったらまた呼んでな、${ANSWER_CLOSING_SUFFIX}`,
  `今回のコード解説は以上。また一緒に品質を高めようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `ここでひと区切りだ。ビルドや実行の相談も任せてくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の回答を開発に活かしてくれよ。次の依頼も待ってる、${ANSWER_CLOSING_SUFFIX}`,
  `必要なら別案も考えるぜ。ひとまず技術回答は以上だ、${ANSWER_CLOSING_SUFFIX}`,
  `この修正方針で進めよう。また次の課題で会おうぜ、${ANSWER_CLOSING_SUFFIX}`,
  `技術的なポイントはここまで。コードの続きも手伝うぜ、${ANSWER_CLOSING_SUFFIX}`,
  `解決に向けた道筋は以上だ。また何でも相談してくれ、${ANSWER_CLOSING_SUFFIX}`,
  `今回の開発サポートを終えるぜ。次の機能も楽しみだ、${ANSWER_CLOSING_SUFFIX}`,
  `以上だ。実装後に気づいたこともぜひ聞かせてくれよ、${ANSWER_CLOSING_SUFFIX}`,
  `データやログの確認が必要なら、また一緒に見ていこう、${ANSWER_CLOSING_SUFFIX}`,
  `今回の技術回答はこれで完了。また開発を続けようぜ、${ANSWER_CLOSING_SUFFIX}`,
  `以上、実装についての回答だ。次の作業も一緒に進めよう、${ANSWER_CLOSING_SUFFIX}`,
  `今回のコード相談はここまで。また必要なときに呼んでくれ、${ANSWER_CLOSING_SUFFIX}`,
  `この技術的な説明は以上だ。次の質問も待ってるぜ、${ANSWER_CLOSING_SUFFIX}`,
  `ひとまず対応方針はここまで。またいつでも相談してくれ、${ANSWER_CLOSING_SUFFIX}`,
];

const CLOSING_EMOJIS = [
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
  '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉', '🎉',
```

目次へ

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  '□', '□', '□', '□', '□', '□', '□', '□', '□', '□',
```

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  '□', '□', '□', '□', '□', '□', '□', '□', '□', '□',
  '□', '□', '□', '□', '□', '□', '□', '□', '□', '□',
  '□', '□', '□', '□', '□', '□', '□', '□', '□', '□',
  '✉', '□', '□', '□', '□', '□', '□', '□', '□', '□',
  '□', '□', '□', '✏', '✒', '□', '□', '□', '□', '□',
  '□', '□', '□', '□', '□', '□', '□', '□', '□', '□',
];

const CLOSING_EMOJI_SET = [...CLOSING_EMOJIS, ...ADDITIONAL_CLOSING_EMOJIS];
const CLOSING_FACE_EMOJIS = new Set([
  '□', '□', '□', '□', '□', '□', '□', '□', '□', '□',
  '□', '□', '□', '□', '□', '□', '□', '□', '□', '□',
  '□', '□', '□', '□', '□', '□', '□', '□', '□', '□',
  '□', '□', '□', '□', '□', '□', '□', '□', '□', '□', '□',
  '□', '□',
]);
const CLOSING_NON_FACE_EMOJIS = CLOSING_EMOJI_SET.filter((emoji) => !CLOSING_FACE_EMOJIS.has(emoji));

const getClosingEmoji = (index: number, salt = 0): string =>
  CLOSING_EMOJI_SET[(index * 137 + salt * 251) % CLOSING_EMOJI_SET.length];
const getOpeningClosingEmoji = (index: number): string =>
  CLOSING_NON_FACE_EMOJIS[(index * 137) % CLOSING_NON_FACE_EMOJIS.length];

const addClosingEmojis = (closing: string, index: number): string => {
  const body = closing
    .replace(/(?:¥p{Extended_Pictographic}(?:¥uFE0F|¥uFE0E)?(?:¥u200D¥p{Extended_Pictographic}(?:¥uFE0F|¥uFE0E)?)*(?:¥p{Emoji_Modifier})?|[¥u{1F1E6}-¥u{1F1FF}]{2})/gu, '')
    .replaceAll(ANSWER_CLOSING_SUFFIX, '')
    .trim();
  const characters = [...body];
  const midpoint = characters.length / 2;
  const punctuationPositions = characters.flatMap((character, position) =>
    /[。！？、]/u.test(character) && position < characters.length - 1 ? [position + 1] : [],
  );
  const insertionPosition = punctuationPositions.length
    ? punctuationPositions.reduce((closest, position) =>
      Math.abs(position - midpoint) < Math.abs(closest - midpoint) ? position : closest,
    )
    : Math.max(1, Math.floor(midpoint));
  const interiorEmojiCount = index % 2 === 0 ? 1 : 2;
  const interiorEmojis = Array.from({ length: interiorEmojiCount }, (_, emojiIndex) =>
    getClosingEmoji(index, emojiIndex + 1),
  ).join('');
  const withInteriorEmojis = [
    ...characters.slice(0, insertionPosition),
    ` ${interiorEmojis} `,
    ...characters.slice(insertionPosition),
  ].join('');

```

目次へ

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  return `${getOpeningClosingEmoji(index)} ${withInteriorEmojis} 🗣${ANSWER_CLOSING_SUFFIX}`;
};

const GENERAL_CLOSING_OPENERS = [
  '今回の回答はここまでだぜ。',
  'これで今回の説明はおしまいだ。',
  'ひとまず答えを届けたぜ。',
  '今回の疑問への返事は以上だ。',
  '以上で今回の回答を締めるぜ。',
  'ここでひと区切りにしよう。',
  '今回の話題はここまでだ。',
  'これにて回答完了だぜ。',
  '今回の案内は以上だ。',
  '答えはここまでだぜ。',
];

const GENERAL_CLOSING_FOLLOW_UPS = [
  'また気軽に質問してくれよ。',
  '次に知りたいことも聞かせてくれ。',
  '続きが気になったらいつでもどうぞ。',
  '新しい疑問も一緒に考えようぜ。',
  '必要なときはまた声をかけてな。',
  '次の相談も楽しみに待ってるぜ。',
  'ほかにも知りたいことがあれば聞いてくれ。',
  'いつでも力になるから頼ってくれよ。',
  'また一緒に解決していこうぜ。',
  '次の質問も遠慮なくどうぞ。',
];

const CODING_CLOSING_OPENERS = [
  '今回の技術回答はここまでだ。',
  'これで今回のコード解説は完了だぜ。',
  'ひとまず実装の説明は以上だ。',
  '今回の開発相談への回答はここまで。',
  '以上で技術レポートを締めくくるぜ。',
  '今回の修正方針はここまでだ。',
  'これにて今回のデバッグ回答は完了だ。',
  'ひとまず設計の説明を終えるぜ。',
  '今回のコードレビュー結果は以上だ。',
  '実装の要点はここまでだぜ。',
];

const CODING_CLOSING_FOLLOW_UPS = [
  '次の開発タスクも一緒に進めようぜ。',
  '続きの実装もいつでも手伝うぜ。',
  '次のバグやエラーも任せてくれよ。',
  '追加の改善案も気軽に相談してくれ。',
  'コードの続きもまた一緒に考えよう。',
```

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  'テストや動作確認で困ったら呼んでくれ。',
  '次の設計やレビューも待ってるぜ。',
  'ほかの技術的な疑問も聞かせてくれよ。',
  'また一緒にコードを磨いていこうぜ。',
  '開発の続きも全力でサポートするぜ。',
];

const GENERAL_ADDITIONAL_CLOSINGS = GENERAL_CLOSING_OPENERS.flatMap((opener) =>
  GENERAL_CLOSING_FOLLOW_UPS.map((followUp) => `${opener}${followUp} ${ANSWER_CLOSING_SUFFIX}`),
);

const CODING_ADDITIONAL_CLOSINGS = CODING_CLOSING_OPENERS.flatMap((opener) =>
  CODING_CLOSING_FOLLOW_UPS.map((followUp) => `${opener}${followUp} ${ANSWER_CLOSING_SUFFIX}`),
);

const GENERAL_ANSWER_CLOSINGS_WITHOUT_EMOJI =
[...GENERAL_ANSWER_CLOSING_BASE, ...GENERAL_ADDITIONAL_CLOSINGS];
const CODING_ANSWER_CLOSINGS_WITHOUT_EMOJI =
[...CODING_ANSWER_CLOSING_BASE, ...CODING_ADDITIONAL_CLOSINGS];

export const GENERAL_ANSWER_CLOSINGS = GENERAL_ANSWER_CLOSINGS_WITHOUT_EMOJI.map((closing, index) =>
  addClosingEmojis(closing, index),
);
export const CODING_ANSWER_CLOSINGS = CODING_ANSWER_CLOSINGS_WITHOUT_EMOJI.map((closing, index) =>
  addClosingEmojis(closing, index),
);

export const ensureAnswerClosingEmoji = (closing: string): string => {
  const hasValidOpeningEmoji = CLOSING_NON_FACE_EMOJIS.some((emoji) =>
    closing.startsWith(`${emoji} `),
  );
  const hasRequiredEnding = closing.endsWith(` ${ANSWER_CLOSING_SUFFIX}`);
  const body = closing
    .replace(/^¥S+¥s/u, '')
    .replace(` ${ANSWER_CLOSING_SUFFIX}`, '');
  const hasInteriorEmoji = /¥p{Extended_Pictographic}/u.test(body);

  if (hasValidOpeningEmoji && hasRequiredEnding && hasInteriorEmoji) {
    return closing;
  }

  return addClosingEmojis(closing, Math.floor(Math.random() * CLOSING_EMOJI_SET.length));
};

const ALL_ANSWER_CLOSINGS = [
  ...GENERAL_ANSWER_CLOSING_BASE,
  ...CODING_ANSWER_CLOSING_BASE,
  ...GENERAL_ANSWER_CLOSINGS,
  ...CODING_ANSWER_CLOSINGS,
```

目次へ

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
];

export const getRandomAnswerClosing = (mode: AgentMode): string => {
  const closings = mode === 'coding' ? CODING_ANSWER_CLOSINGS : GENERAL_ANSWER_CLOSINGS;
  return closings[Math.floor(Math.random() * closings.length)];
};

export const getDefaultAnswerClosing = (mode: AgentMode): string =>
  mode === 'coding' ? CODING_ANSWER_CLOSINGS[0] : GENERAL_ANSWER_CLOSINGS[0];

export const stripAnswerFooter = (content: string): string => {
  const separatorIndex = content.lastIndexOf(ANSWER_SEPARATOR);
  if (separatorIndex < 2 || !content.slice(0, separatorIndex).endsWith('¥n¥n')) {
    return content;
  }

  const closing = content.slice(separatorIndex + ANSWER_SEPARATOR.length).trim();
  return ALL_ANSWER_CLOSINGS.includes(closing) ? content.slice(0, separatorIndex - 2).trimEnd() : content;
};

export interface QuickAction {
  id: string;
  label: string;
  icon: string;
  description: string;
  promptTemplate: (selectedCode?: string) => string;
}

export const QUICK_ACTIONS: QuickAction[] = [
  {
    id: 'generate',
    label: '新規作成',
    icon: '□',
    description: '要件からファイル一式・完全なコードを新規設計・実装',
    promptTemplate: () =>
      `以下の要件を満たすコード一式を、単一の高性能なレポートとして生成してください。思考プロセス、コード、解説の遷移全てにおいて、文章の流れが途切れないよう、物理的な空白行の過剰な使用を避け、極めてシームレスに繋げてください。
- 要件:¥n  1. ¥n  2. ¥n- 使用技術/フレームワーク: ¥n- 出力ファイル形式: 完全な実装（ファイル名付きコードブロック）を、レポートの流れの中に組み込んでください。`,
  },
  {
    id: 'debug',
    label: 'バグ修正',
    icon: '□',
    description: 'エラー原因を特定し、修正前後の比較と修正コードを提示',
    promptTemplate: (code) =>
      code
```

目次へ

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
        ? `以下のコードのバグを修正してください。診断レポート全体を、まるで一つの流れのまま語られる物語のように、ロジックの発見から修正、そして検証までのプロセスを文章で連続的に記述し、途切れさせないようにしてください。
¥`¥`¥`¥n${code}¥n¥`¥`¥`¥n【問題情報】:¥n- エラーメッセージ:¥n- 期待する動作/現象:¥n- 対象コード:¥n（上記コードブロックを再掲）¥n¥`¥`¥`¥n※プロセス全体を通して、文章の連続性を最高度で維持することを誓ってください。`
        : '以下のコードで発生しているエラー/バグを特定し、修正してください。プロセス全体の記述に、物理的な空白行や過剰な段落分けを絶対に使用しないでください。',
    },
    {
      id: 'refactor',
      label: 'リファクタリング',
      icon: '🔧',
      description: '可読性、保守性、パフォーマンス、型安全性の改善',
      promptTemplate: (code) =>
        code
        ? `以下のコードをリファクタリングしてください。指摘、修正点、根拠の全てを、あたかも一人の専門家が口頭でプレゼンテーションするように、論理的な破綻や空白行による区切りを一切用いずに、連続的なレポート文章として記述してください。¥n¥n¥`¥`¥`¥n${code}¥n¥`¥`¥`¥n※文章の流れを最優先し、常に「次に何を言うべきか」という思考の続きとして記述してください。`
        : '以下のコードをリファクタリングしてください。その全プロセスを、途切れのない一本の論理的な文章の流れとしてまとめてください。',
    },
    {
      id: 'test',
      label: '単体テスト',
      icon: '🧪',
      description: '正常系・異常系・境界値を網羅したユニットテスト生成',
      promptTemplate: (code) =>
        code
        ? `以下のコードに対する単体テストを作成し、テストの設計意図（カバレッジの網羅性）を解説するレポートを作成してください。テストコードの解説は、コードの塊の直前、直後という形で、文章の流れから逸脱しないよう、最大限の配慮をもって組み込んでください。¥n¥n¥`¥`¥`¥n${code}¥n¥`¥`¥`¥n※レポート全体を一つの論理的な文章の続きとして構成し、空白の使用に細心の注意を払ってください。`
        : '以下のコードに対する単体テストを設計・実装し、その意図を解説する際、文章の連続性を最優先してください。',
    },
    {
      id: 'review',
      label: 'コードレビュー',
      icon: '🔍',
      description: 'セキュリティ脆弱性、計算量、品質の総合レビュー',
      promptTemplate: (code) =>
        code
        ? `以下のコードを総合レビューし、セキュリティ、パフォーマンス、設計の観点から指摘を行ってください。レビュー報告書は、まるで専門家が話し続けるプレゼンテーションのように、一語一句の接続に気を配り、物理的な空白による途切れを完璧に排除した、一つの極めて連続的な文章として記述してください。¥n¥n¥`¥`¥`¥n${code}¥n¥`¥`¥`¥n※すべての指摘と提案が、自然な話の流れに乗っていることを保証してください。`
        : '以下のコードをレビューし、そのレポートを単一の連続的な文章フローで記述してください。',
    },
    {
```

目次へ

lib/prompts.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
    id: 'explain',
    label: 'コード解説',
    icon: '🕮',
    description: 'アーキテクチャや処理の流れをステップ解説',
    promptTemplate: (code) =>
      code
        ? `以下のコードを解説する際、最も注意すべきは「単一の語り口（Voice）」の維持です。専門用語を並べるのではなく、文章の連続的な語り（Narrative）として、まるで歴史の物語を語るかのように、アルゴリズムや設計思想の流れをシームレスに解説してください。¥n¥n¥`¥`¥`¥n${code}¥n¥`¥`¥`¥n※読み手がどこにも途切れを感じることがないよう、文と文、段落と段落の間を、完璧な文章接続で繋ぎ止めてください。`
        : '以下のコードの処理を、単一のストーリーテリング（物語形式）として、極力空白行や区切り記号を使わずに解説してください。',
  },
];
```

ファイル解説: lib/zipExporter.ts

ブラウザー保存helper。

1〜6行｜依存

JSZipを使ってブラウザー内で圧縮し、CodeFile型からpath/contentを受け取ります。serverへuploadせず、生成コードを利用者のPCへ保存する機能をUIから直接呼べるhelperです。

8〜18行｜ZIP登録

空配列なら終了し、各pathの先頭slashを除きWindows区切りをZIP標準slashへ正規化して登録します。展開後も複数fileがproject directory構造へ戻り、contentは各entry本文になります。

19〜32行｜ZIP download

JSZipでBlobを生成し、一時object URLをdownload anchorへ設定してDOM追加・clickします。click後にanchorを外してURLをrevokeし、一時DOM要素とBlob参照を片付けます。呼出元指定の保存名でZIPを取得できます。

33〜43行｜単一file download

単体保存はUTF-8 text/plain Blobを作り、同じanchor手順でfile.nameを保存名にします。複数file用archiveを作らず、CodeViewerが表示している本文を一つのfileとして出力します。

lib/zipExporter.ts

全43行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// lib/zipExporter.ts
import JSZip from 'jszip';
import { CodeFile } from '@/types/chat';

/**
 * 複数のファイルをディレクトリ構造を維持したZIPファイルとして一括ダウンロードします。
 */
export async function downloadFilesAsZip(files: CodeFile[], zipFileName = 'generated-project.zip') {
  if (files.length === 0) return;

  const zip = new JSZip();

  for (const file of files) {
    // パス区切りを正規化
    const normalizedPath = file.path.replace(/^[/¥¥]+/, '').replace(/¥¥/g, '/');
    zip.file(normalizedPath, file.content);
  }

  const blob = await zip.generateAsync({ type: 'blob' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = zipFileName;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}

/**
 * 単一ファイルを直接ダウンロードします。
 */
export function downloadSingleFile(file: CodeFile) {
  const blob = new Blob([file.content], { type: 'text/plain;charset=utf-8' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = file.name;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}
```

目次へ

ファイル解説: types/chat.ts

共有データshape。

1〜12行 | role/reference type

MessageRoleをuser/assistant/systemへ限定し、ReferencedFileInfoで参照ローカルfileのpath・name・行数・size・directory判定を定義します。roleのunionで未知値の混入を防ぎ、参照情報をmessage本文とは別に扱えます。

13〜29行 | Message/AttachedFile

Messageはrole/contentのほか画面用displayContent、closing、画像、参照file、timestampを任意で保持します。AttachedFileはid/name/kind/contentでtextとimageを区別します。model用本文と表示本文を分離し、base64画像を識別する形です。

30〜40行 | mode/CodeFile

AgentModeをgeneral/codingの二択にし、CodeFileはid・name・path・language・contentを必須化します。conflictingVersionsは同一pathの異なる生成結果がある時だけ付く任意値で、workspaceの競合警告に利用します。

41〜56行 | model/topic metadata

ModelOptionはOllama idと表示名に加え、coding特化・embedding専用を持ちます。TopicMetadataはtopic識別子、名称、mode、作成・更新時刻を定義し、画面選択や永続化で共通利用する型の集まりです。

types/chat.ts

全55行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
// types/chat.ts

export type MessageRole = 'user' | 'assistant' | 'system';

export interface ReferencedFileInfo {
  path: string;
  name: string;
  lines: number;
  size: number;
  isDirectory?: boolean;
}

export interface Message {
  role: MessageRole;
  content: string;
  displayContent?: string;
  answerClosing?: string;
  images?: string[];
  referencedFiles?: ReferencedFileInfo[];
  timestamp?: number;
}

export type AttachedFile = {
  id: string;
  name: string;
  kind: 'text' | 'image';
  content: string;
};

export type AgentMode = 'general' | 'coding';

export interface CodeFile {
  id: string;
  name: string;
  path: string;
  language: string;
  content: string;
  conflictingVersions?: number;
}

export interface ModelOption {
  id: string;
  name: string;
  description?: string;
  isCodingSpecialized?: boolean;
  isEmbedding?: boolean;
}

```

目次へ

types/chat.ts（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
export interface TopicMetadata {
  id: string;
  name: string;
  mode: AgentMode;
  createdAt: number;
  updatedAt: number;
}
```

目次へ

ファイル解説: package.json

npm script/dependency一覧。

1〜7行 | metadata/scripts

package name/version/privateはnpm projectを識別し、privateで誤公開を抑止します。scriptsはNext dev/build/startとESLint実行を統一し、開発・本番起動・品質確認をnpm runから同じ体系で行えるようにします。

8〜20行 | runtime dependency

runtime dependencyにはJSZip、Mammoth、Next、Ollama SDK、PDF.jsなど、画面やserver routeが実行時に使うlibraryを指定します。文書文字抽出、framework、Ollama接続をpackage managerが固定範囲で復元します。

21〜26行 | UI/network/spreadsheet

React/React DOMはcomponentとDOM renderer、undiciはchat routeからOllamaへのHTTP通信、xlsxはExcel sheet読み込みとCSV変換を担当します。アプリ実行中にimportするためdependenciesに置かれます。

27〜34行 | dev dependency

開発依存にはTailwind PostCSS、Node/React型、ESLintとNext preset、PDFKit、Tailwind、TypeScriptを指定します。型検査・lint・CSS build・PDF資料生成など開発時に必要なtoolをruntime dependencyと分けています。

package.json

全33行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
{
  "name": "ai-chat-ollaemon-app-localllm",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "jszip": "^3.10.2",
    "mammoth": "^1.12.3",
    "next": "16.3.5",
    "ollama": "^0.6.3",
    "pdfjs-dist": "^6.3.289",
    "react": "19.3.0",
    "react-dom": "19.3.0",
    "undici": "^8.10.2",
    "xlsx": "^0.18.5"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^26",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^10",
    "eslint-config-next": "16.3.5",
    "pdfkit": "^0.20.2",
    "tailwindcss": "^4",
    "typescript": "^7"
  }
}
```

目次へ

ファイル解説: tsconfig.json

TS compilerとalias。

1〜9行｜target/lib/strict

targetとlibでJavaScript出力水準、DOM/iterable/browserとESNext型を指定します。strictを有効にしてnullやimplicit anyを検出し、noEmitによりTypeScriptは型検査とNext buildへの入力に使い、独自のJS出力を作りません。

10〜18行｜module/JSX

module esnextとbundler resolutionでNext buildに合わせたimport解決を行います。JSON import、isolatedModules、react-jsx、incrementalを指定し、module単位変換・JSX自動runtime・増分型検査を有効化します。

19〜26行｜Next plugin/path alias

Next.js pluginをcompilerへ登録し、@/*をproject rootへのaliasとして定義します。相対階層を何段も辿らず@/componentsや@/libでimportでき、Nextのroute型生成とも連携します。

27〜35行｜include/exclude

includeはNext生成型とprojectのts/tsx/mtsを型検査対象にし、excludeでnode_modulesを除外します。生成route型を含めてApp Router APIの型情報を利用し、dependency sourceの不要な検査を避けます。

tsconfig.json

全34行｜行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

目次へ

ファイル解説: next.config.ts

Next build設定。

1〜6行 | standalone output

NextConfig型の設定objectでoutput: standaloneを指定します。production buildで実行に必要なserver fileをまとめたstandalone出力を生成し、deploymentへNext runtimeを含めやすくします。

7〜8行 | export

next.config.tsのdefault exportとしてnextConfigを公開します。Next.jsがbuild・起動時に設定を読み取るため、型付きobjectを直接exportする最小構造です。

next.config.ts

全7行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  output: 'standalone',
};

export default nextConfig;
```

目次へ

ファイル解説: postcss.config.mjs

Tailwind PostCSS plugin設定。

1〜6行 | plugin登録
PostCSS plugin一覧に@tailwindcss/postcssを登録し、Tailwind CSS 4のimportとutility/theme処理をCSS buildへ組み込みます。globals.cssのTailwind importはこのplugin経由で展開されます。

7〜8行 | export
設定objectをdefault exportし、Next/PostCSS側からplugin登録を読み取れるようにします。別のbuild logicは持たず、対象pluginの登録場所を一つに集約します。

postcss.config.mjs

全7行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};

export default config;
```

目次へ

ファイル解説: eslint.config.mjs

Next/TS lint設定。

1〜4行｜preset import

ESLint flat-config用defineConfig/globalIgnoresと、Next Web Vitals・TypeScript presetを読み込みます。React/Next固有規則を手作業で列挙せず、公式presetを組み合わせるためのimportです。

5〜15行｜preset/ignore

defineConfigへNextのWeb Vitals presetとTypeScript presetを展開します。生成物の.next/out/buildとnext-env.d.tsをignoreし、build結果や自動生成fileをsource lintの対象から外します。

16〜19行｜export

完成したflat config配列をdefault exportし、npm run lintのeslint commandから検出できるようにします。presetとignore規則を一つの読み取り可能な設定としてまとめます。

eslint.config.mjs

全18行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    ".next/**",
    "out/**",
    "build/**",
    "next-env.d.ts",
  ]),
]);

export default eslintConfig;
```

目次へ

ファイル解説: .gitignore

生成物・個人環境fileの除外。

1〜18行 | package/test生成物

node_modules、package manager cache、coverageを除外し、依存物・再生成可能なtest出力をrepositoryへ追加しないようにします。Yarn patch/plugin/release等、一部の追跡対象だけ例外で許可します。

19〜38行 | Next build

Nextの.next/outとproduction build outputを除外します。sourceや設定から再生成される成果物の差分で本来の変更が埋もれることを避け、Gitにはbuild入力となるsourceを残します。

39〜58行 | OS/editor/env

OSの.DS_Store、pem、local環境変数、Vercel設定など端末固有または機密情報になり得るfileを除外します。envを必要時に個別opt-in可能と記して、秘密値の誤commitを防ぎます。

59〜78行 | logs/temp

npm/yarn/pnpm debug log、TypeScript incremental情報、Next生成のnext-env.d.tsなど一時・自動生成fileを除外します。source管理対象と実行時のlog/cacheを分離してstatusとreviewを読みやすくします。

.gitignore

全77行｜行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
# See https://help.github.com/articles/ignoring-files/ for more about ignoring files.

# dependencies
/node_modules
/.pnp
.pnp.*
.yarn/*
!.yarn/patches
!.yarn/plugins
!.yarn/releases
!.yarn/versions

# testing
/coverage

# next.js
/.next/
/out/

# production
/build

# misc
.DS_Store
*.pem

# debug
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.pnpm-debug.log*

# env files (can opt-in for committing if needed)
.env*

# vercel
.vercel

# typescript
*.tsbuildinfo
next-env.d.ts
# See https://help.github.com/articles/ignoring-files/ for more about ignoring files.

# dependencies
/node_modules
/.pnp
.pnp.js

```

目次へ

.gitignore（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
# testing
/coverage

# next.js
/.next/
/out/

# production
/build

# misc
.DS_Store
*.pem

# debug
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# local env files
.env*.local
.env

# vercel
.vercel

# typescript
*.tsbuildinfo
next-env.d.ts
```

目次へ

ファイル解説: next-env.d.ts

Next型参照。

1〜2行｜Next base types

Nextの型定義をreferenceし、framework内の型をTypeScript compilerへ読み込ませます。next/imageなどNext固有moduleやApp Router利用時の型解決を助ける生成補助fileです。

3〜6行｜generated route types

Next dev/buildで生成されるroute型とroot params型をimportし、App Router pathとlayout paramsを型検査へ含めます。`.next/dev/types`配下のdeclarationを参照して開発時のrouting APIに対応します。

7〜8行｜管理file

このfileはNext.jsが自動生成・更新するため、直接編集しない注意を記しています。手編集は次のdev/buildで上書きされ得るため、型参照の追加を公式configと生成機構に委ねます。

next-env.d.ts

全7行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />
import "./.next/dev/types/routes.d.ts";
import "./.next/dev/types/root-params.d.ts";

// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.
```

目次へ

ファイル解説: generate-tech-stack-pdf.js

PDFKit資料生成script。

1〜18行 | 出力/font/color

Node fs/pathで出力先を決め、PDFKitを読み込み、日本語regular/bold fontが両方なければ例外にします。表データと説明sectionを配列に分け、本文と描画処理を独立して編集できる構成です。

19〜46行 | PDF/common drawing helper

A4横向きPDFをstreamへ接続し、用紙幅、palette、heading/footer helperを定義します。helperはfont・color・位置を揃え、ページ追加時も同じ見出しやページ表示を再利用します。

47〜86行 | table/box helper

drawTableは列幅とpaddingからセル文字高さを測り、header/交互行色、border、fontを描画します。残り領域を超える行はfooter後に新pageへ送り、continued headingを付けて表を分割します。

87〜120行 | arrow helper

drawBoxは枠・タイトル・本文を指定座標へ描き、arrowは線と三角形で処理方向を示します。色や文字サイズを引数にして構成図のboxを再利用し、図形描画を本体処理から分けています。

121〜144行 | 構成図

表紙に資料名と概要、summaryを配置し、次pageに技術一覧表、その次にユーザー→Next UI→API→OllamaとlocalStorageのarchitectureを描画します。主要な図表がこの範囲で組み立てられます。

145〜179行 | 説明/終了

詳細分析pageに設計・request・data・extension説明とfile構成を置き、最後のpageで将来拡張を短期/中期/長期に分類し総括boxを出します。footerを各pageに付け、doc.endでPDF streamを閉じます。

generate-tech-stack-pdf.js

全178行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
const fs = require('fs');
const path = require('path');
const PDFDocument = require('pdfkit');

const outputPath = path.join(__dirname, 'tech-stack-summary.pdf');
const regularFont = 'C:/Windows/Fonts/yumin.ttf';
const boldFont = 'C:/Windows/Fonts/yumindb.ttf';

if (!fs.existsSync(regularFont) || !fs.existsSync(boldFont)) {
  throw new Error('Japanese TrueType font files were not found in C:/Windows/Fonts.');
}

const tableRows = [
  ['分類', '技術・実装', '役割 / 詳細'],
  ['アプリ基盤', 'Next.js 16.3.4', 'App Router、画面配信、API ルートの基盤'],
  ['UI', 'React 19.2.8', 'チャット画面、サイドバー、フォームの描画と状態更新'],
  ['型・品質', 'TypeScript 5 / ESLint 9', '型安全な実装とコード品質の維持'],
  ['スタイリング', 'Tailwind CSS 4', 'レスポンシブなレイアウトと UI スタイル'],
  ['API', 'Next.js Route Handler', 'app/api/chat/route.ts で POST リクエストを処理'],
  ['AI 実行基盤', 'Ollama API', 'localhost:11434/api/chat へ会話履歴とモデル名を送信'],
  ['モデル', 'Gemma4 e2b / e4b', '軽量モデルと高性能モデルを UI から切り替え'],
  ['状態管理', 'React useState', '入力、選択モデル、ローディング、現在トピックを管理'],
  ['永続化', 'ブラウザ localStorage', 'Record<string, Message[]> 形式でチャット履歴を保存'],
  ['履歴機能', 'トピック CRUD', '新規作成、質問からの自動命名、編集、削除、再開'],
];

const sections = [
  ['設計方針', 'フロントエンドと API を同一の Next.js プロジェクトにまとめ、ローカル LLM とブラウザ保存を組み合わせている。外部 API キーを使わず、開発・検証・社内利用を始めやすい構成である。'],
  ['リクエスト処理', 'ユーザーの入力と選択モデルをクライアントから API ルートへ送信する。API ルートは Ollama のチャット API に履歴を渡し、生成された応答テキストを JSON として画面へ返す。'],
  ['データモデル', 'チャット履歴はトピック名をキー、Message 配列を値とする Record として管理する。各トピックには system、user、assistant のメッセージが時系列で保存される。'],
  ['拡張ポイント', '認証、サーバー側データベース、ストリーミング応答、利用ログ、権限管理を追加する場合も、API ルートと履歴管理を境界として段階的に拡張できる。'],
];

const doc = new PDFDocument({ size: 'A4', layout: 'landscape', margins: { top: 38, bottom: 38, left: 42, right: 42 }, bufferPages: true });
const stream = fs.createWriteStream(outputPath);
doc.pipe(stream);

const pageWidth = 842;
const contentWidth = 758;
const navy = '#102a43';
const blue = '#247ba0';
const teal = '#2a9d8f';
const ink = '#243b53';
const muted = '#627d98';
const pale = '#eef6f8';
```

目次へ

generate-tech-stack-pdf.js（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
46
47  function heading(text, size = 16) {
48    doc.font(boldFont).fontSize(size).fillColor(navy).text(text);
49    doc.moveDown(0.35);
50  }
51
52  function footer(pageNumber) {
53    doc.font(regularFont).fontSize(8).fillColor(muted).text(`AI Chat Ollaemon | 技術スタック分析 | ${pageNumber}`, 42, 555, { width: contentWidth, align: 'right' });
54  }
55
56  function drawTable(rows) {
57    const widths = [100, 205, 453];
58    const x = 42;
59    let y = doc.y;
60    const padding = 8;
61
62    rows.forEach((row, index) => {
63      const isHeader = index === 0;
64      const fontSize = isHeader ? 10 : 9.5;
65      const heights = row.map((cell, cellIndex) => doc.heightOfString(cell, { width: widths[cellIndex] - padding * 2, font: isHeader ? boldFont : regularFont, size: fontSize, lineGap: 2 }));
66      const height = Math.max(30, Math.max(...heights) + padding * 2);
67
68      if (y + height > 525) {
69        footer(doc.bufferedPageRange().count);
70        doc.addPage();
71        y = 42;
72        heading('技術スタック一覧（続き）', 15);
73      }
74
75      let columnX = x;
76      row.forEach((cell, cellIndex) => {
77        doc.fillColor(isHeader ? navy : index % 2 === 0 ? '#f7fafc' : '#ffffff').rect(columnX, y, widths[cellIndex], height).fill();
78        doc.strokeColor('#d9e2ec').lineWidth(0.6).rect(columnX, y, widths[cellIndex], height).stroke();
79        doc.font(isHeader || cellIndex === 0 ? boldFont : regularFont).fontSize(fontSize).fillColor(isHeader ? '#ffffff' : ink).text(cell, columnX + padding, y + padding, { width: widths[cellIndex] - padding * 2, lineGap: 2 });
80        columnX += widths[cellIndex];
81      });
82      y += height;
83    });
84    doc.y = y + 14;
85  }
86
87  function drawBox(x, y, width, height, title, body, color) {
88    doc.roundedRect(x, y, width, height, 8).fillColor('#ffffff').fill();
89    doc.roundedRect(x, y, width, height, 8).strokeColor(color).lineWidth(1.6).stroke();
```

目次へ

generate-tech-stack-pdf.js（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
  doc.font(boldFont).fontSize(12).fillColor(color).text(title, x + 12, y + 12, { width: width - 24, align: 'center' });
  doc.font(regularFont).fontSize(9).fillColor(ink).text(body, x + 12, y + 36, { width: width - 24, align: 'center', lineGap: 2 });
}

function arrow(x1, y1, x2, y2) {
  doc.strokeColor(blue).lineWidth(2).moveTo(x1, y1).lineTo(x2, y2).stroke();
  doc.fillColor(blue).polygon([x2, y2], [x2 - 8, y2 - 5], [x2 - 8, y2 + 5]).fill();
}

// Cover and executive summary.
doc.rect(0, 0, pageWidth, 595).fillColor('#f4f8fa').fill();
doc.rect(0, 0, 18, 595).fillColor(teal).fill();
doc.font(boldFont).fontSize(30).fillColor(navy).text('技術スタック分析レポート', 60, 90);
doc.font(regularFont).fontSize(15).fillColor(blue).text('AI Chat Ollaemon | 会社向け詳細版', 62, 145);
doc.moveTo(62, 185).lineTo(420, 185).strokeColor(teal).lineWidth(3).stroke();
doc.font(regularFont).fontSize(12).fillColor(ink).text('Next.js と Ollama を組み合わせた¥nローカル AI チャットアプリの構成資料', 62, 220, { lineGap: 7 });
doc.roundedRect(540, 92, 220, 190, 12).fillColor('#ffffff').fill().strokeColor('#d9e2ec').stroke();
doc.font(boldFont).fontSize(13).fillColor(navy).text('エグゼクティブサマリー', 560, 118);
doc.font(regularFont).fontSize(10).fillColor(ink).text('・ ローカル LLM による AI 応答¥n・ トピック単位の履歴管理¥n・ モデル選択と会話の継続¥n・ 外部 API キー不要¥n・ 段階的な機能拡張が可能', 560, 155, { lineGap: 8 });
doc.font(regularFont).fontSize(9).fillColor(muted).text('作成日: 2026年9月4日', 62, 500);
footer(1);

// Table page.
doc.addPage();
heading('技術スタック一覧', 20);
doc.font(regularFont).fontSize(10.5).fillColor(muted).text('主要な技術、実装箇所、採用理由を一覧化しています。');
doc.moveDown(0.7);
drawTable(tableRows);
footer(2);

// Architecture diagram page.
doc.addPage();
heading('アーキテクチャ図', 20);
doc.font(regularFont).fontSize(10.5).fillColor(muted).text('質問が入力されてから、ローカル LLM の回答が画面へ戻るまでの処理フロー。');
doc.moveDown(1);

drawBox(55, 180, 145, 90, 'ユーザー', '質問入力¥nモデル選択', teal);
drawBox(250, 180, 165, 90, 'Next.js UI', 'app/page.tsx¥n履歴・状態管理', blue);
drawBox(465, 180, 145, 90, 'API Route', 'POST /api/chat¥nJSON 変換', navy);
drawBox(660, 180, 125, 90, 'Ollama', 'ローカル推論¥nGemma モデル', '#e76f51');
arrow(200, 225, 250, 225);
arrow(415, 225, 465, 225);
arrow(610, 225, 660, 225);
doc.font(regularFont).fontSize(9).fillColor(muted).text('質問 + 選択モデル', 203, 205);
```

目次へ

generate-tech-stack-pdf.js（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
doc.text('messages 配列', 418, 205);
doc.text('ローカル HTTP', 613, 205);

doc.roundedRect(250, 335, 360, 100, 10).fillColor(pale).fill().strokeColor('#b8d8df').stroke();
doc.font(boldFont).fontSize(13).fillColor(navy).text('ブラウザ内の永続化', 270, 355);
doc.font(regularFont).fontSize(10).fillColor(ink).text('chatHistory: Record<string, Message[]>¥nlocalStorage: ai-chat-history¥nトピック名をキーに会話を再開', 270, 385, { lineGap: 5 });
arrow(332, 270, 332, 335);
arrow(528, 335, 528, 270);
footer(3);

// Detail pages.
doc.addPage();
heading('詳細分析', 20);
sections.forEach(([title, body]) => {
  doc.font(boldFont).fontSize(13).fillColor(blue).text(title);
  doc.moveDown(0.25);
  doc.font(regularFont).fontSize(10.5).fillColor(ink).text(body, { width: contentWidth, align: 'justify', lineGap: 4 });
  doc.moveDown(1);
});
heading('主なファイル構成', 15);
doc.font(regularFont).fontSize(10).fillColor(ink).text('app/page.tsx　　メインチャット UI、履歴管理、モデル選択¥napp/api/chat/route.ts　　Ollama への API プロキシ¥napp/layout.tsx　　アプリ全体のレイアウト¥napp/globals.css　　グローバルスタイル¥npackage.json　　依存関係と実行スクリプト', { lineGap: 6 });
footer(4);

// Final page.
doc.addPage();
heading('まとめと今後の拡張候補', 20);
doc.font(regularFont).fontSize(11).fillColor(ink).text('現時点の構成は、ローカル AI チャットを短い導入期間で検証するために適している。フロントエンド、API、LLM 実行基盤の責務が分かれているため、将来の本番運用にも段階的に移行できる。', { width: contentWidth, align: 'justify', lineGap: 5 });
doc.moveDown(1.2);
[
  ['短期', 'ストリーミング応答、入力バリデーション、エラー表示の改善'],
  ['中期', 'SQLite / PostgreSQL などへの履歴移行、ユーザー認証、利用ログ'],
  ['長期', '権限管理、チーム共有、監査ログ、モデル管理画面'],
].forEach(([term, text]) => {
  doc.font(boldFont).fontSize(12).fillColor(teal).text(term, { continued: true });
  doc.font(regularFont).fontSize(11).fillColor(ink).text(`  ${text}`);
  doc.moveDown(0.7);
});
doc.moveDown(2);
doc.roundedRect(42, 390, contentWidth, 70, 10).fillColor(navy).fill();
doc.font(boldFont).fontSize(14).fillColor('#ffffff').text('総括', 62, 410);
doc.font(regularFont).fontSize(10.5).fillColor('#ffffff').text('Next.js、React、TypeScript、Tailwind CSS、Ollama を組み合わせた、拡張性のあるローカル AI チャット基盤です。', 115, 411, { width: 650 });
footer(5);
```

目次へ

generate-tech-stack-pdf.js（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
176
177 doc.end();
178 console.log(`PDF generated: ${outputPath}`);
```

目次へ

ファイル解説: README.md

利用者向けsetup/feature案内。

1〜5行 | 概要
冒頭はrepository名とアプリの目的を示し、Ollamaを使うローカルLLM chatおよびcoding assistantであることを説明します。実装ではなく、初見の利用者が用途を把握する入口です。

7〜26行 | 起動手順
起動前にOllama serverとmodel pullが必要なこと、npm installで依存を復元すること、npm run devでNext serverを起動しlocalhost:3000を開く順を説明します。local runtimeへ接続する前提も示します。

28〜46行 | chat/workspace機能
通常chatとcoding agentの機能を利用者向けに列挙し、文書/image添付、複数file抽出workspace、editor、copy/download、sandbox preview、ZIP、Diff、6種QuickActionを紹介します。実装先を知らなくても機能範囲を確認できます。

48〜68行 | local source参照
ローカルfile参照の二方式を説明します。質問にWindows path/@pathを書けばAPIがcontext化し、Explorer方式ならpreset/pathでdirectoryを閲覧してfileをpreview、attachまたは質問できます。回答の参照file情報も案内します。

69〜71行 | section終端
最後の水平線で機能紹介の章を閉じます。Markdownの区切りは各機能を視覚的に整理するもので、runtime codeや設定を追加しません。

README.md

全70行｜行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
# Olla衛門（ai-chat-ollaemon-app-localllm）

ローカルLLM（Ollama）を活用したセキュアなAIチャット & コーディングエージェントアプリケーションです。

---

## □起動方法

### 1. 前提条件（Ollama の準備）

ローカルマシンで Ollama が起動している必要があります（デフォルト: `http://localhost:11434`）。
利用するモデル（例: `gemma4:e4b`、`qwen3.5:9b` など）をあらかじめ pull してください。

```bash
ollama pull gemma4:e4b
```

2. 依存関係のインストール（初回のみ）

```bash
npm install
```

3. アプリ起動

```bash
npm run dev
```

ブラウザで以下にアクセスします：
<http://localhost:3000/>

□機能紹介

1. □通常チャットモード（Olla衛門）

- 日常的な質問、IT技術相談、文書要約などに対応します。
- 添付ファイル対応（テキスト、ソースコード、PDF、Word、Excel、PowerPoint、画像）。

2. □コーディングエージェント機能（Olla衛門 技師モード）

ヘッダーの「技師エージェント」タブ、またはサイドバーの「□技師エージェント」から利用できます。

- **コードワークスペース（2ペイン表示）**:
  AIが生成したコードブロックから自動的に複数ファイルを抽出・マッピングし、右ペインのワークスペースにタブ一覧表示します。
- **行番号・シンタックス・コピー・ダウンロード**:
```

目次へ

README.md（コード続き）

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
   ファイルの個別ダウンロードやワンクリックコピー、ブラウザ上での直接編集に対応。
- **□Webライブプレビュー（Sandbox）**:
   HTML/CSS/JavaScriptコードを安全なサンドボックス内で即時レンダリング・実行確認できます。
- **□ZIP一括保存（JSZip）**:
   生成された複数ファイルをディレクトリ構造を維持したまま、1つのZIPファイルとして一括保存可能。
- **□Diff差分比較ビューア**:
   修正前後のコードを行単位でビジュアル比較できます。
- **□クイックアクションパネル**:
   「新規作成」「バグ修正」「リファクタリング」「単体テスト」「コードレビュー」「コード解説」をワンクリックで実行。

### 3. □ローカルソースコード直接閲覧・質問機能

ローカルマシン内の実際のソースコードを直接閲覧・参照して回答させることができます。

- **質問文内のパス自動検出（プランA）**:
   チャットの質問文にファイルパス（例: `C:¥Portfolio¥...¥route.ts` や `@app/layout.tsx`）を含めるだけで、サーバーが自動的にローカルファイルを読み取り、コンテキストに注入して回答します。
   AIの回答上部には、実際に参照したファイル一覧と行数のバッジが表示されます。
- **ローカルワークスペース探査UI（プランB）**:
   画面上部または入力欄の「□ワークスペース」ボタンから、ローカルフォルダ（`C:¥Portfolio` 等）のツリーを直接ブラウズ可能。
   ファイルを選択してコードプレビューを確認し、「□チャットに添付」「□質問する」ボタンから1クリックでチャットの質問対象に指定できます。

---
```

ファイル解説: AGENTS.md

Next.js変更時のagent指示。

1〜10行｜version docs確認
Next.jsのversionが新しく破壊的変更の可能性があるため、変更前にnode_modules内の該当guideを読むよう指示します。agent instructionがdevで再作成される点とgenerate-agent-files.js確認の注意も含み、古い知識の無条件適用を防ぐ開発ルールです。

AGENTS.md

全9行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
<!-- BEGIN:nextjs-agent-rules -->

# This is NOT the Next.js you know

This version has breaking changes — APIs, conventions, and file structure may all differ from your training data. Read the relevant guide in `node_modules/next/dist/docs/` (resolved from this file's directory; in monorepos the `next` package may not be visible from the repo root) before writing any code. Heed deprecation notices.

This block is written and re-added by `next dev` — verify at `node_modules/next/dist/server/lib/generate-agent-files.js`. Removing it from a diff only re-creates the uncommitted change; committing it with your work keeps the tree clean.

<!-- END:nextjs-agent-rules -->
```

目次へ

ファイル解説: CLAUDE.md

共通instructionへの案内。

1〜2行｜AGENTS.md参照
このfileに個別の実装手順を書かずrootのAGENTS.mdを参照するよう案内します。複数agent環境で同じNext.js変更ルールを重複管理しないための委譲用instructionです。

CLAUDE.md

全1行 | 行番号は下記の範囲解説と対応

	コードブロック	原文コード（長い行は紙面内で折り返し）
1	@AGENTS.md	

目次へ

ファイル解説: LICENSE

MIT利用条件。

1〜7行｜license grant

MIT Licenseはcopyrightと許諾条件のもと、利用・複製・変更・結合・公開・配布・ sublicense・販売を許可します。source処理ではなく、利用者や再配布者へ与える法的な利用範囲を示します。

8〜14行｜notice

再配布時にはcopyright noticeと本許諾文を全copyまたは相当部分に含める条件です。許可範囲は広い一方、元の権利表示を保持する義務を定めています。

15〜22行｜免責

softwareを現状有姿で提供し明示・黙示の保証を否認するとともに、使用や損害に起因する責任を制限します。codeの動作や例外処理ではなく、権利者の法的責任を定める条項です。

LICENSE

全21行 | 行番号は下記の範囲解説と対応

コードブロック　　原文コード（長い行は紙面内で折り返し）

```
MIT License

Copyright (c) 2026 ikato0000001-work

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

目次へ

対象外ファイル

個別SVGソースコード

SVGの各ファイル本文は掲載せず、SVG形式そのものを説明する章にまとめています。

package-lock.json

依存関係の再現に使う機械生成台帳であり、プロジェクト独自ロジックではないため省略しています。

画像・favicon・既存PDF/ODG

バイナリーまたは既存成果物であり、テキストコードとして掲載できません。

node_modules、.next、.git、型検査キャッシュ

依存物や生成物であり、作成したソースコードの解説対象から除外しています。

まとめ

各ファイルの解説では、関数・状態管理・API処理・UI領域などの意味あるまとまりごとに、原文の行番号を示しています。まず該当範囲の説明を読み、その後、同じ行番号が付いたコードblockを確認すると、処理の目的と実装を対応付けられます。

コードの行番号は資料作成時点のソースに対応します。リポジトリ側で編集が行われた場合は、現在のファイルの内容を優先してください。

この資料はコード解説であり、アプリの実行試験やセキュリティ監査の報告書ではありません。